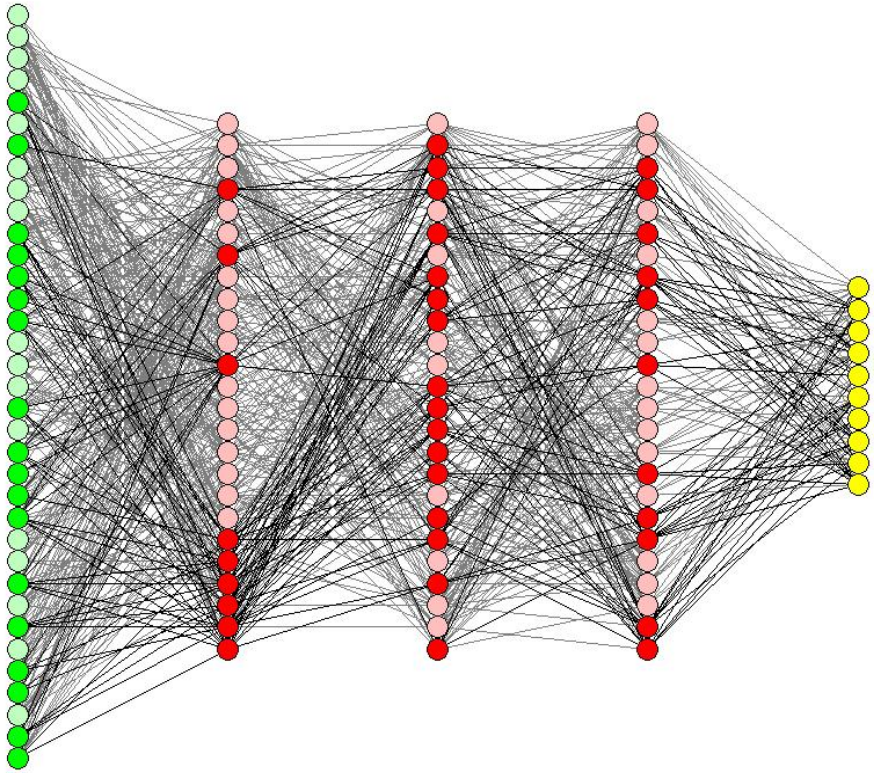
神经网络与深度学习



深度学习

1.1 人工神经网络

1.2 深度学习原理

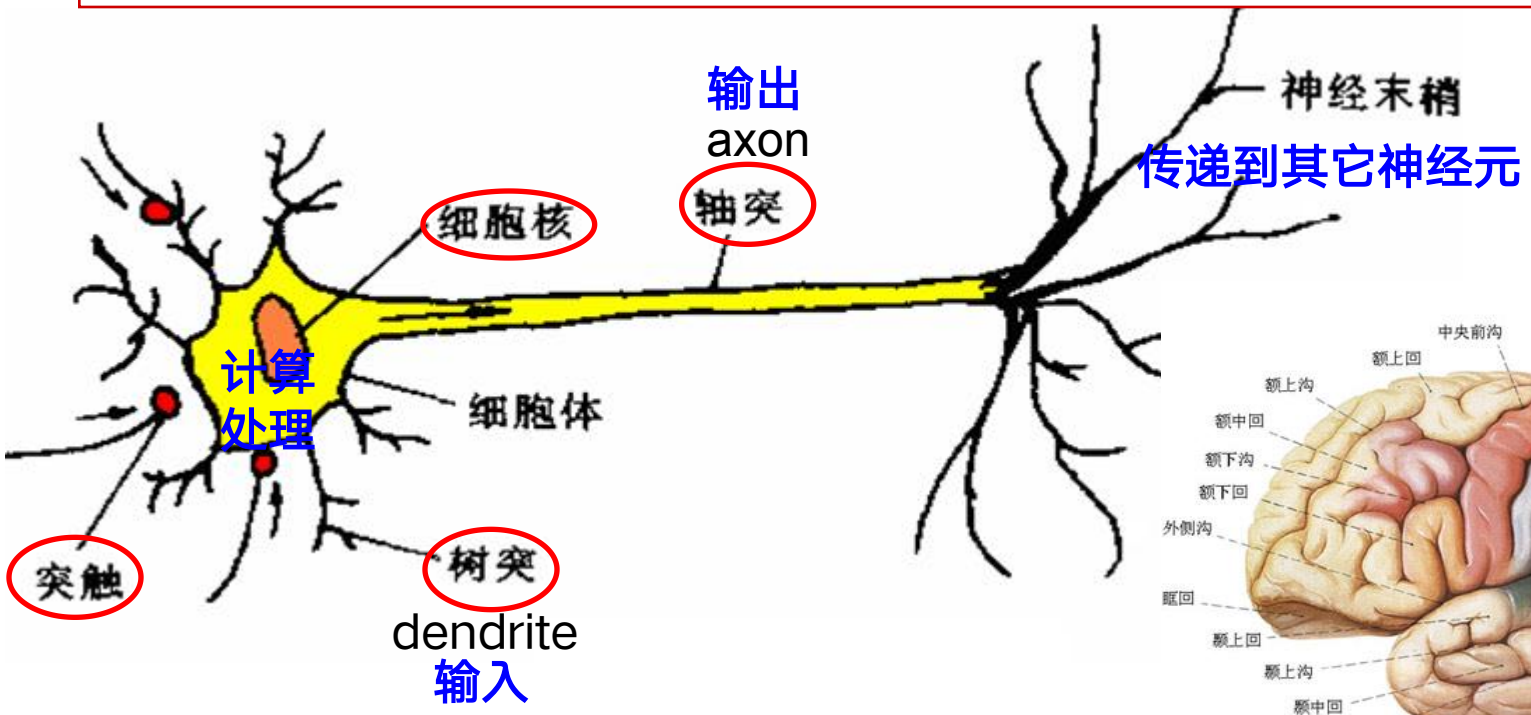
1.3 深度学习模型

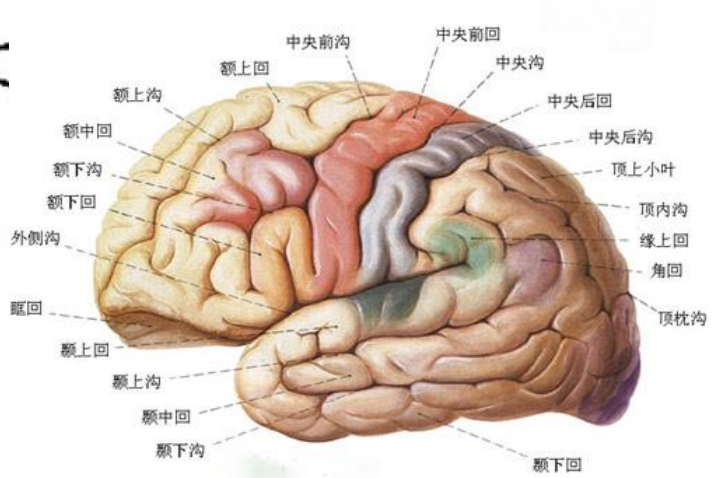
1.4 深度学习应用

1.1 人工神经网络

■ 生物神经元结构

- ✓ 神经元是大脑计算处理信息的基本单元。
- ✓ 人脑约由数百亿个神经元组成，神经元之间通过突触联接，形成复杂灵活的神经网络。
- ✓ 单个神经元只有两种状态：兴奋和抑制

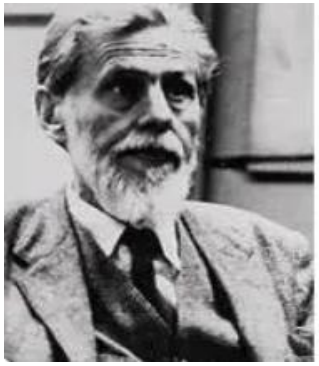




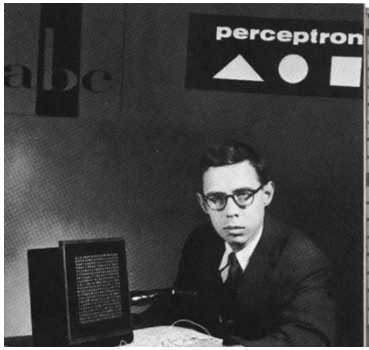
1.1 人工神经网络

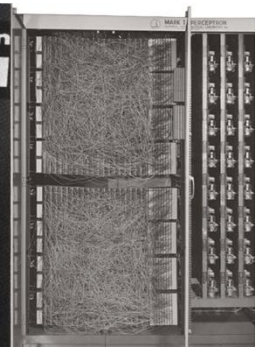
■ 人工神经网络（ANN）发展

- 1943年，心理学家McCulloch和数学家Pitts首次提出了形式神经元的数学模型，简称**M-P模型**。
- 1957年，F.Rosenblatt提出“**感知器”(Perceptron) 模型**，第一次把神经网络的研究从纯理论的探讨付诸工程实践。
- 1986年，Rumelhart和Mcclelland提出多层网络的**误差反传**(back propagation)**学习算法，简称BP算法**。BP算法是目前最为重要、应用最广的人工神经网络算法之一。







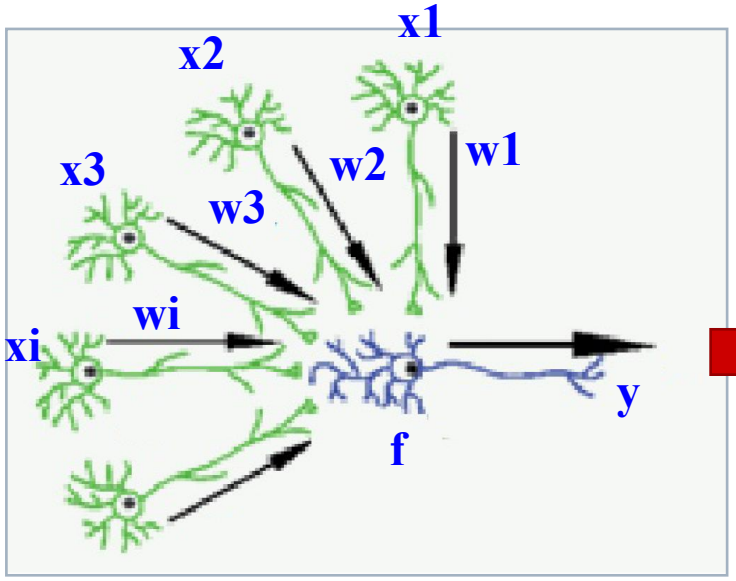


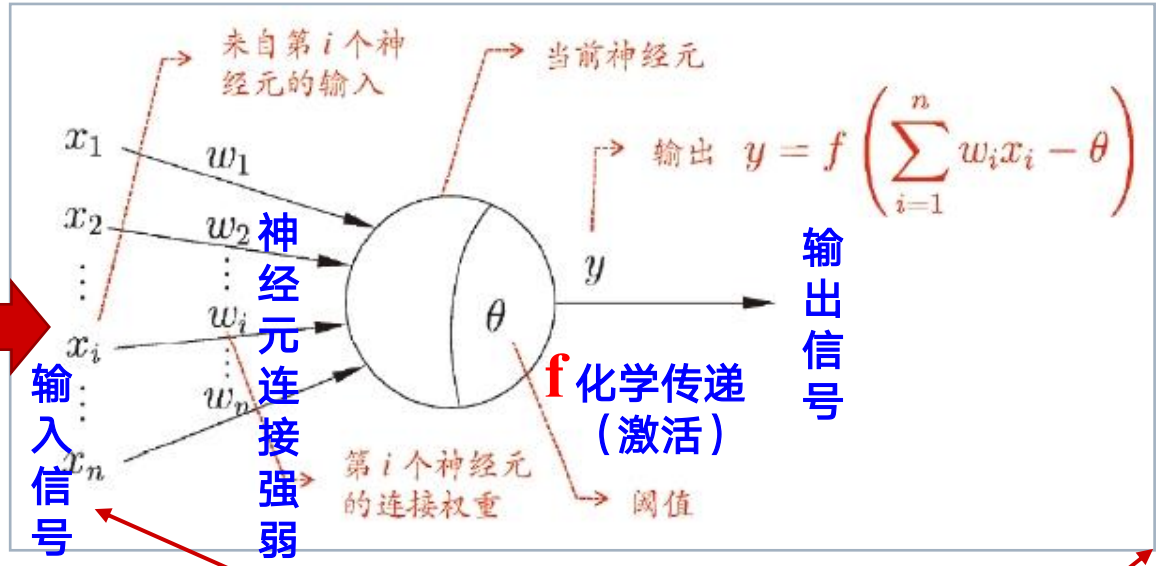


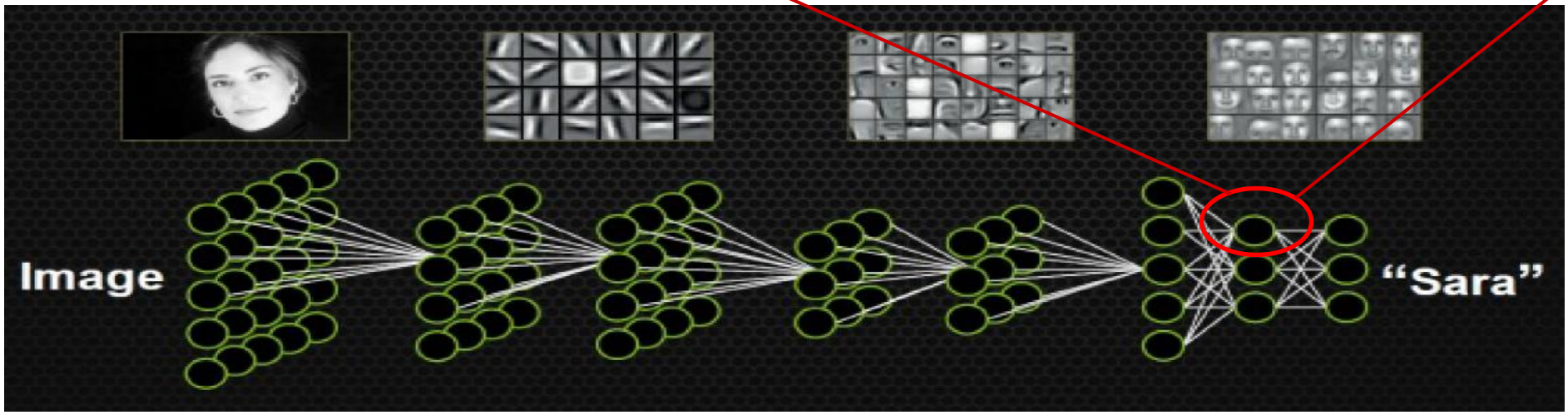


1.1 人工神经网络

- MP（McCulloch-Pitts ）模型　　生物神经元》人工神经元







1.1 人工神经网络

■ 感知器Perceptron模型

- 基于M-P模型的一个应用
- 感知器引入了学习规则，即通过监督学习的方式完成逐渐调整其权重以正确地预测输出

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b) = \begin{cases} +1, & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0 \\ -1, & \text{otherwise} \end{cases}$$

■ 权重更新规则：

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot y \cdot \mathbf{x}$$

η 是学习率，通常取 0<η≤1；y 是真实标签（±1），x 是输入样本

- 如果真实标签是 $y = +1$，但模型预测为 -1（即 $\mathbf{w} \cdot \mathbf{x} < 0$），说明权重方向“远离”了正样本。
  → 加上 $+1 \cdot \mathbf{x}$，让权重向该正样本方向靠拢。
- 如果真实标签是 $y = -1$，但模型预测为 +1（即 $\mathbf{w} \cdot \mathbf{x} \geq 0$），说明权重“误把负样本当正样本”。
  → 加上 $-1 \cdot \mathbf{x}$，即减去 $\mathbf{x}$，让权重远离该负样本。

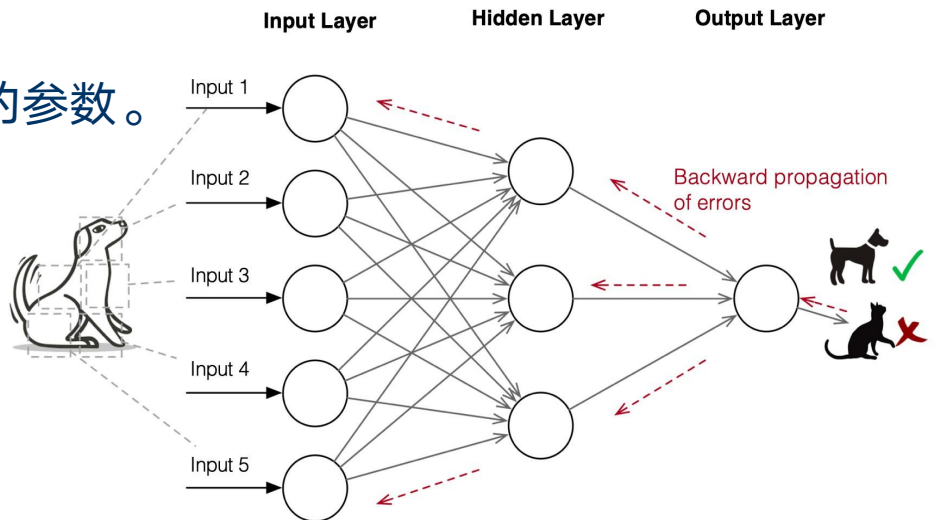
1.1 人工神经网络

■ ANN设计

人工神经网络（Artificial Neural Network）是一种受生物神经系统启发的计算模型，由大量简单处理单元（人工神经元） 通过连接权重相互连接而成。

核心设计要素

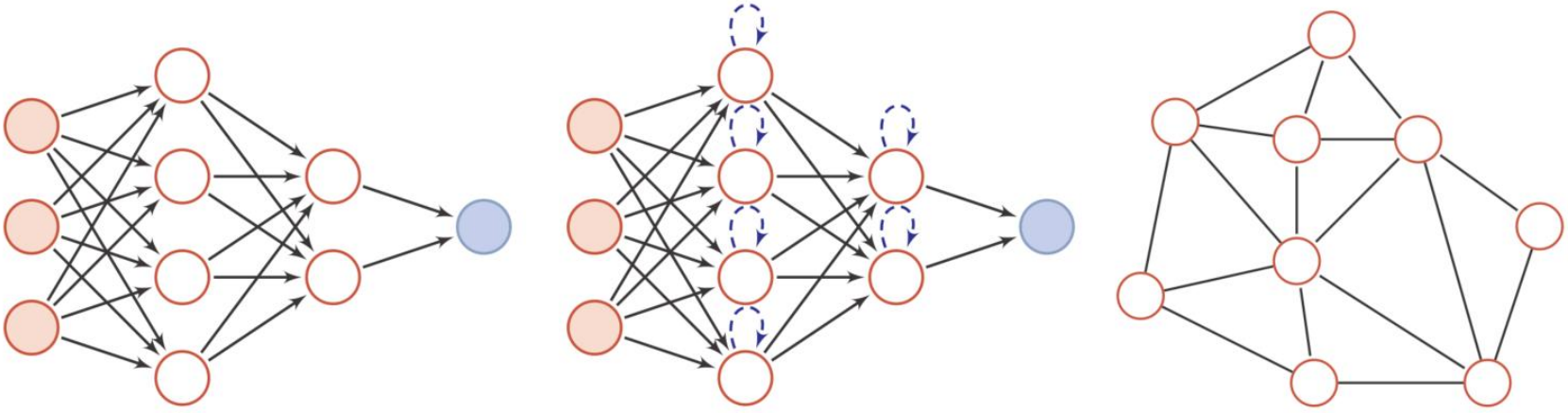
- ✓ **拓扑结构**
  - ■ 不同神经元之间的连接关系设计。
- ✓ **激活规则**
  - ■ 神经元输入到输出之间的映射关系设计。
- ✓ **学习算法**
  - ■ 数据驱动来学习神经网络的参数。



1.1 人工神经网络

- **ANN设计：拓扑结构**
  - **经典结构设计：前向神经网络**

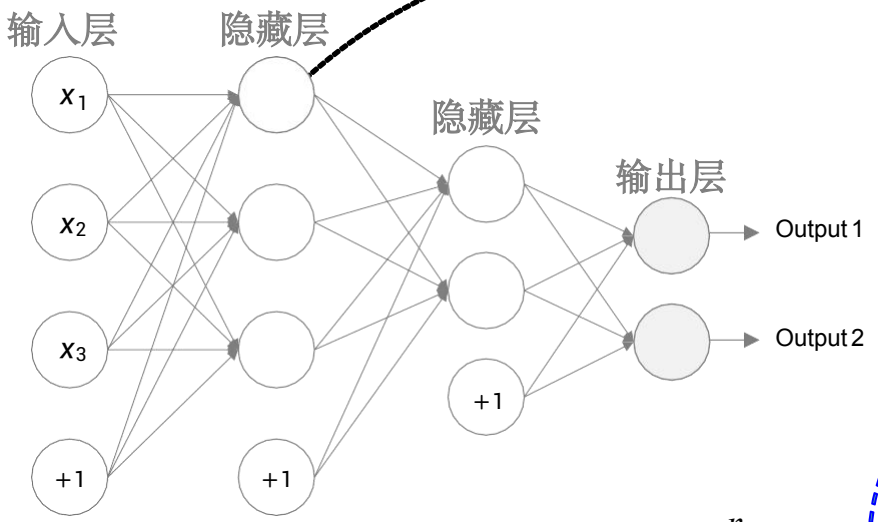
    神经元分层排列，顺序连接。由输入层施加输入信息，通过中间各层，加权后传递到输出层后输出。每层的神经元只接受前一层神经元的输入，各神经元之间不存在反馈。
  - **其它拓扑结构：反馈、循环、图网络…**



(a) 前馈网络　(b) 反馈网络　(c) 图网络

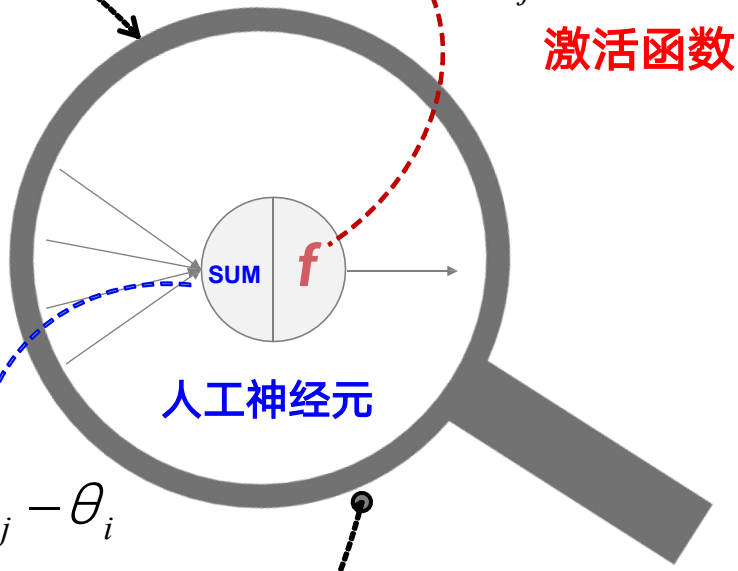
1.1 人工神经网络

- ANN设计：**激活规则**

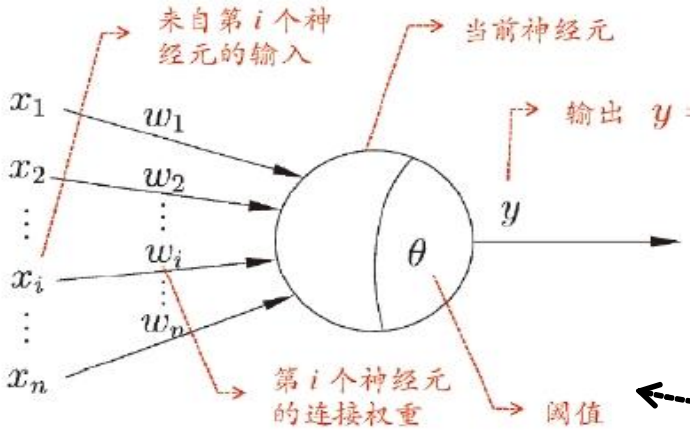


$$y_i = f(net_i) = f(\sum_{j=1}^{n} w_{ji}\, x_j - \theta_i)$$

激活函数



$$net_i = \sum_{j=1}^{n} w_{ji}\, x_j - \theta_i$$



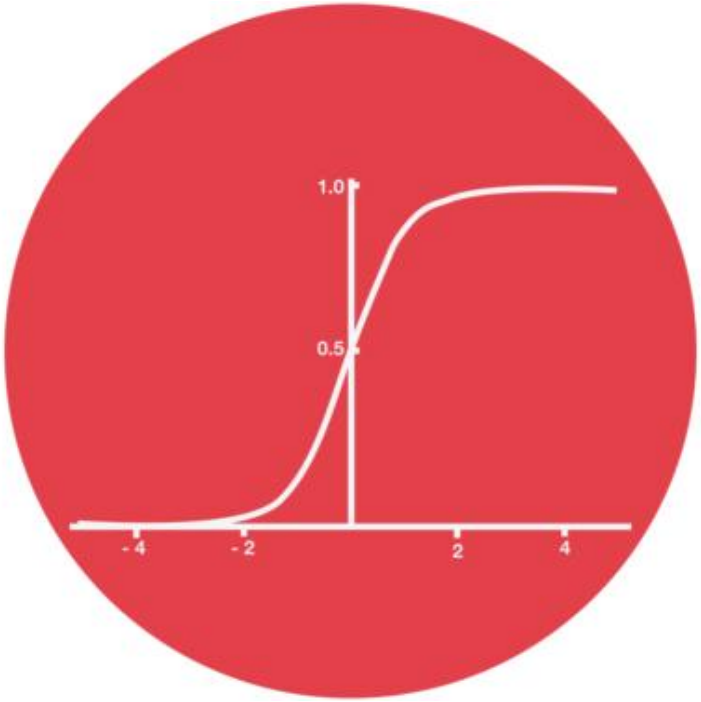
McCulloch-Pitts

MP模型

1.1 人工神经网络

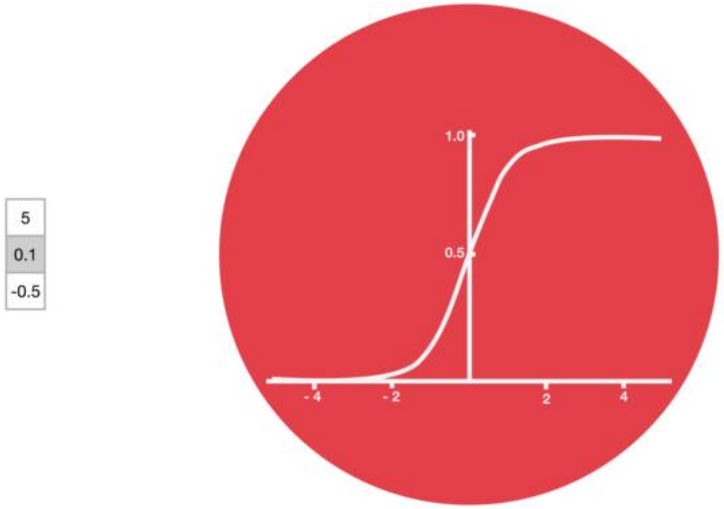
- ANN设计：激活规则　　为什么需要激活？
  - ✓ 控制输入对输出的激活作用
  - ✓ 对输入、输出进行函数转换
  - ✓ 将可能无限域的输入变换成指定的有限范围内的输出

5
0.1
-0.5



1.1 人工神经网络

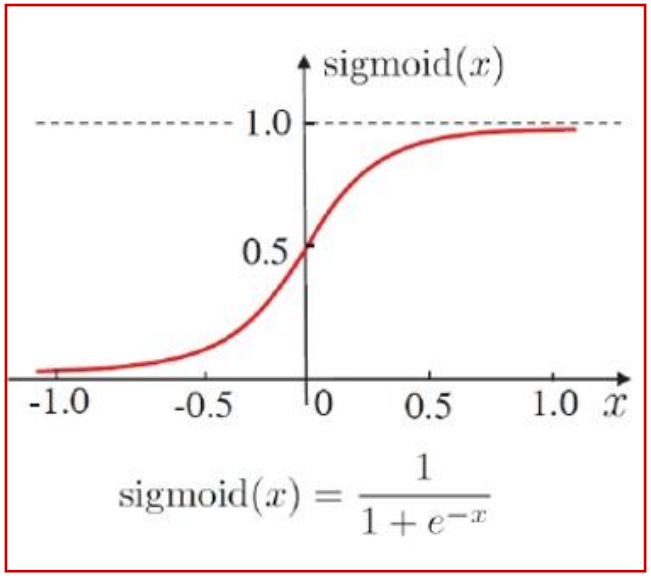
- **激活函数的性质** **为什么需要激活？**
  - ✓ 1、连续并可导（允许少数点上不可导）的非线性函数。
    可导的激活函数可以直接利用数值优化的方法来学习网络参数。
  - ✓ 2、激活函数及其导函数要尽可能的简单
    有利于提高网络计算效率。
  - ✓ 3、激活函数的导函数的值域要在一个合适的区间内
    不能太大也不能太小，否则会影响训练的效率和稳定性。



1.1 人工神经网络

- ANN设计：**激活规则**　　常见激活函数

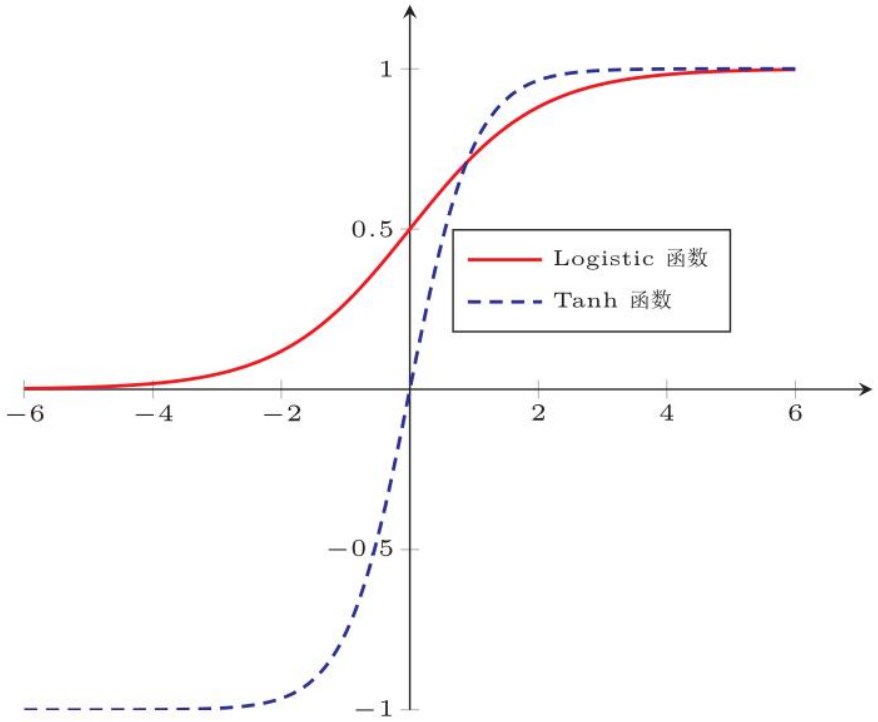
激活函数	函数公式	输出范围	主要性质	优势	局限性
Logistic (Sigmoid)	$\sigma(x) = \frac{1}{1+e^{-x}}$	(0, 1)	连续、可微；导数: $\sigma(x)(1-\sigma(x))$；非线性映射	输出概率值，适用于二分类；	梯度消失问题；输出非零中心可能导致后续层梯度更新不平衡



1.1 人工神经网络

- ANN设计：**激活规则**　　常见激活函数

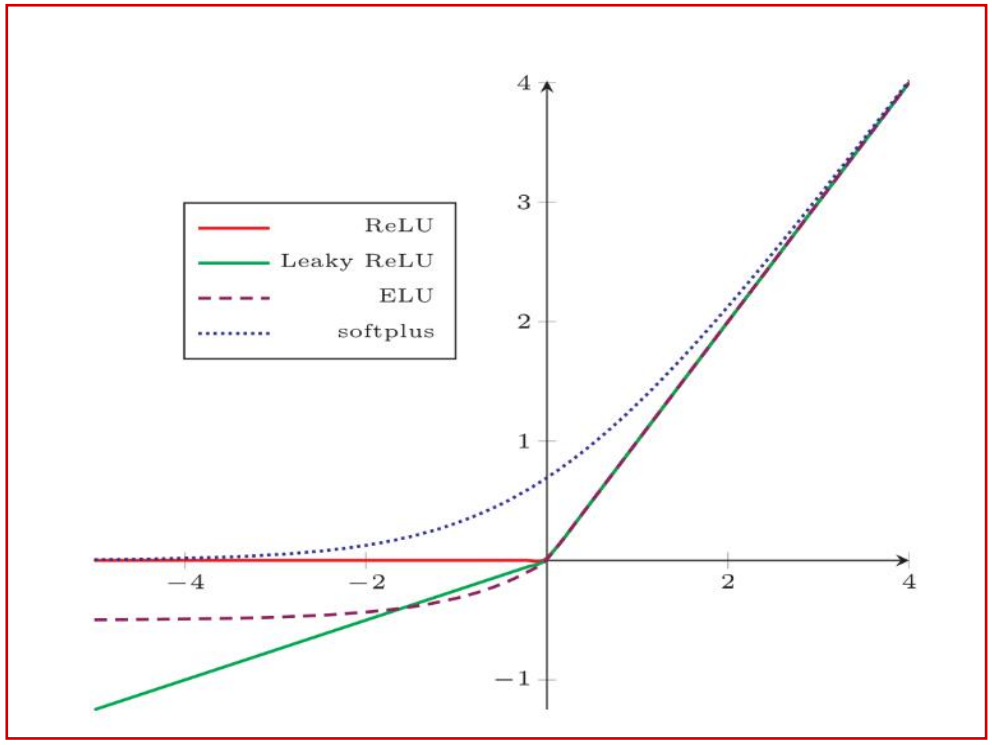
激活函数	函数公式	输出范围	主要性质	优势	局限性
Tanh	$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	(-1, 1)	连续、可微；导数:$1 - \tanh^2(x)$)；输出零中心化	零中心化有助于加快收敛；非线性强，能捕捉复杂关系	在输入较大时同样容易饱和，导致梯度消失



1.1 人工神经网络

- ANN设计：**激活规则**　　常见激活函数

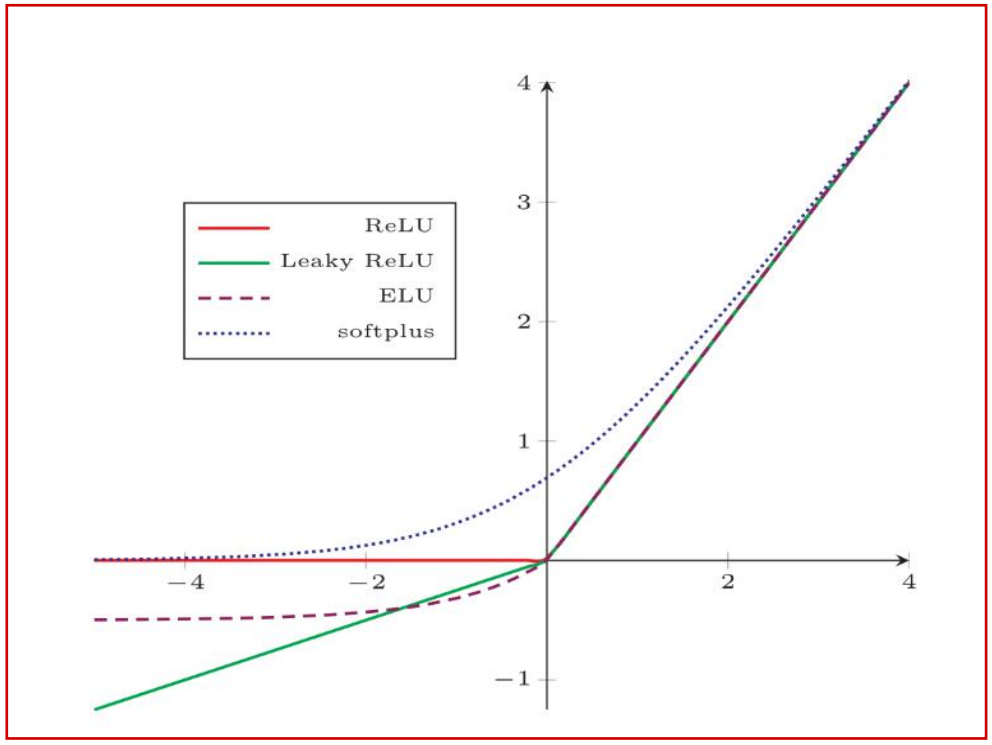
激活函数	函数公式	输出范围	主要性质	优势	局限性
ReLU	$ReLU(x) = max(0, x)$	[0, +∞)	分段线性：$x > 0$时为线性，$x \leq 0$时输出0；	正半轴梯度恒定，缓解梯度消失；引入稀疏激活，降低计算复杂度	负区域梯度为0，可能导致“死神经元”问题



1.1 人工神经网络

- ANN设计：**激活规则**　　常见激活函数

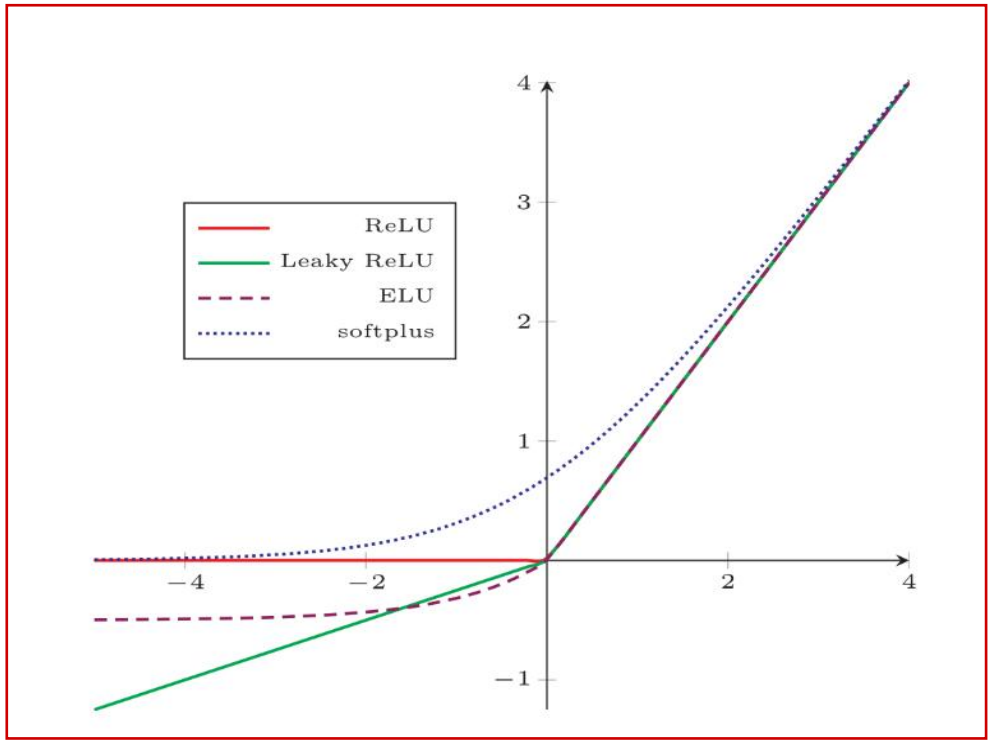
激活函数	函数公式	输出范围	主要性质	优势	局限性
ELU	$ELU(x) = \begin{cases} x, x \geq \\ \alpha(e^x - 1), \end{cases}$	(-α, +∞)	在x=0连续可微；负区输出非零，有助于输出均衡	输出近似零均值，加速收敛；负区有非零梯度，避免神经元完全失活	相较于ReLU，多了指数计算，计算开销较高



1.1 人工神经网络

- ANN设计：**激活规则**　　常见激活函数

激活函数	函数公式	输出范围	主要性质	优势	局限性
SoftPlus	$SoftPlus(x) = log(1 + e^x)$	(0, +∞)	全局平滑、可微；为ReLU的平滑近似形式	平滑连续，梯度稳定；无死神经元问题	计算效率低于ReLU；梯度在极端输入下较小，可能影响学习速率

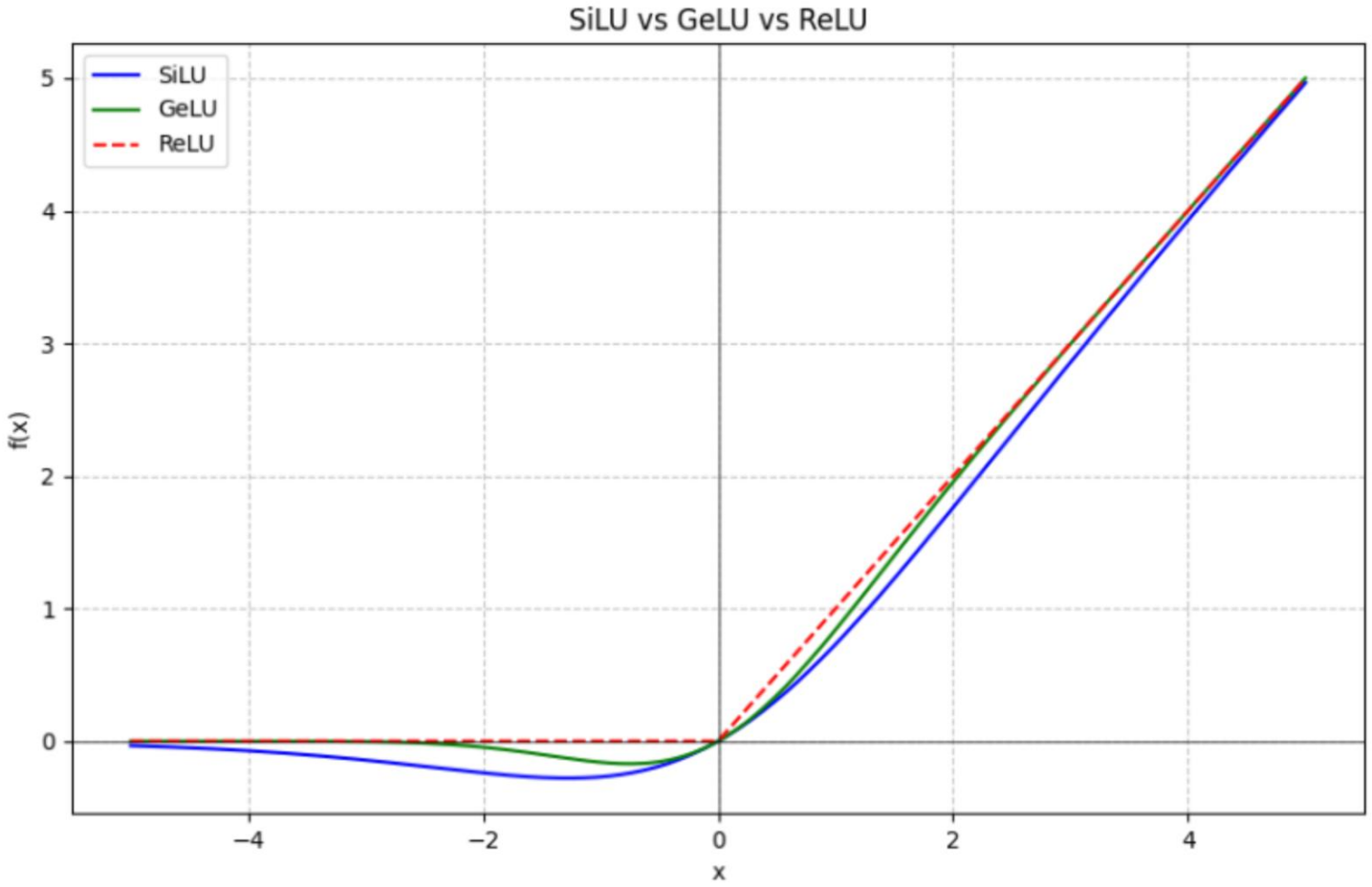


1.1 人工神经网络

激活函数	函数公式	输出范围	主要性质	优势	局限性
Logistic (Sigmoid)	$\sigma(x) = \frac{1}{1+e^{-x}}$	(0, 1)	连续、可微；导数: $\sigma(x)(1-\sigma(x))$；非线性映射	输出概率值，适用于二分类；	梯度消失问题；输出非零中心可能导致后续层梯度更新不平衡
Tanh	$tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	(-1, 1)	连续、可微；导数:$1-\tanh^{2}(x)$)；输出零中心化	零中心化有助于加快收敛；非线性强，能捕捉复杂关系	在输入较大时同样容易饱和，导致梯度消失
ReLU	$ReLU(x) = max(0, x)$	[0, +∞)	分段线性：$x > 0$时为线性，$x \leq 0$时输出0；	正半轴梯度恒定，缓解梯度消失；引入稀疏激活，降低计算复杂度	负区域梯度为0，可能导致“死神经元”问题
ELU	$ELU(x) = \begin{cases} x, x \geq \\ \alpha(e^{x}-1), \end{cases}$	(-α, +∞)	在x=0连续可微；负区输出非零，有助于输出均衡	输出近似零均值，加速收敛；负区有非零梯度，避免神经元完全失活	相较于ReLU，多了指数计算，计算开销较高
SoftPlus	$SoftPlus(x) = log(1+e^{x})$	(0, +∞)	全局平滑、可微；为ReLU的平滑近似形式	平滑连续，梯度稳定；无死神经元问题	计算效率低于ReLU；梯度在极端输入下较小，可能影响学习速率

1.1 人工神经网络

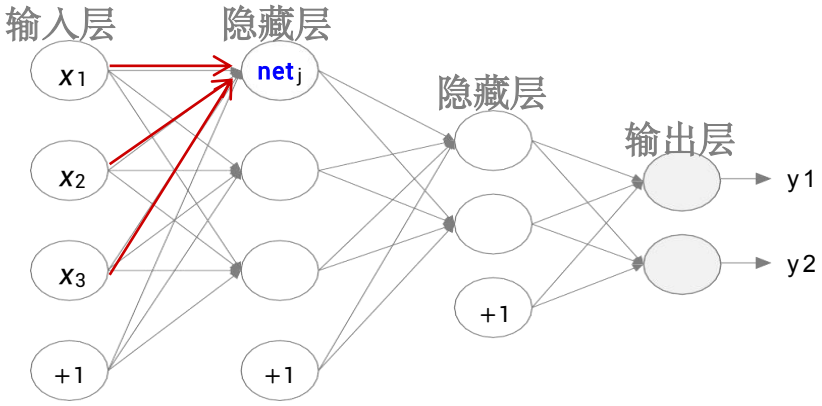
- 现今对激活函数的优化

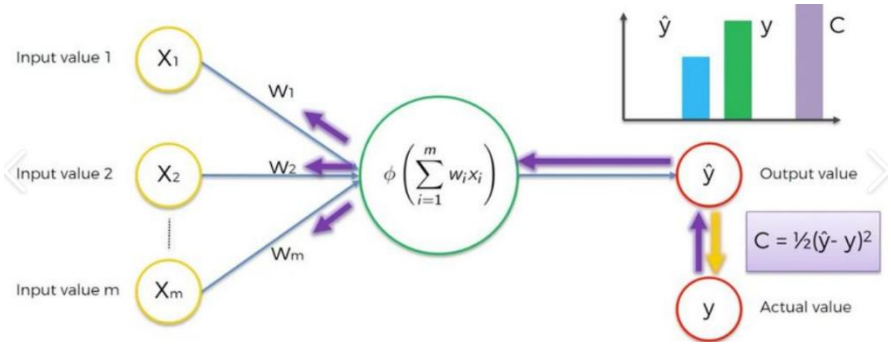


1.1 人工神经网络

- ANN设计：学习算法

学习是改变各神经元连接权值的有效方法，也是实现人工神经网络智能特性最主要的手段。ANN根据实际输出与期望输出的偏差，按照一定的准则调整各神经元连接的权系数，期望输出又称为导师信号，是评价学习的标准，这种学习方式又称为有监督（导师）学习。





特点：

- 不能保证得到全局最优解
- 要求大量训练样本，收敛速度慢
- 对样本表示次序变化比较敏感

1.1 人工神经网络

经典ANN:BP(Back Propagation)反向传播神经网络

1、权初始化；

2、**向前传播阶段：**

取样本(X_p, Y_p)输入网络；

计算相应的实际输出O_p：

$O_p=F_L(\cdots(F_2(F_1(X_pW^{(1)})W^{(2)})\cdots)W^{(L)})$

3、**向后传播阶段**——误差传播阶段：

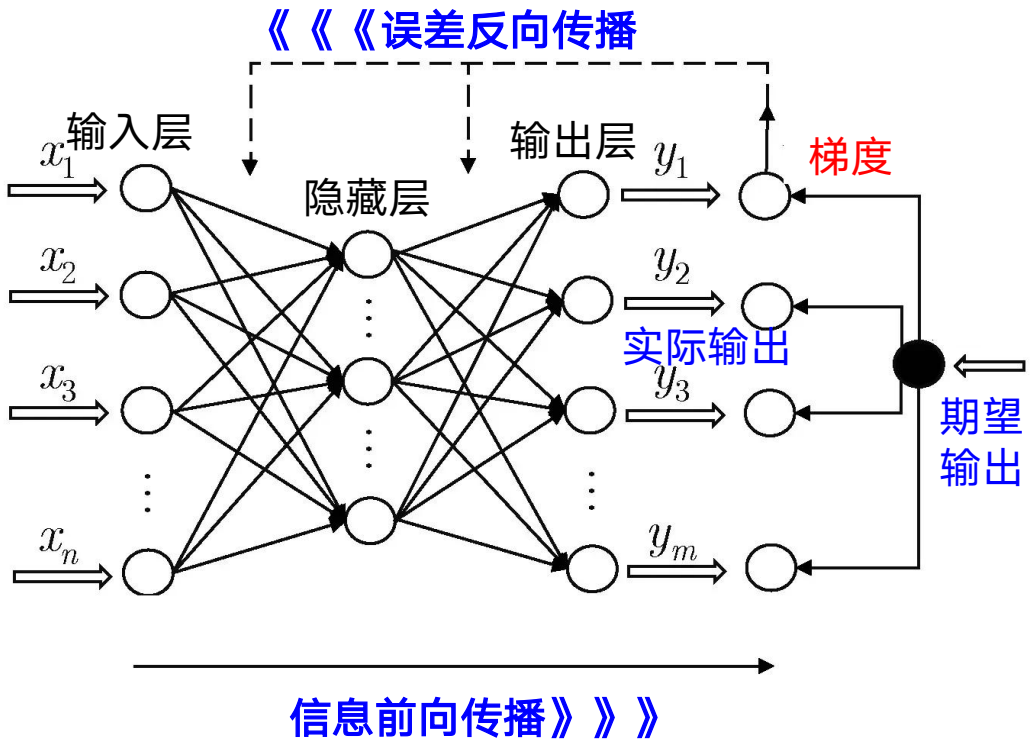
1）计算实际输出O_p与期望输出Y_p的差。

2）按反向传播算法调整权值矩阵。

3）累计网络关于整个样本集的误差。

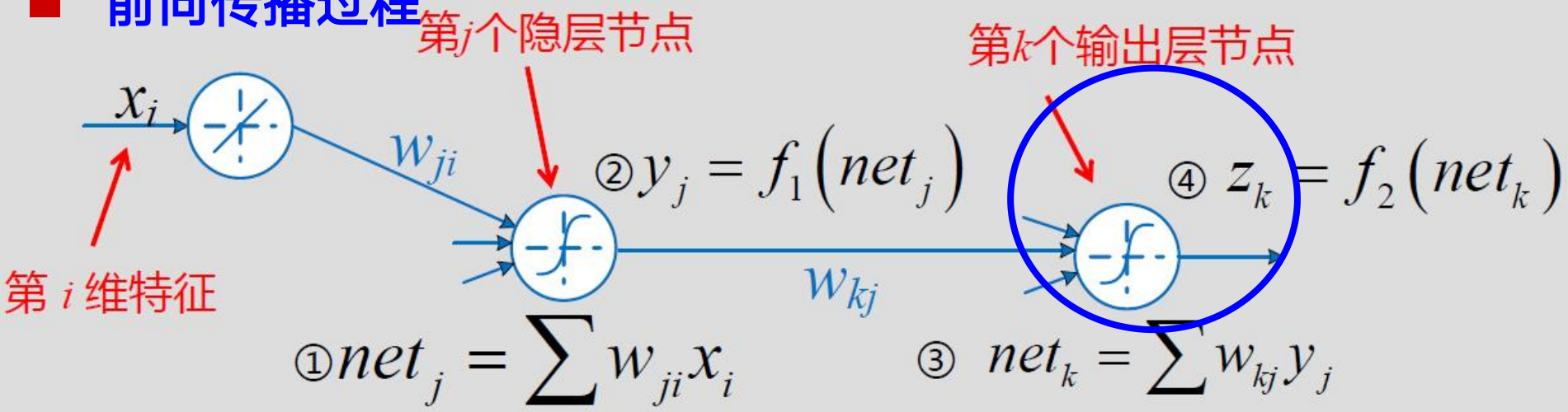
4、如果网络误差足够小，则停止训练。

否则重复第2、3步。



BP学习算法过程

- **前向传播过程**



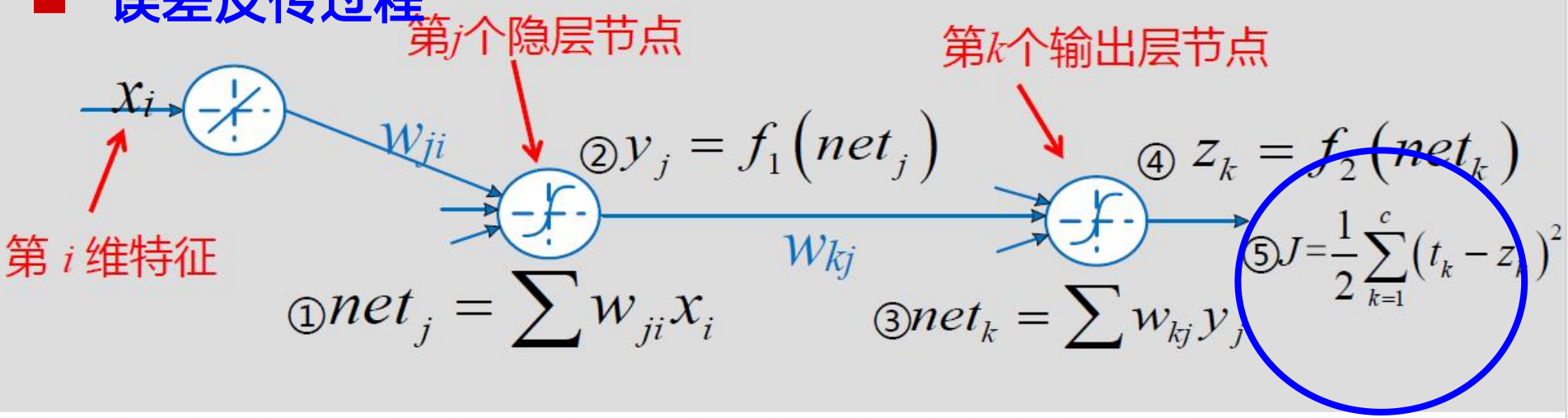
输出层第k节点输出：

共 n_H 个隐层节点　　共 d 维特征　　第j隐节点偏置　　第k输出节点偏置

$$z_k(\mathbf{x}) = f_2\left(\sum_{j=1}^{n_H} w_{kj} \underbrace{f_1\left(\sum_{i=1}^{d} w_{ji}x_i + w_{j0}\right)}_{y_j} + w_{k0}\right)$$

BP学习算法过程

- **误差反传过程**



BP算法：训练样本 **x**，期望输出 **t**，网络实际输出 **z**，基于误差最小化的梯度下降法，求解连接权值 **w**= $\{w_{ji}, w_{kj}\}$

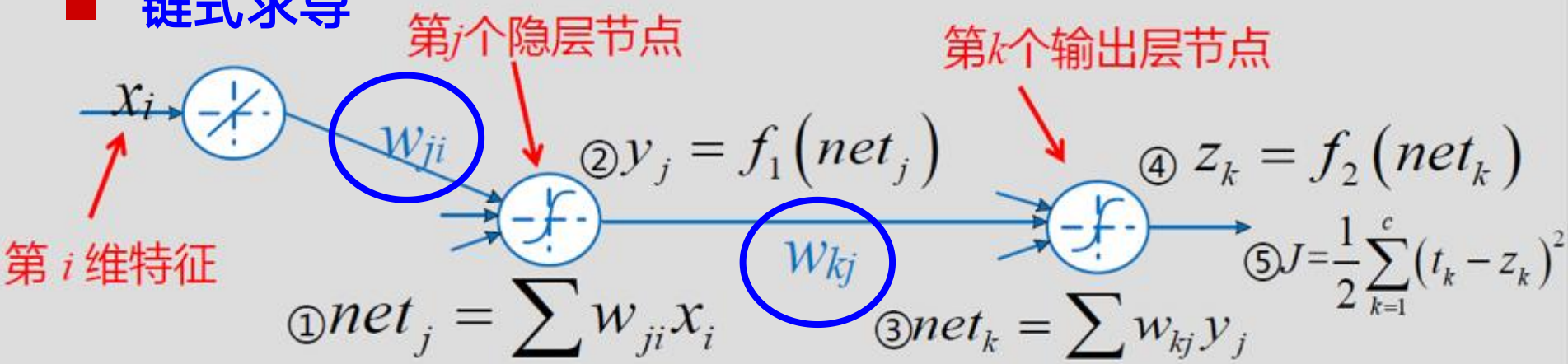
目标（误差）函数：$J(\mathbf{w}) = \frac{1}{2}\|\mathbf{t}-\mathbf{z}\|^2 = \frac{1}{2}\sum_{k=1}^{c}(t_k - z_k)^2$

如何求导？

梯度下降：$\mathbf{w}(m+1) = \mathbf{w}(m) + \Delta\mathbf{w}(m) = \mathbf{w}(m) - \eta\frac{\partial J}{\partial \mathbf{w}}$

BP学习算法过程

- **链式求导**



链式求导：计算误差函数对“隐含—输出”连接权值 w_{kj}，“输入—隐含层”连接权值 w_{ji} 的导数

$$\frac{\partial J}{\partial w_{kj}} = \frac{\partial J}{\partial z_k}\frac{\partial z_k}{\partial net_k}\frac{\partial net_k}{\partial w_{kj}}$$

输出层

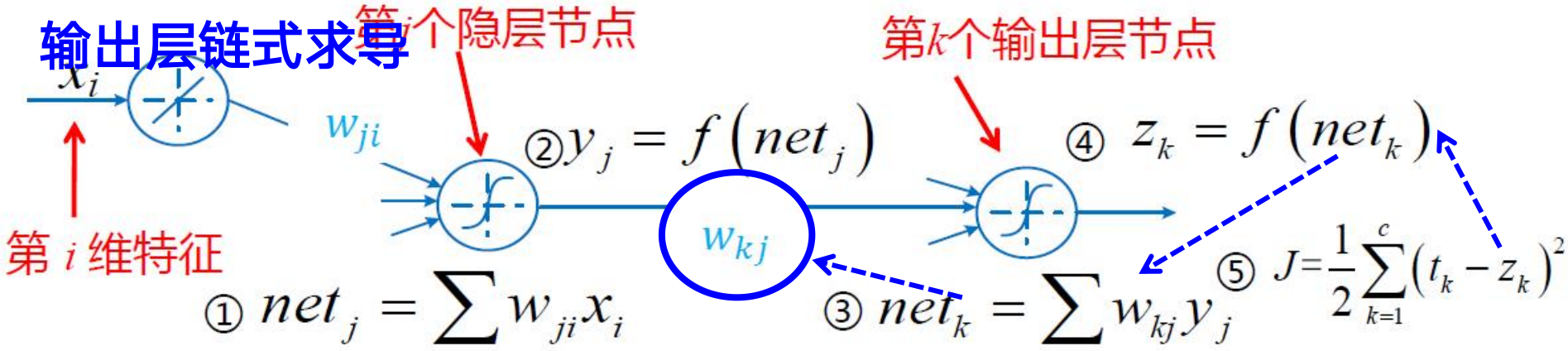
$$\frac{\partial J}{\partial w_{ji}} = \frac{\partial J}{\partial y_j}\frac{\partial y_j}{\partial net_j}\frac{\partial net_j}{\partial w_{ji}}$$

隐含层

——误差函数 J 沿着公式⑤-④-③-②-①反向传播

BP算法过程

输出层链式求导



输出层求导：

$$\frac{\partial J}{\partial w_{kj}} = \frac{\partial J}{\partial z_k}\frac{\partial z_k}{\partial net_k}\frac{\partial net_k}{\partial w_{kj}}$$

由⑤得：$\frac{\partial J}{\partial z_k} = -(t_k - z_k)$

由④得：$\frac{\partial z_k}{\partial net_k} = f'(net_k)$

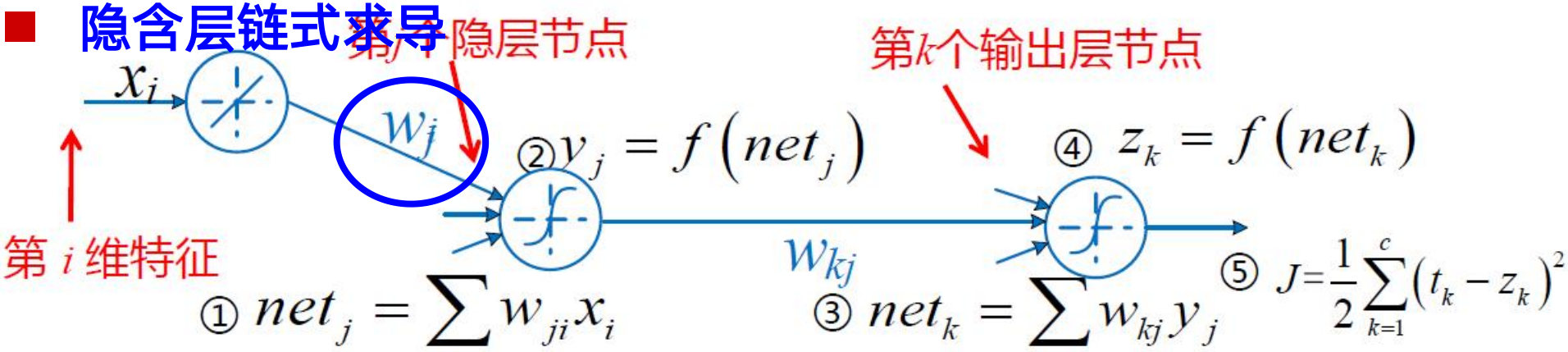
由③得：$\frac{\partial net_k}{\partial w_{kj}} = y_j$

$$\frac{\partial J}{\partial w_{kj}} = -(t_k - z_k) f'(net_k) y_j = -\delta_k y_j$$

令 $\delta_k = (t_k - z_k) f'(net_k)$

BP算法过程

- **隐含层链式求导**



隐含层

$$\frac{\partial J}{\partial w_{ji}} = \frac{\partial J}{\partial y_j}\frac{\partial y_j}{\partial net_j}\frac{\partial net_j}{\partial w_{ji}}$$

由①得: $\frac{\partial net_j}{\partial w_{ji}} = \frac{\partial}{\partial w_{ji}}\left(\sum_{m=1}^{d} w_{jm} x_m\right) = x_i$

由②得: $\frac{\partial y_j}{\partial net_j} = f'(net_j)$

⑤

$$\frac{\partial J}{\partial y_j} = \frac{\partial}{\partial y_j}\left[\frac{1}{2}\sum_{k=1}^{c}(t_k - z_k)^2\right] = -\sum_{k=1}^{c}(t_k - z_k)\frac{\partial z_k}{\partial y_j}$$

$$= -\sum_{k=1}^{c}(t_k - z_k)\frac{\partial z_k}{\partial net_k}\frac{\partial net_k}{\partial y_j} = -\sum_{k=1}^{c}\underbrace{(t_k - z_k) f'(net_k)}_{\delta_k} w_{kj} = -\sum_{k=1}^{c}\delta_k w_{kj}$$

$$\frac{\partial J}{\partial w_{ji}} = -\left[\sum_{k=1}^{c}\delta_k w_{kj}\right] f'(net_j) x_i = -\delta_j x_i$$

令 $\delta_j = \left(\sum_{k=1}^{c}\delta_k w_{kj}\right) f'(net_j)$

BP算法的缺点

1. 梯度消失（Vanishing Gradient）

- 现象：在深层网络中，反向传播时梯度逐层相乘，若激活函数导数小于 1（如 Sigmoid、Tanh），梯度会指数级衰减。
- 后果：靠近输入层的权重几乎不更新，深层网络难以训练。
- 典型场景：使用 Sigmoid 激活的 5 层以上网络，训练停滞。

2. 梯度爆炸（Exploding Gradient）

- 现象：某些权重过大时，梯度在反向传播中指数级增长。
- 后果：参数更新剧烈，损失发散，训练崩溃。
- 典型场景：RNN 中更常见，但在深层 MLP 中也可能出现。

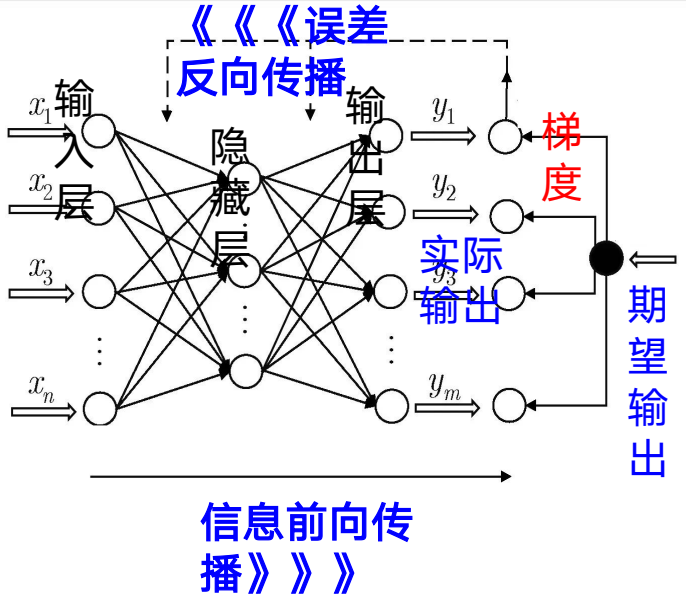
感知机与多层感知机(MLP)

基本定义对比

Multilayer Perceptron, MLP

模型	感知机（Perceptron）	多层感知机（MLP）
提出时间	1957年（Rosenblatt）	1960s–1980s 发展，1986年 BP 算法成熟
网络结构	**单层**：输入 → 输出（无隐藏层）	**多层**：输入层 + **一个或多个隐藏层** + 输出层
激活函数	阶跃函数（如 sign 函数）	非线性可微函数（如 Sigmoid、Tanh、ReLU）
学习方式	感知机学习规则（仅适用于线性可分）	反向传播（Backpropagation, BP）算法

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b) = \begin{cases} +1, & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0 \\ -1, & \text{otherwise} \end{cases}$$



感知机与多层感知机(MLP)

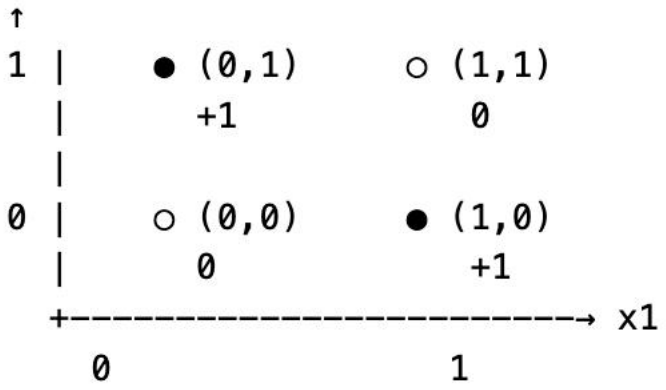
能力对比

能力	感知机	MLP
可分性	仅能解决**线性可分**问题（如 AND、OR）	可解决**线性不可分**问题（如 XOR、复杂图像分类）
决策边界	**直线（或超平面）**	**任意复杂曲面**（理论上可逼近任意连续函数）
泛化能力	弱，仅适用于简单模式	强，可通过增加层数/神经元拟合复杂模式

XOR（异或）运算真值表

输入 x_1	输入 x_2	输出 $y = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

- **正类（+1）**：(0,1) 和 (1,0)
- **负类（-1 或 0）**：(0,0) 和 (1,1)



线性不可分

1.1 人工神经网络

- **其它经典神经网络模型（略）**

✓ Hopfield神经网络
✓ 随机神经网络
✓ 自组织映射网络
✓ 玻尔兹曼机
✓ Kohonen网络
✓ 自适应共振网络
✓ 混沌网络
✓ …

名　　称	学习方式	拓扑结构	典 型 应 用
感知机(Perceptron)	有导师	前向	线性分类
误差反向传播网(BP)	有导师	前向	模式分类、映射、特征抽取
自适应线性元件(Adaline)	有导师	前向	控制、预测、分类
自适应共振理论(ART)	无导师	反馈	模式识别、分类
双向联想记忆(BAM)	不学习	反馈	图像识别
波尔茨曼机(BM)	有导师	反馈前向	模式识别、组合优化
柯西机(CM)	有导师	反馈	模式识别、组合优化
盒中脑(BSB)	有导师	反馈	数据库知识提取
反传网络(CPN)	有导师	前向	联想记忆、图像压缩、统计分析
Hopfield 网络 (DHNN) (CHNN)	 无导师 不学习	反馈	 联想记忆 组合优化
多层自适应线性元件(Madaline)	有导师	前向	自适应控制
新认知机 Neocognitron	有导师	前向	字符识别
自组织映射(SOM)	无导师	前向	聚类、特征抽取
细胞神经网络(CNN)	不学习	反馈	图像处理、图形辨识

深度学习

1.1 人工神经网络

1.2 深度学习原理

1.3 深度学习模型

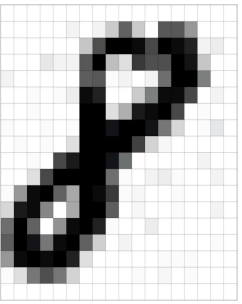
1.4 深度学习应用

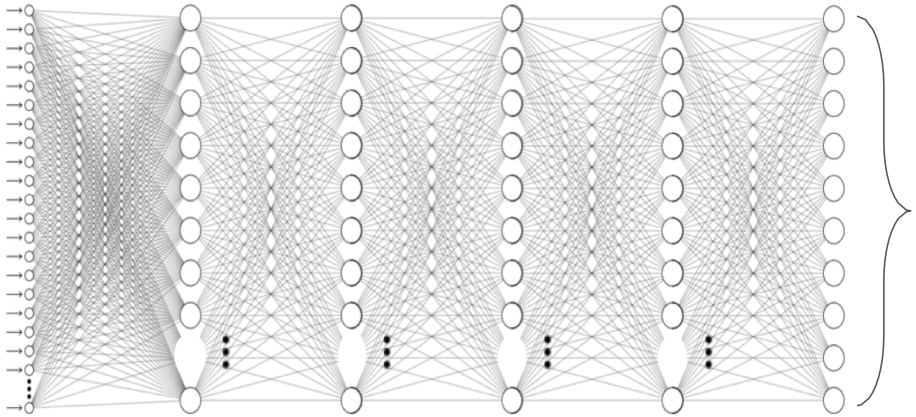
1.2 深度学习原理

■ 深度学习的提出

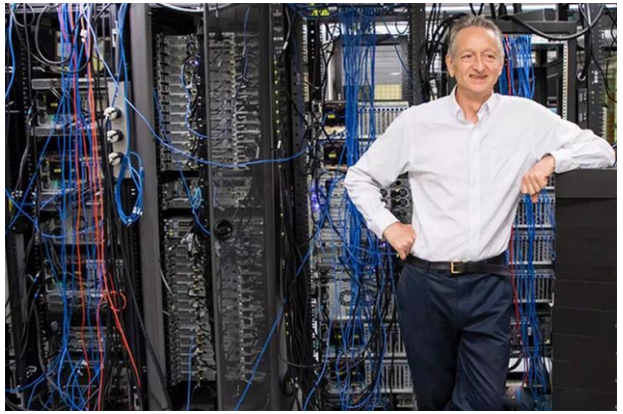
2006年，加拿大多伦多大学教授Geoffrey Hinton和他的学生在*Science*上发表文章，提出了两个重要观点：

- ✓ 多隐层的人工神经网络具有更优异的特征学习能力，学习得到的特征对数据有更本质的刻画，从而有利于可视化或分类；
- ✓ 深度神经网络在训练上的难度，可以通过“逐层初始化”来有效克服。逐层初始化可以通过无监督学习来实现。





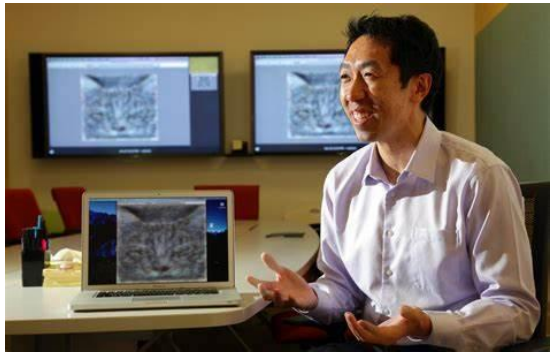




1.2 深度学习原理

■ 深度学习元年-2012

- ✓ 2012年，Google Brain项目用16000个CPU Core的并行计算平台训练深度神经网络（DNN），自动识别出猫。
- ✓ 2012年，Hinton课题组首次参加ImageNet机器视觉比赛，构建的CNN网络AlexNet一鸣惊人夺得冠军，并且碾压传统机器学习方法的识别性能，从此深度学习成为学术和企业界焦点。
- ✓ 2012年，微软在中国天津的一次活动上公开演示了一个全自动的同声传译系统，讲演者用英文演讲，后台的计算机自动完成语音识别、英中机器翻译和中文语音合成。



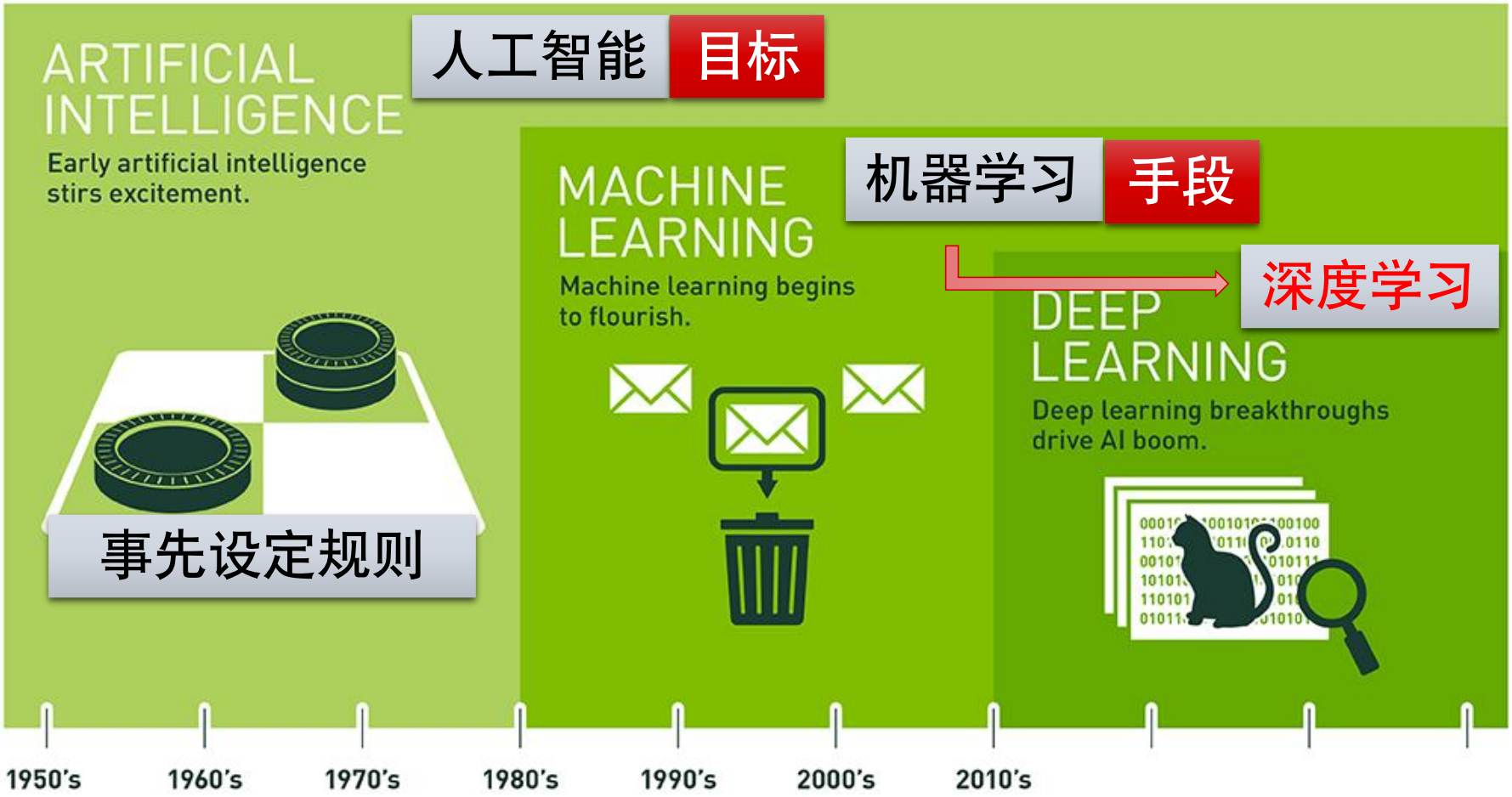




这些应用都是基于2006 年Hinton论文思想
迅速发展起来的深度学习技术。

1.2 深度学习原理

- 深度学习vs机器学习

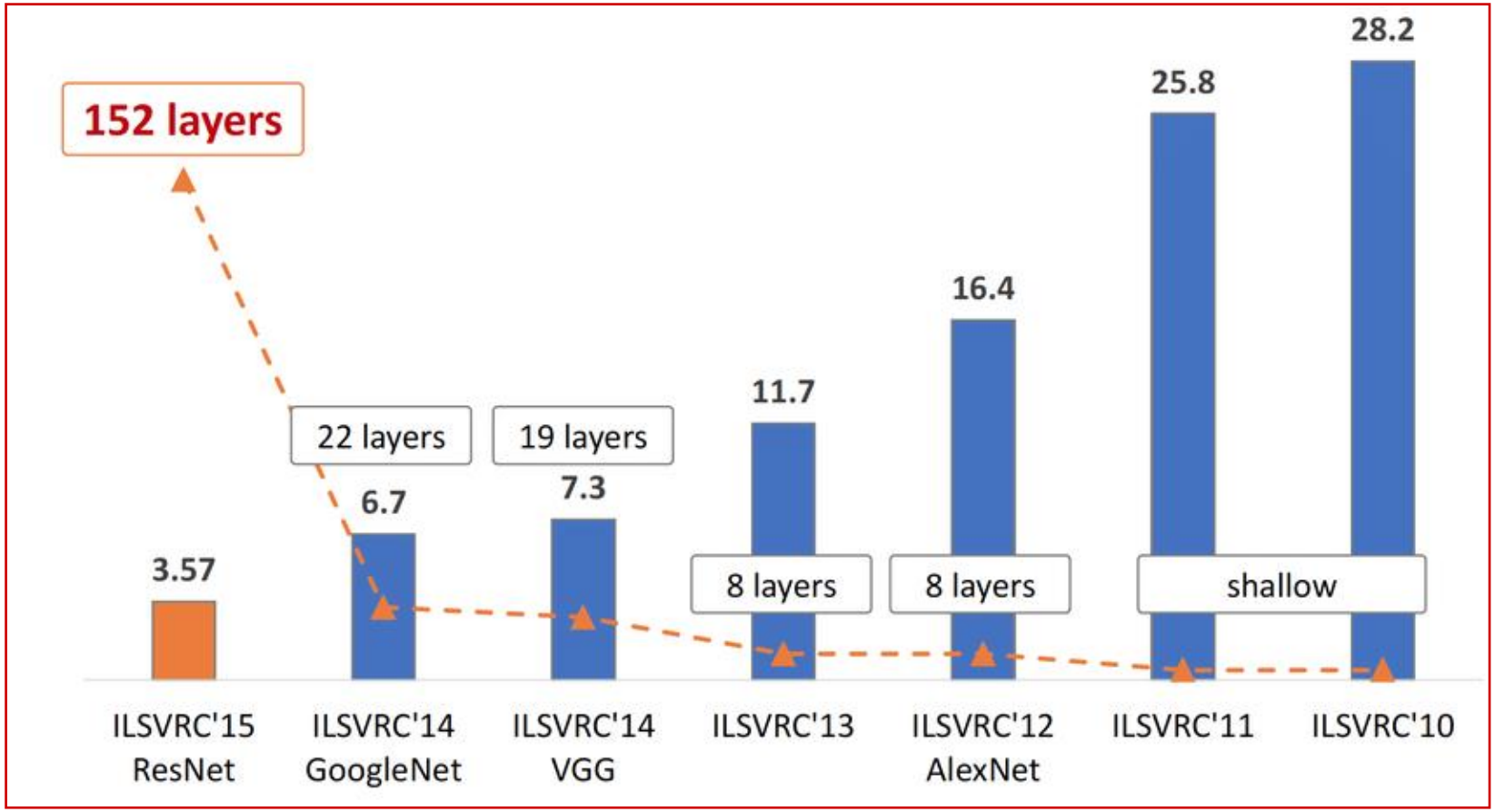


Since an early flush of optimism in the 1950s, smaller subsets of artificial intelligence – first machine learning, then deep learning, a subset of machine learning – have created ever larger disruptions.

来源: https://blogs.nvidia.com.tw/2016/07/whats-difference-artificial-intelligence-machine-learning-deep-learning-ai/

1.2 深度学习原理

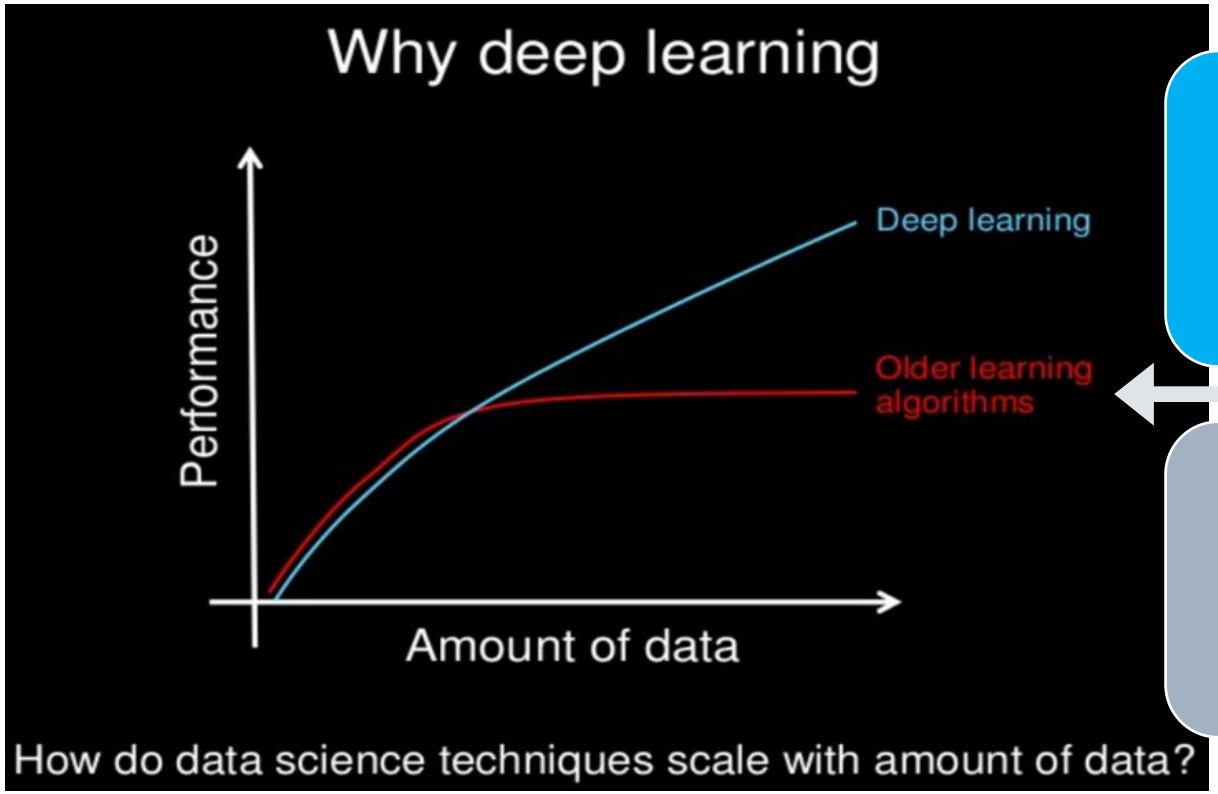
- 深度学习为什么被认为是通向人工智能的关键技术

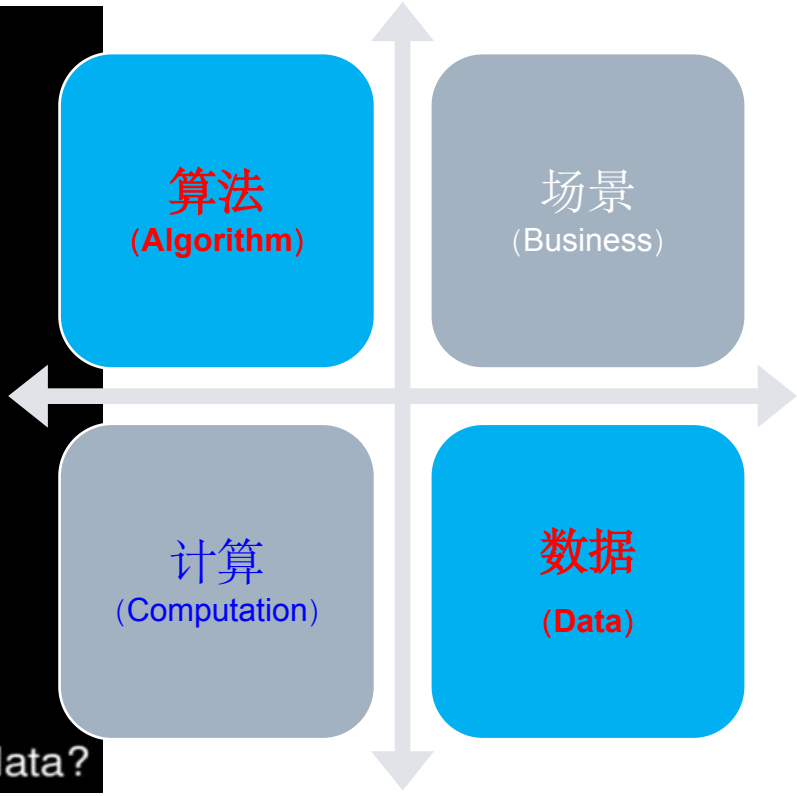


The evolution of the winning entries on the ImageNet Large Scale Visual Recognition Challenge from 2010 to 2015.

1.2 深度学习原理

- 深度学习为什么被认为是通向人工智能的关键技术

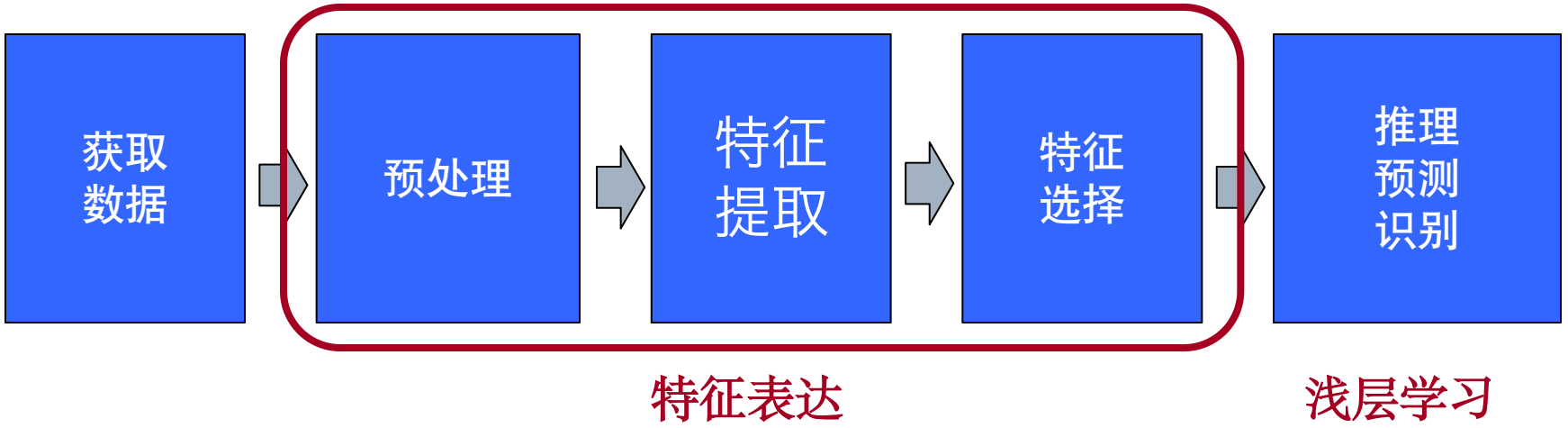




- ✓ 特征越多，给出信息就越多，识别准确性会得到提升；
- ☐ 但特征多，计算复杂度增加，探索的空间大，计算资源要求高。

1.2 深度学习原理

- 为什么需要深度学习？浅层机器学习的问题

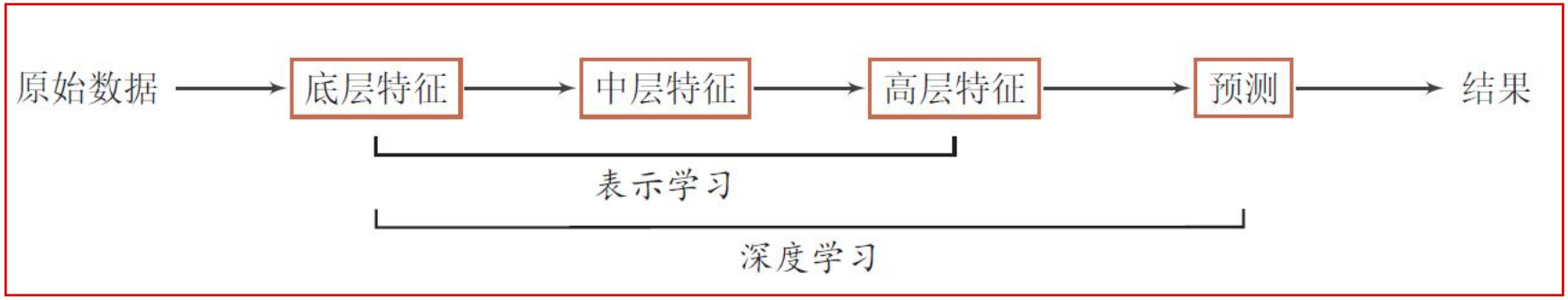


- 良好的特征表达，是识别成功的关键；
- 识别系统的计算和测试工作耗时主要集中在特征表达部分；
- 传统的方法是手工设计良好的特征提取器，这需要大量的工程技术和专业领域知识。

问题：有限样本和有限计算单元情况下对复杂函数的表示能力有限，针对复杂分类问题其泛化能力受到一定制约。

1.2 深度学习原理

- 深度学习描述：表示学习+浅层机器学习



浅层学习

$$y = f(x)$$

$$y = f^2(f^1(x))$$

增加非线性转换次数

$$\dots$$

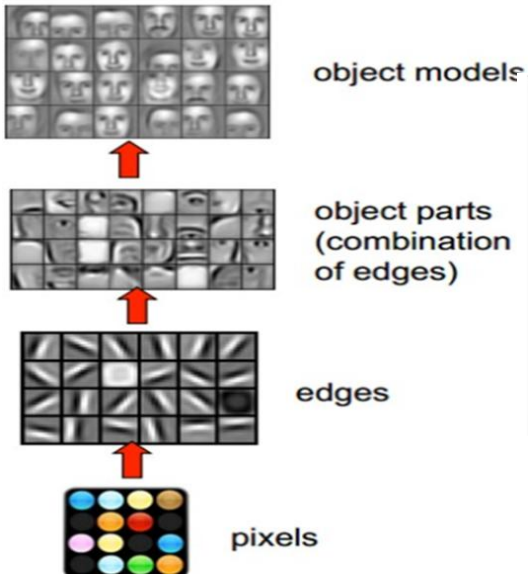
深度学习

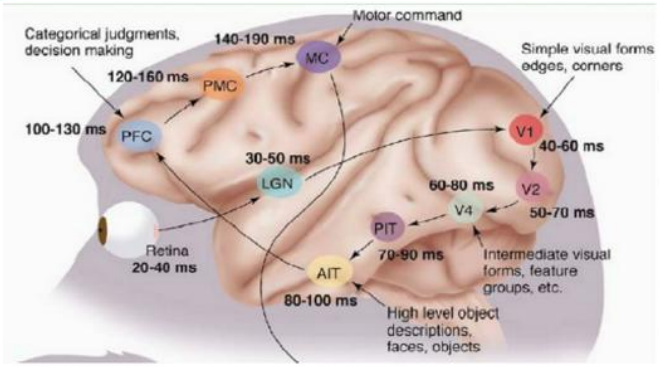
$$y = f^5(f^4(f^3(f^2(f^1(x)))))$$

- 模型选择：
  多层神经网络
- 学习准则：
  平方差/交叉熵损失等
- 优化方法：
  随机梯度下降

1.2 深度学习原理

- 设计思想：大脑视觉皮层学习的启发



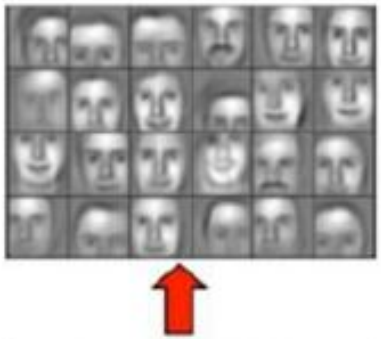




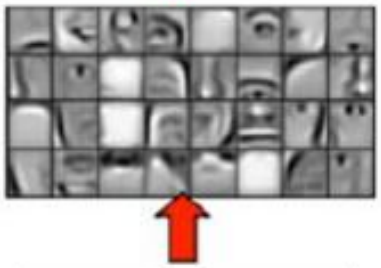
- ✓ 1981年诺贝尔医学奖获得者美国神经生物学家David Hubel和TorstenWiesel发现：人的视觉系统的信息处理是分级的。
- ✓ 高层的特征是低层特征的组合，从低层到高层的特征表示越来越抽象，越来越能表现语义或者意图。

1.2 深度学习原理

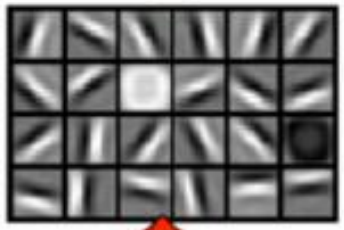
- 设计思想：大脑视觉皮层学习的启发



object models



object parts (combination of edges)

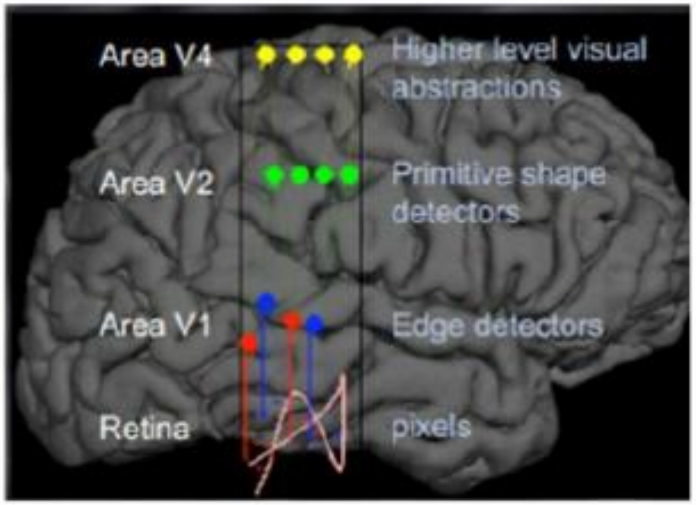


edges



pixels

我们的大脑视觉皮层如何工作?



关键：抽象和迭代

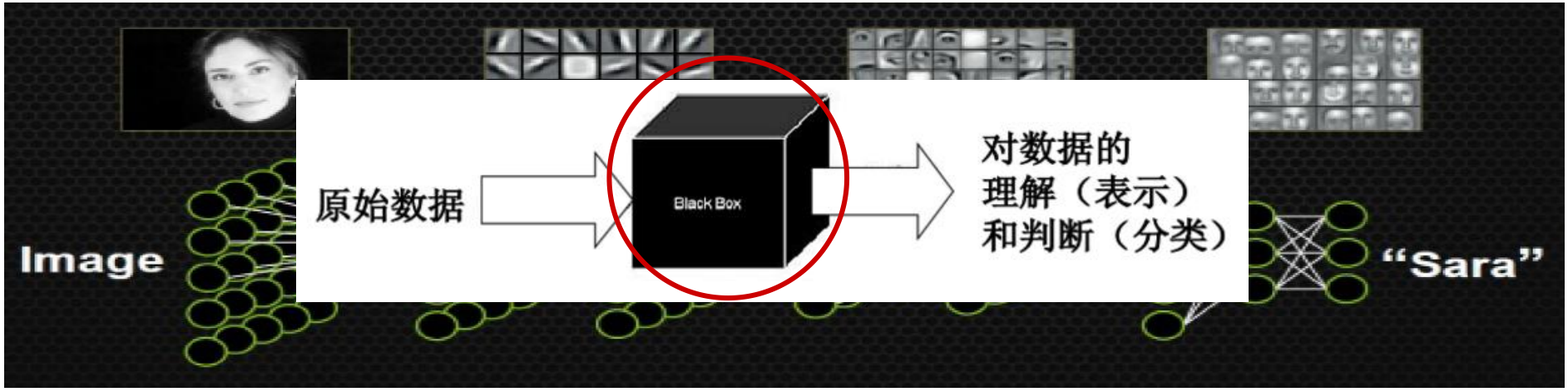
从原始信号开始，从低级抽象向高级抽象迭代

原始信号（视网膜素）->初步处理（边缘和方向）->抽象（轮廓和形状）->进一步抽象（谁的脸）

1.2 深度学习原理

■ 深度学习原理

✓ 深度学习使用一种通用的学习过程，从数据中自动学习多层次/多粒度的深度特征，而不是像经典机器学习需要手工设计特征提取算法。



对黑盒子的限定

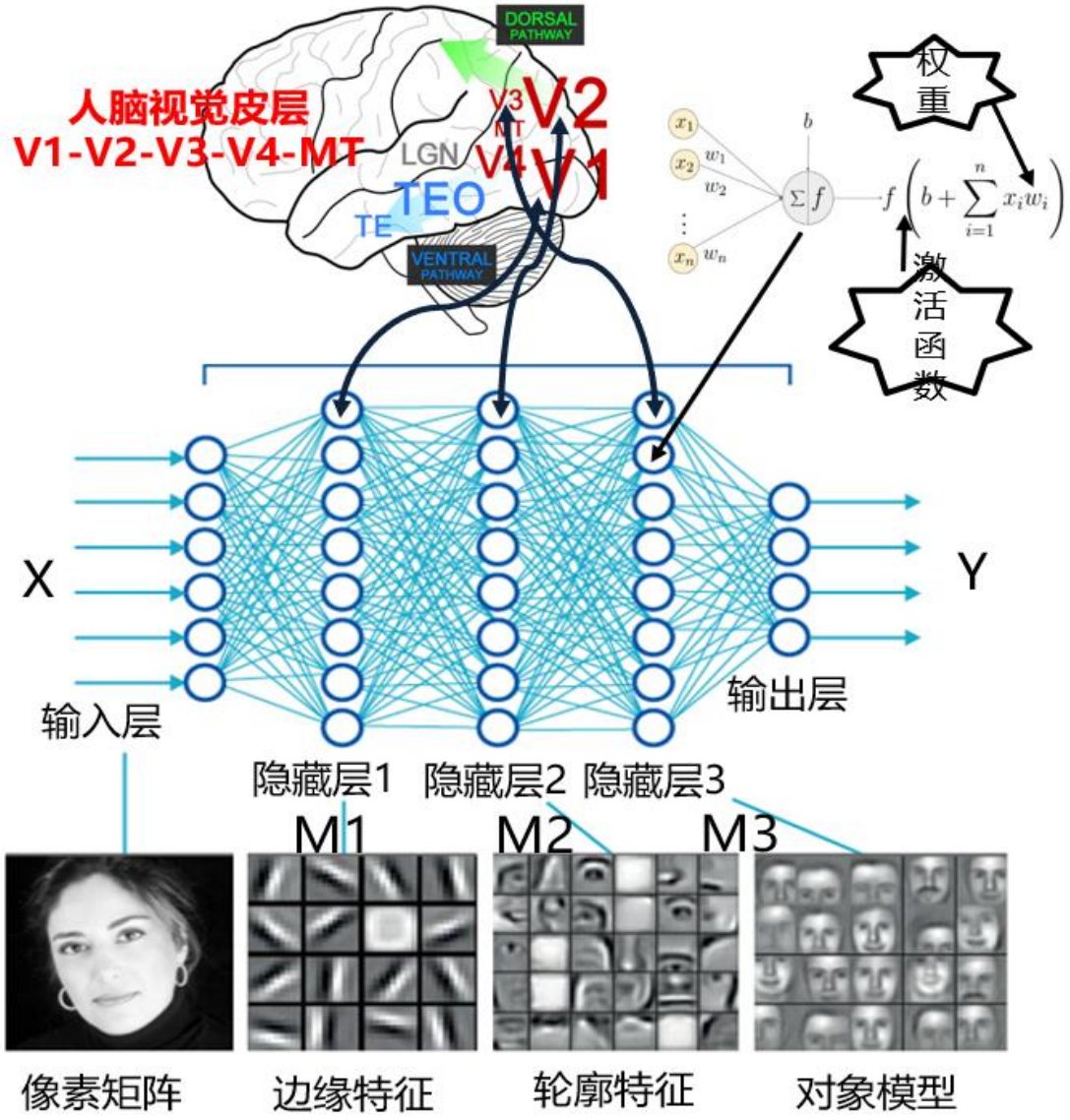
- **深度：** 多层非线性信息处理
  - Input => L_1 => L_2 => … => L_n => Output
- **学习：** 内部结构通过学习涌现

假设系统S有n层（L1,…,Ln），它的输入是I，输出是O，表示为： I =>L1=>L2=>…..=>Ln => O。如果调整系统中参数，使得它的输出O等于输入I，那么就可以自动地获得输入I的一系列层次特征，即L1，…, Ln。

$$y = f^5(f^4(f^3(f^2(f^1(x))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x}\frac{\partial f^2}{\partial f^1}\frac{\partial f^3}{\partial f^2}\frac{\partial f^4}{\partial f^3}\frac{\partial f^5}{\partial f^4}$$

1.2 深度学习原理

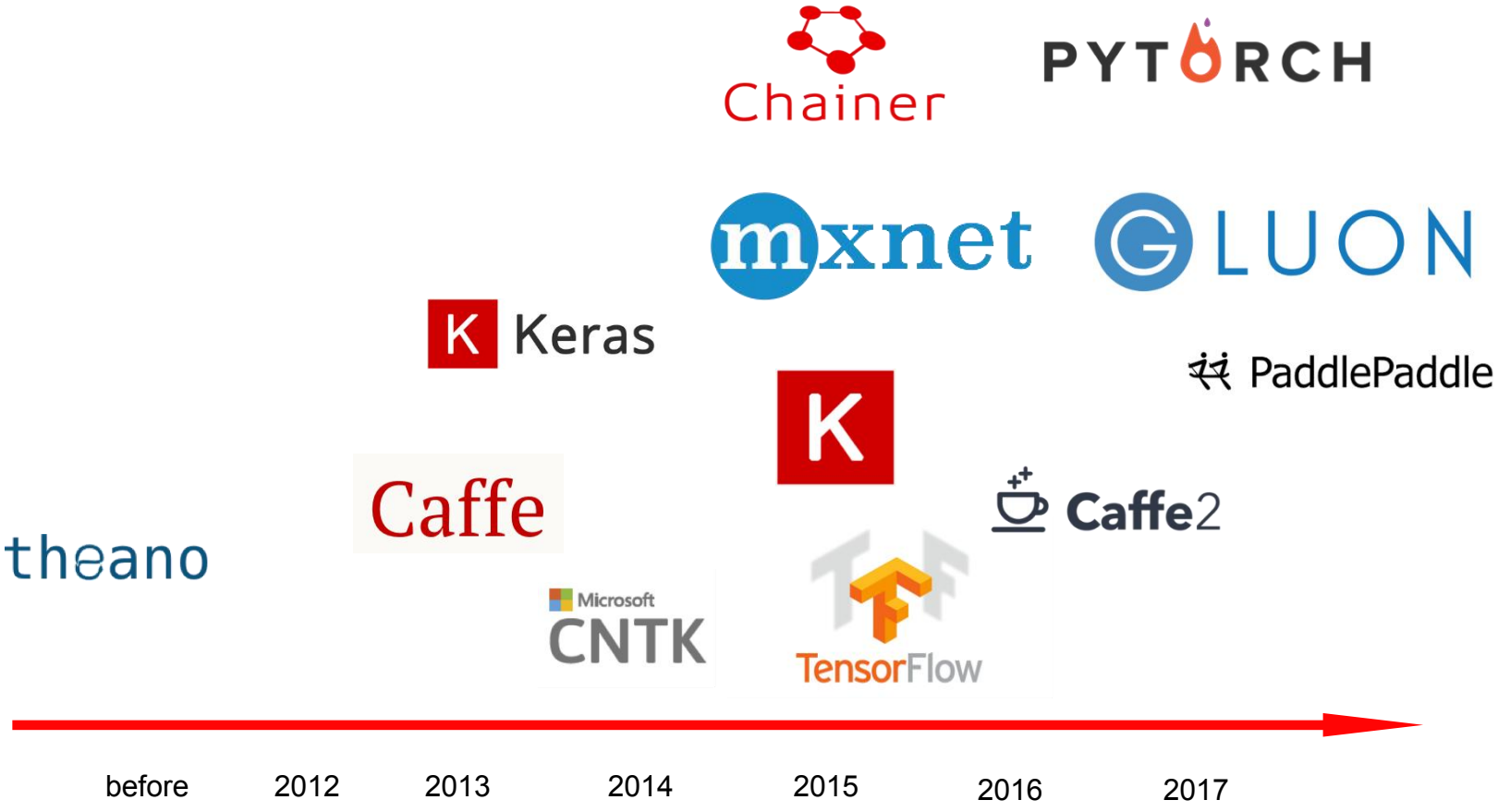
深度学习原理



- 模拟视觉皮层机制逐层特征抽象
- 多个隐藏层做通用近似学习
- 端到端学习过程

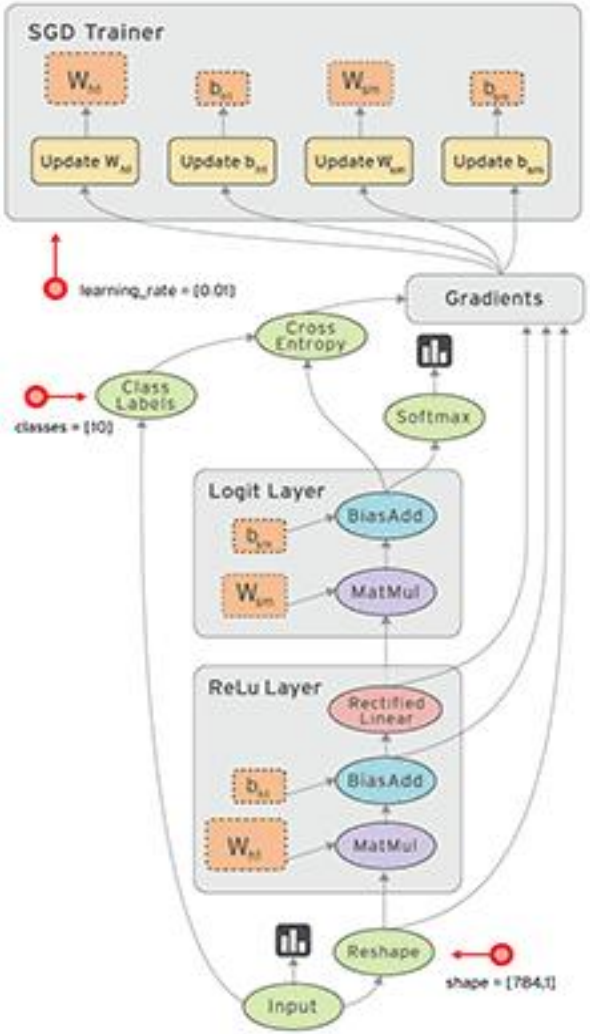
1.2 深度学习原理

- 深度学习框架



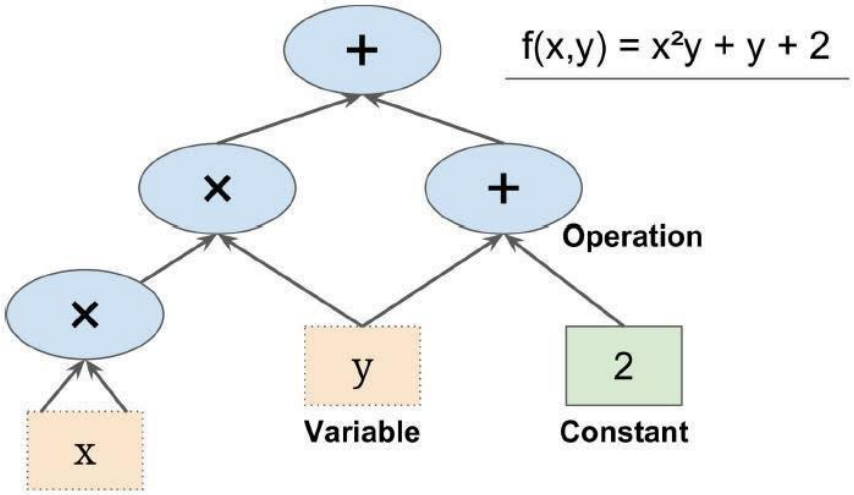
1.2 深度学习原理

■ 深度学习框架 Tensorflow



Tensorflow：

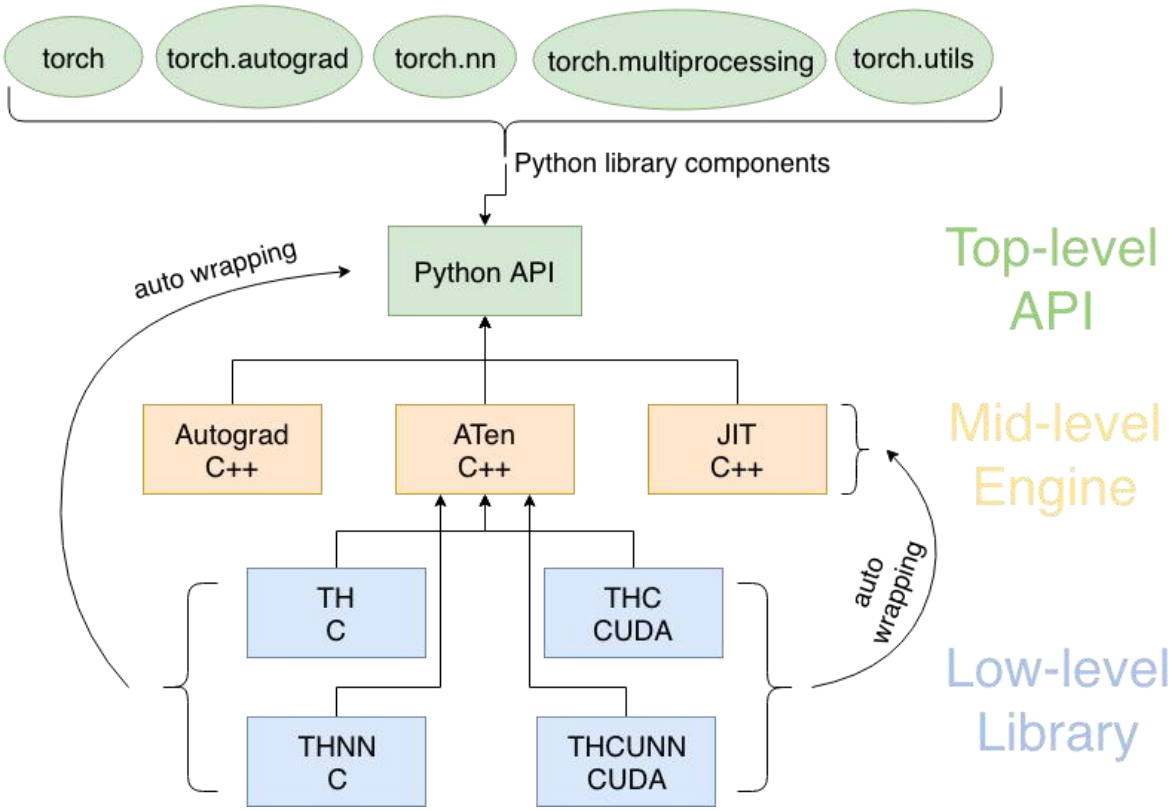
- 计算图：数学计算的有向图(有向无环图)
- 节点：操作(Operation)
- 边：Tensor的流动

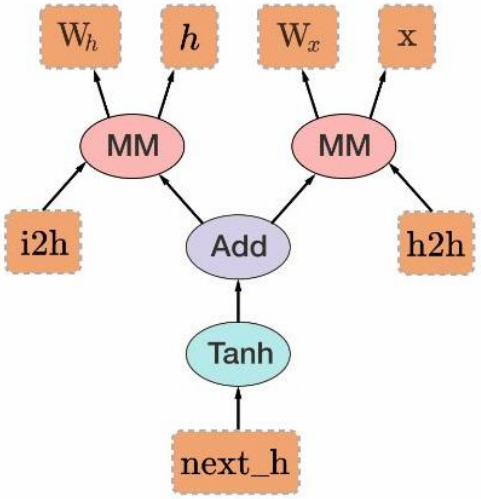


✓ 丰富的算子
✓ 丰富的功能
□ 代码可读性有限

1.2 深度学习原理

■ 深度学习框架 Pytorch






```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

i2h = torch.mm(W_x, x.t())
h2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
next_h = next_h.tanh()
```

- ✓ 反向求导, 实现速度快
- ✓ 支持动态神经网络,底层代码易于理解；命令式体验，自定义扩展。
- ◘ 缺点：针对移动端、嵌入式部署以及高性能服务器端的部署，全面性支持不足，科研学术圈流行。

深度学习

1.1 人工神经网络

1.2 深度学习原理

1.3 深度学习模型

1.4 深度学习应用

1.3 深度学习模型

■ 主流的深度学习模型

✓ 前馈式深度网络：

多层感知机MLP、卷积神经网络CNN等。

✓ 反馈式深度网络：

循环神经网络RNN、长短时记忆网络LSTM等。

✓ 生成式深度网络：

自编码器SAE、TRANSFORMER、GAN等。

1.3 深度学习模型

卷积神经网络CNN　　什么是卷积

- 卷积经常用在信号处理中，用于计算信号的延迟累积。
- 假设一个信号发生器每个时刻 t 产生一个信号 x_t，其信息的衰减率为w_k，即在 $k-1$ 个时间步长后，信息为原来的 w_k 倍
  - 假设 $w_1 = 1, w_2 = 1/2, w_3 = 1/4$
- 时刻 t 收到的信号 y_t 为当前时刻产生的信息和以前时刻延迟信息的叠加。

$$
\begin{aligned}
y_t &= 1 \times x_t + 1/2 \times x_{t-1} + 1/4 \times x_{t-2} \\
&= w_1 \times x_t + w_2 \times x_{t-1} + w_3 \times x_{t-2} \\
&= \sum_{k=1}^{3} w_k \cdot x_{t-k+1}.
\end{aligned}
$$

滤波器（filter）或卷积核（convolution kernel）

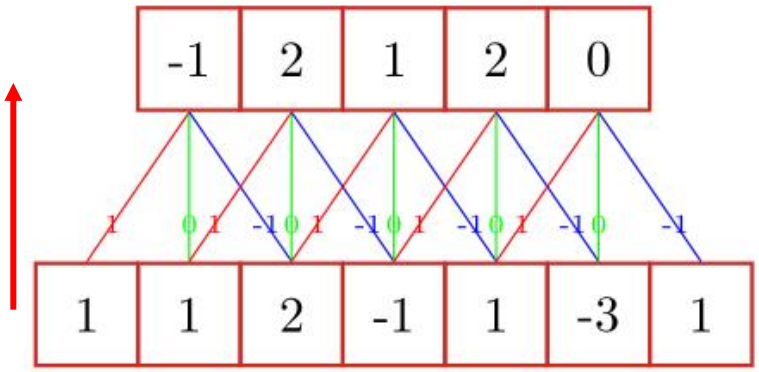
1.3 深度学习模型

卷积神经网络CNN　　什么是卷积

- 给定一个输入信号序列x和滤波器w,卷积的输出为:

$$y_t = \sum_{k=1}^{K} w_k x_{t-k+1}$$

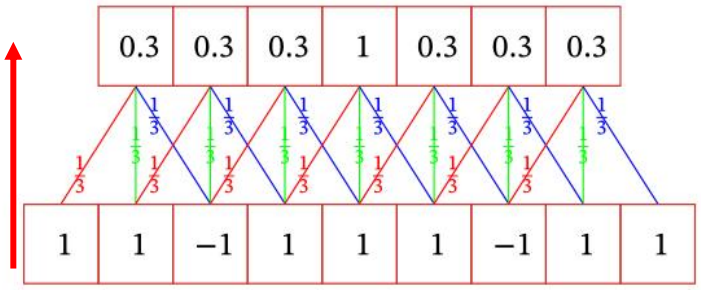
Filter: [−1,0,1]



1.3 深度学习模型

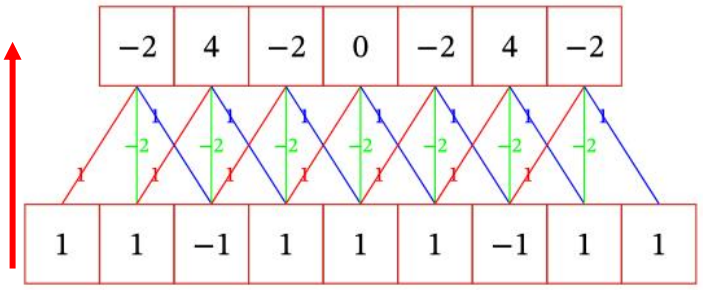
卷积神经网络CNN　　什么是卷积

- 不同的滤波器来提取信号序列中的不同特征



(a) 滤波器 [1/3, 1/3, 1/3]

低频信息



(b) 滤波器 [1, −2, 1]

高频信息

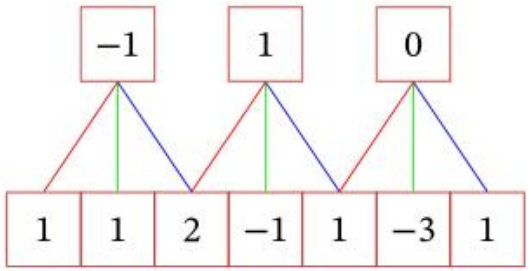
- 低通滤波器（提取低频）：
  - 卷积核所有元素之和 ≈ 1（或 > 0）
  - 作用：保留平均信号值，抑制突变

- 高通滤波器（提取高频）：
  - 卷积核所有元素之和 ≈ 0
  - 作用：抑制平滑区域，突出变化

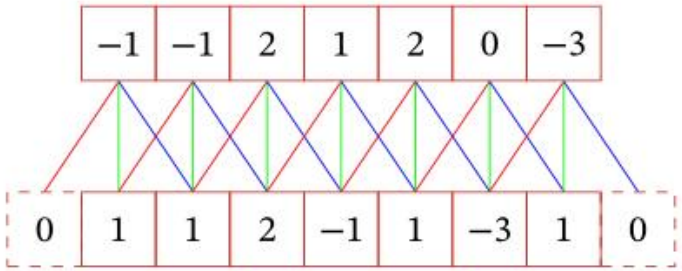
1.3 深度学习模型

卷积神经网络CNN　　卷积扩展

- 引入滤波器的滑动步长 S 和零填充 P



(a) 步长 $S = 2$



(b) 零填充 $P = 1$

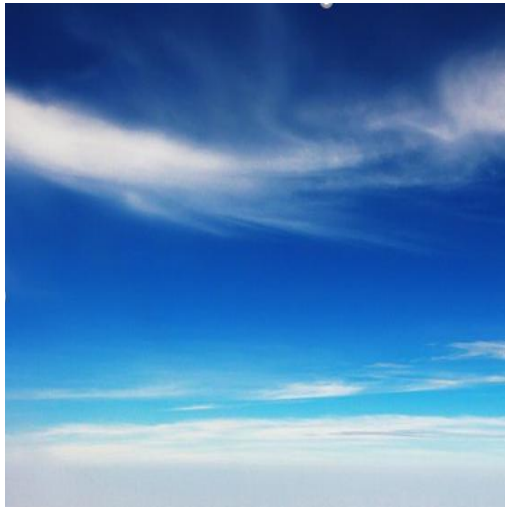
在图像处理中，使用的是二维卷积操作*

*卷积神经网络中的“卷积”在数学本质上其实是互相关操作，后简化统称为了卷积

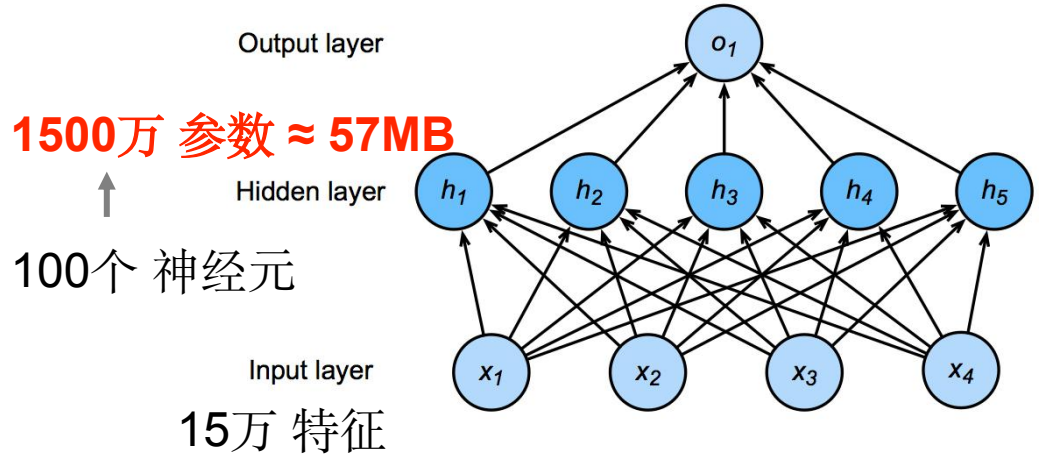
1.3 深度学习模型

卷积神经网络CNN

处理图像为什么需要卷积？



224 × 224 × 3 = 150,528 个像素



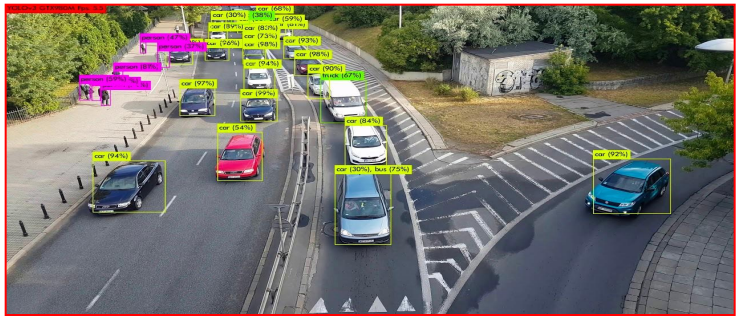
多层叠加轻松占用超GB级

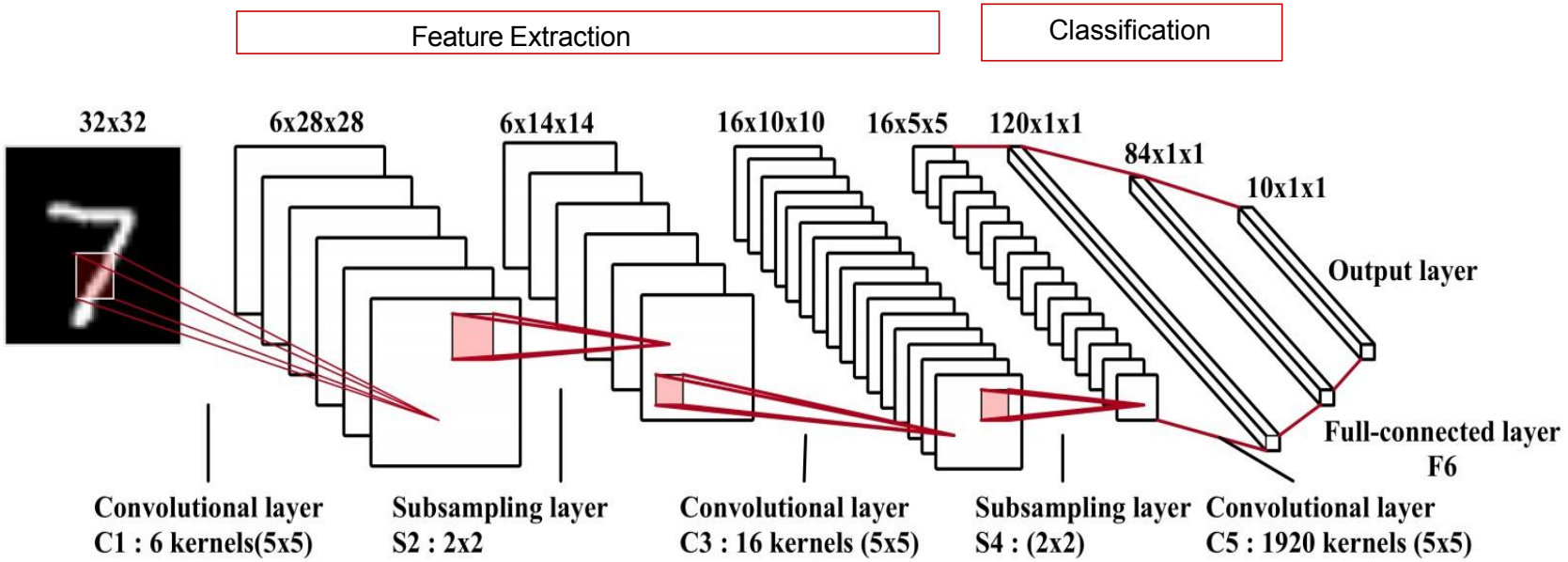
- 参数爆炸：计算和存储成本高
- 缺乏空间结构建模能力：MLP将图像视为无序向量，忽略了像素间的局部相关性
- 无法实现平移不变性（Translation Invariance）：同一物体出现在图像不同位置时，MLP无法识别为同一类

1.3 深度学习模型

■ 卷积神经网络CNN　　卷积网络构成

卷积神经网络是一种前馈神经网络，它的人工神经元可以响应一部分覆盖范围内的周围单元，对于大型图像处理有出色表现。核心包括卷积层(convolutional layer)和池化层(pooling layer)。



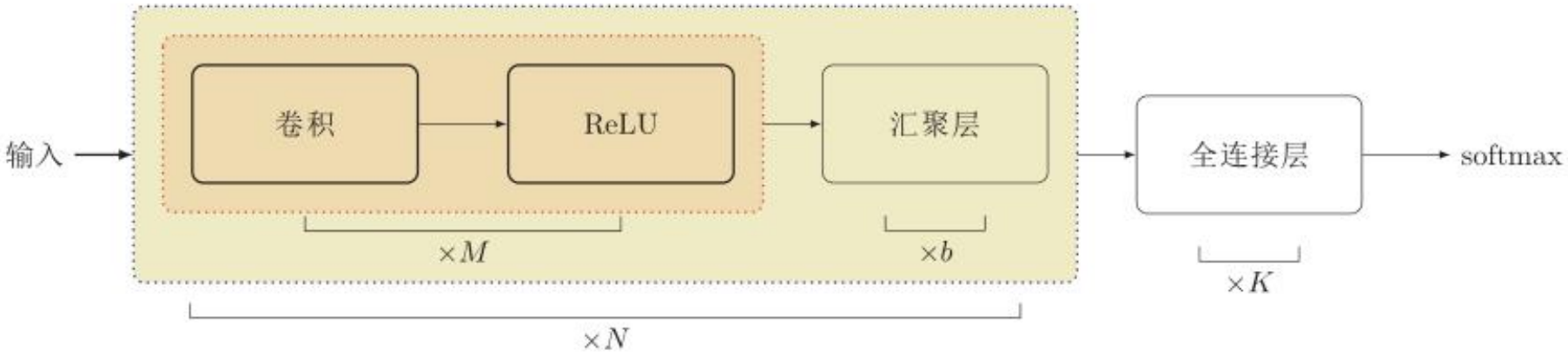


1.3 深度学习模型

■ 卷积神经网络CNN　　卷积网络构成

一个卷积块为连续M 个卷积层和b个汇聚层（M通常设置为2 ~ 5，b为0或1）。

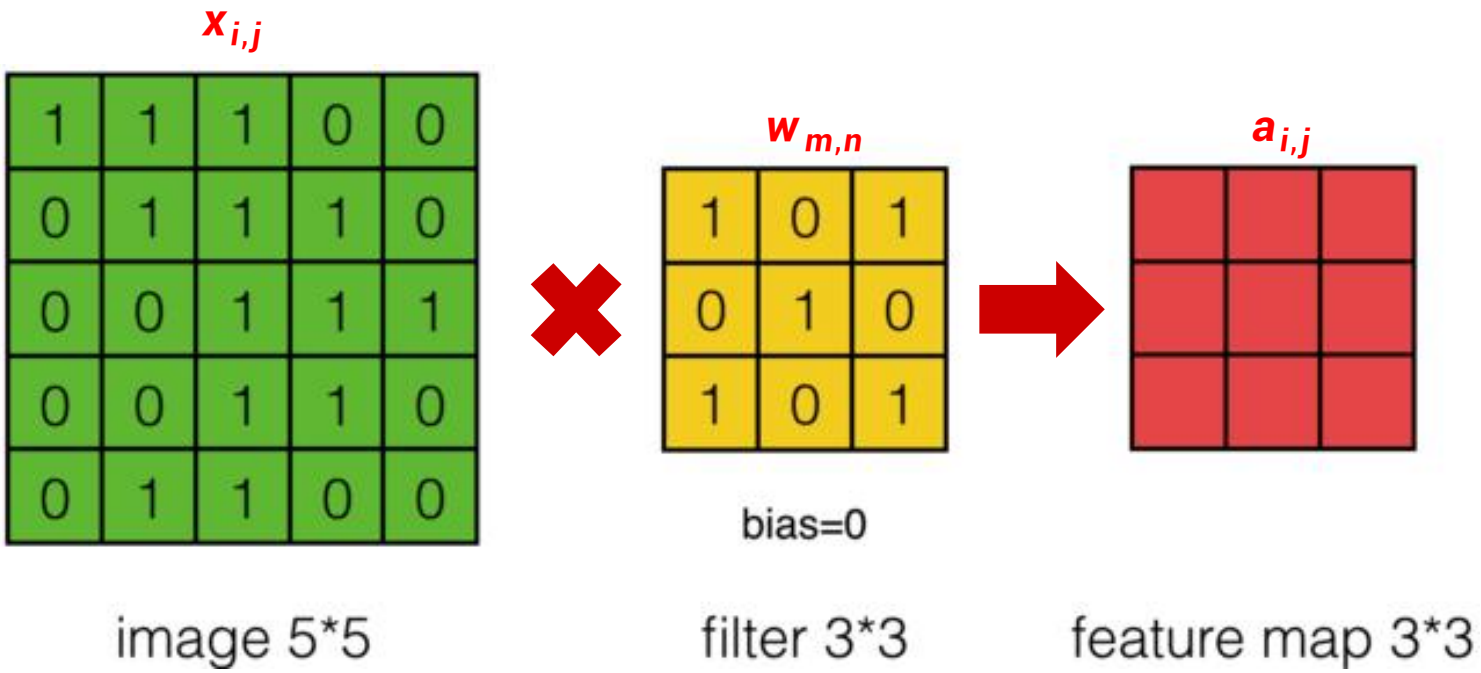
一个卷积网络中可以堆叠N 个连续的卷积块，然后在接着K 个全连接层（N 的取值区间比较大，比如1 ~ 100或者更大；K一般为0 ~ 2）



1.3 深度学习模型

- 卷积神经网络CNN

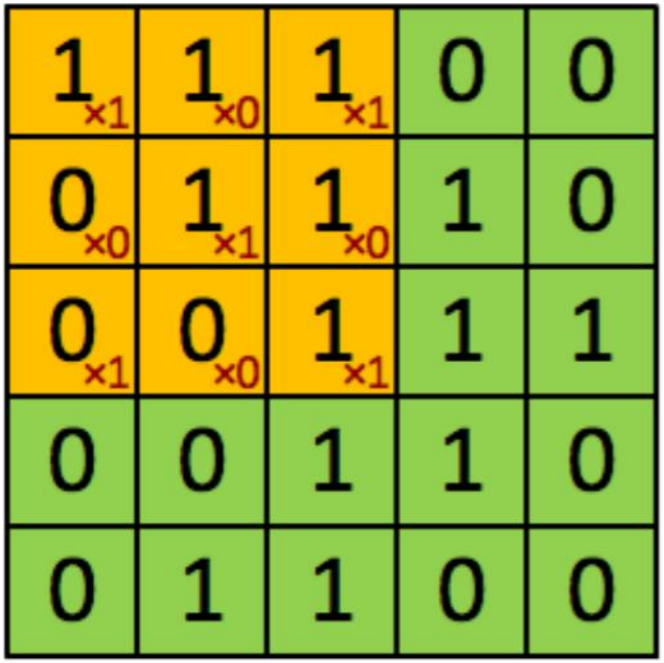
卷积层-Conv计算



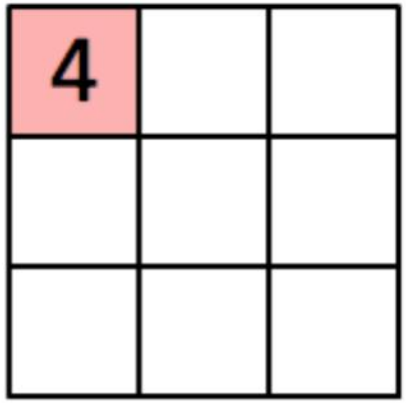
$$a_{i,j} = f\left(\sum_{m=0}^{2}\sum_{n=0}^{2} w_{m,n}x_{i+m,j+n} + w_b\right)$$

1.3 深度学习模型

- 卷积神经网络CNN　　卷积层-Conv计算



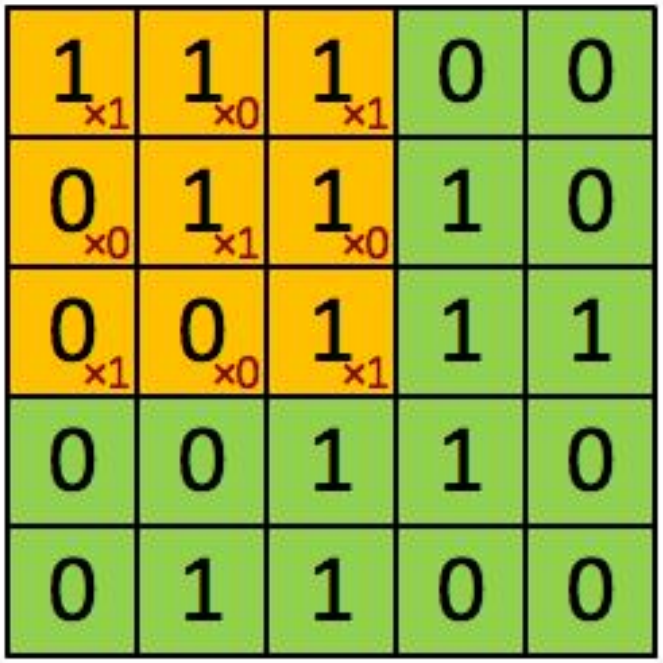
Image



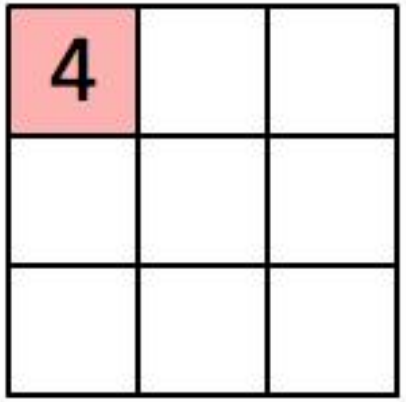
Convolved Feature

1.3 深度学习模型

- 卷积神经网络CNN　　卷积层-Conv计算



Image



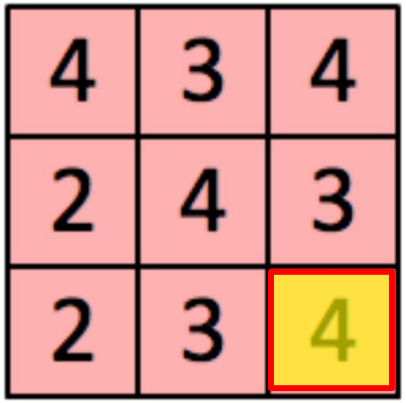
Convolved Feature

1.3 深度学习模型

- 卷积神经网络CNN　　卷积层-Conv计算



Image

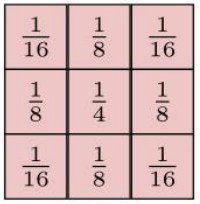


Convolved Feature

1.3 深度学习模型

■ 卷积神经网络CNN

设计不同卷积核以得到不同特征



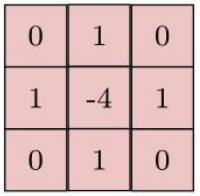
=



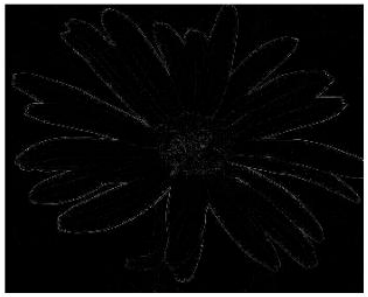
呈现对中心像素较大、对周边像素较小的加权模式，这是典型的 高斯核 分布，也可能是简化的 平均平滑。



⊗

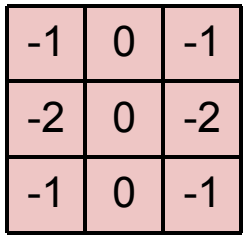


=

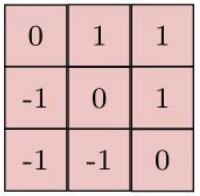


拉普拉斯算子，它可以突出图像的“二阶导数”信息，从而增强或检测图像的边缘。

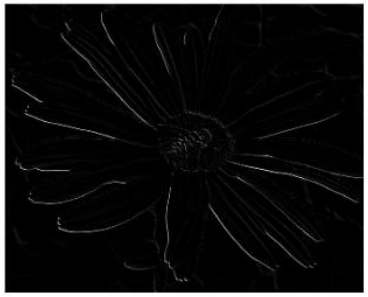
原始图像



对哪个方向敏感？



=



边缘检测算子：对某些方向的细节更敏感

滤波器

输出特征映射

1.3 深度学习模型

■ 卷积神经网络CNN　　卷积层-Conv计算

Input
3x3

0	1	2
3	4	5
6	7	8

*

Kernel
2x2

0	1
2	3

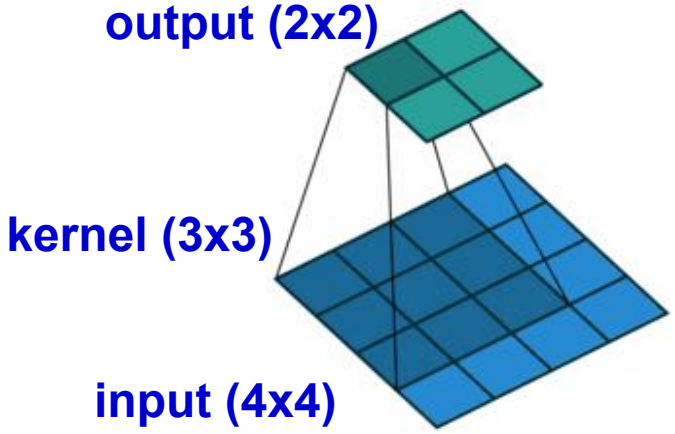
=

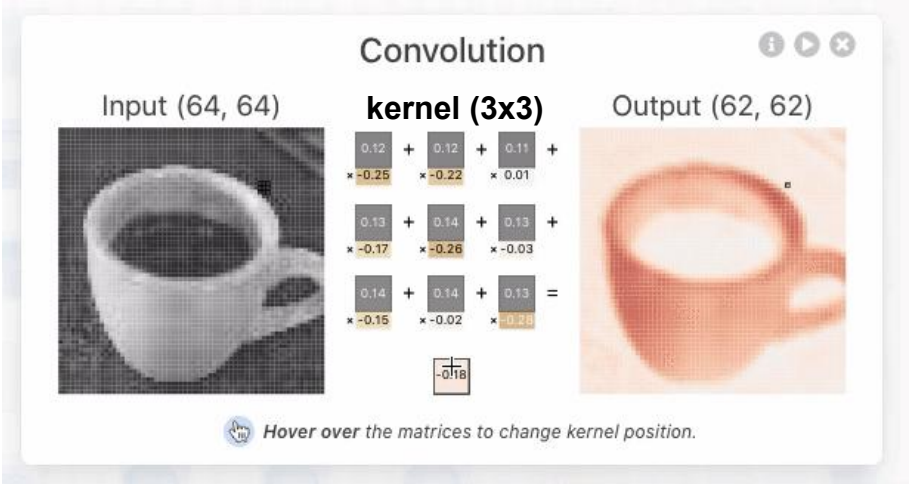
Output
2x2

19	25
37	43

$$0\times0+1\times1+3\times2+4\times3=19,$$
$$1\times0+2\times1+4\times2+5\times3=25,$$
$$3\times0+4\times1+6\times2+7\times3=37,$$
$$4\times0+5\times1+7\times2+8\times3=43.$$

C层（Convolution layer）





1.3 深度学习模型

■ 卷积神经网络CNN

卷积层设计-填充Padding

填充： 在输入周围添加额外的行 / 列

Input 3x3 → 5x5

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

*

Kernel 2x2

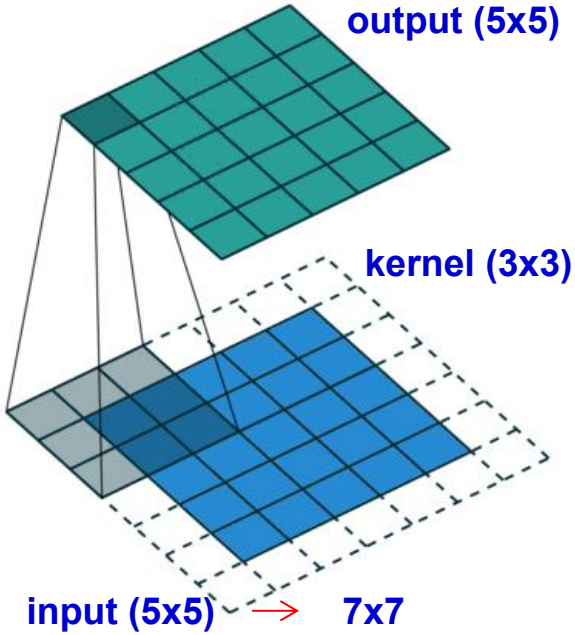
0	1
2	3

=

Output 4x4

0	3	8	4
9	19	25	10
21	37	43	16
6	7	8	0

$$0 \times 0 + 0 \times 1 + 0 \times 2 + 0 \times 3 = 0$$



为什么使用零填充？

数学简洁性

- 0 不影响卷积计算：x*0=0
- 梯度回传时，填充区域的梯度也为 0，不影响原始输入的参数更新

0 是“中性”值

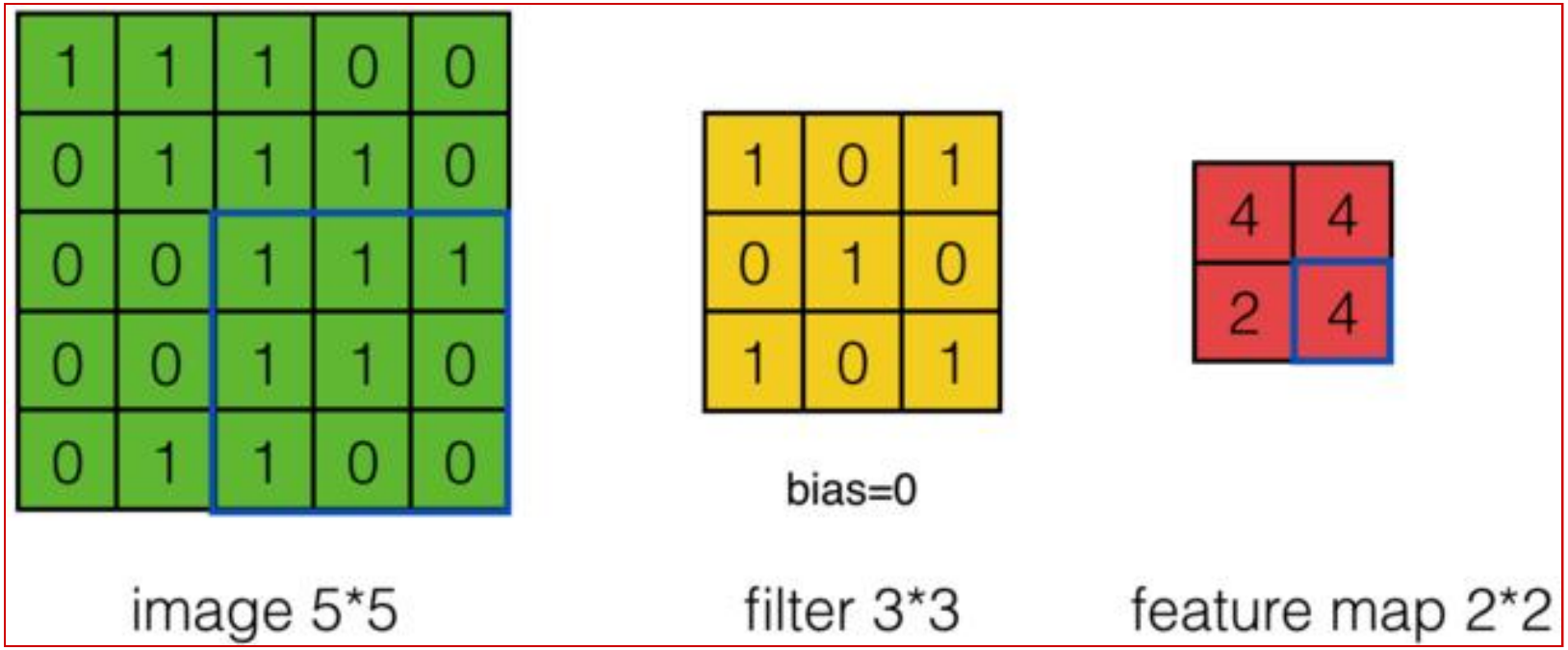
- 对于图像，像素值通常归一化到 [0,1] 或 [-1,1]
- 0 填充相当于“黑色背景”，在大多数场景下是无信息的默认值
- 不会引入额外的语义干扰（如用 255 填充会引入“白色斑块”）

1.3 深度学习模型

■ 卷积神经网络CNN

卷积层设计-步幅Stride

步幅：行/列的滑动步长---卷积后的feature map大小与步幅大小有关



Stride=2

步幅会影响输出特征的维度

1.3 深度学习模型

■ 卷积神经网络CNN　　步幅Stride的特性

✓ 控制输出分辨率（空间下采样）

如果卷积都用 stride=1，特征图尺寸几乎不变（靠 padding 维持），网络无法逐步缩小空间维度。但实际任务中，我们希望：

浅层：保留高分辨率，捕捉细节（如边缘）

深层：降低分辨率，聚焦语义，减少计算量

步长提供了一种“可控的降采样”方式，替代或补充池化层。

✓ 大幅减少计算量和参数量

✓ 扩大感受野（Receptive Field）

感受野：输出特征图上一个点，对应输入图像的区域大小。

使用更大步长，相同层数下，感受野增长更快。

✓ 避免冗余计算

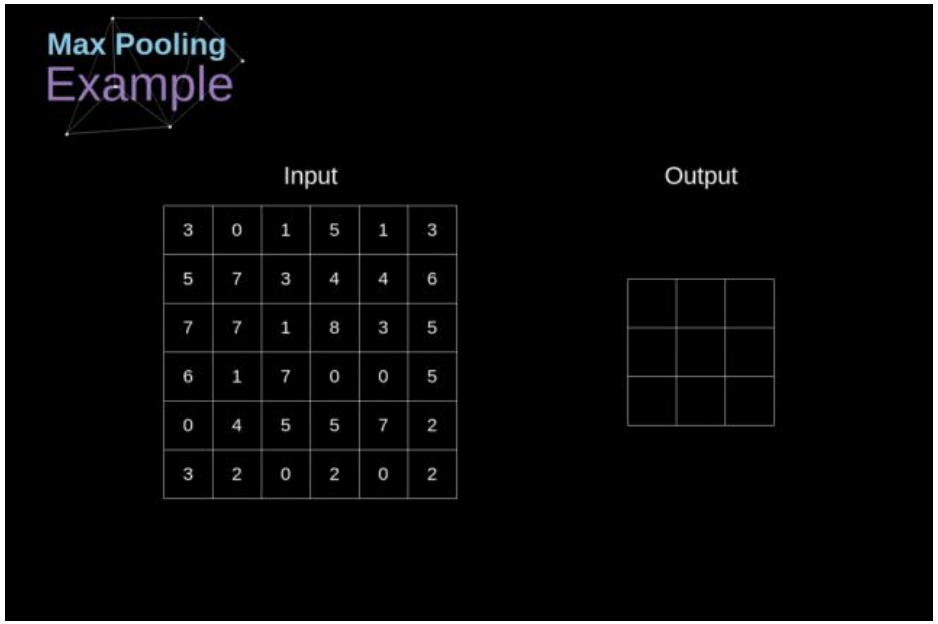
1.3 深度学习模型

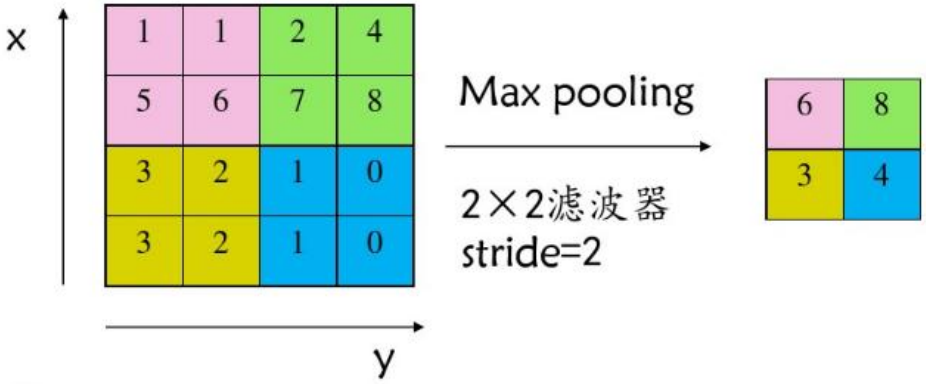
- **卷积神经网络CNN　　汇聚层/池化层　（Pooling layer）**

卷积层的作用是探测上一层特征的局部连接,如边缘；

池化层的作用是在语义上把相似的特征合并起来，去掉不重要的样本

最大池化操作（Max Pooling）





此外，还有平均池化(Average Pooling)

池化也会影响输出特征的维度

1.3 深度学习模型

■ 卷积神经网络CNN　　汇聚层/池化层 （Pooling layer）

✓ 空间下采样（Spatial Downsampling）

减少特征图的高和宽，降低后续层的计算量；

与 stride>1 的卷积类似，但无参数、不可学习；

使网络能构建多尺度特征金字塔（浅层细节 + 深层语义）。

✓ 增强平移不变性（Translation Invariance）

若物体在图像中微小平移（如移动1像素），卷积响应会偏移；

但只要该特征仍在池化窗口内，最大值仍会被保留；

✓ 减少过拟合风险

降低参数量

1.3 深度学习模型

■ 卷积神经网络CNN　　使用池化与步长的差异

维度	步长（Stride in Convolution）	池化层（Pooling Layer）
是否可学习?	✅ 是：卷积核参数通过反向传播学习	❌ 否：固定操作（如取最大值、平均值）
操作本质	**特征提取 + 下采样 同时进行**	**仅下采样**，不提取新特征
信息保留方式	通过**加权组合**（卷积）保留结构信息	通过**启发式规则**（max/avg）丢弃部分信息
参数量	有参数（如 3×3×C_in×C_out）	**无参数**
计算量	较高（需乘加运算）	极低（仅比较或求和）
灵活性	高（可适应任务学习最优下采样）	低（行为固定）
典型用法	stride=2 的卷积层直接输出更小特征图	卷积后接 MaxPooling(2×2, stride=2)

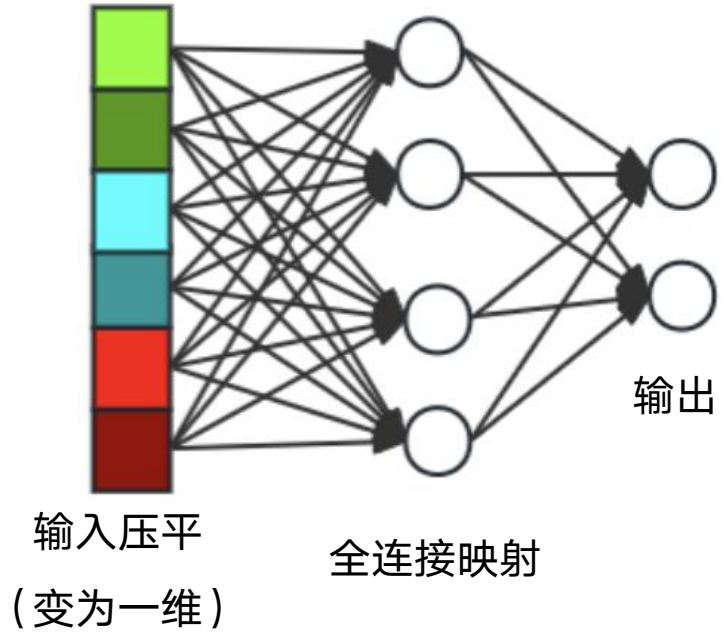
- **早期 CNN（LeNet, AlexNet）：**
  - 卷积（stride=1） + MaxPooling → 下采样主力是池化
- **现代 CNN（ResNet, EfficientNet）：**
  - 直接使用 stride=2 的卷积 进行下采样
  - 池化仅用于特定位置（如 ResNet 的第一个 block 用 MaxPool）

1.3 深度学习模型

- **卷积神经网络CNN**　　**全连接层 FC层（Fully Connected layer）**

卷积层和池化层提取的是局部或全局的空间特征

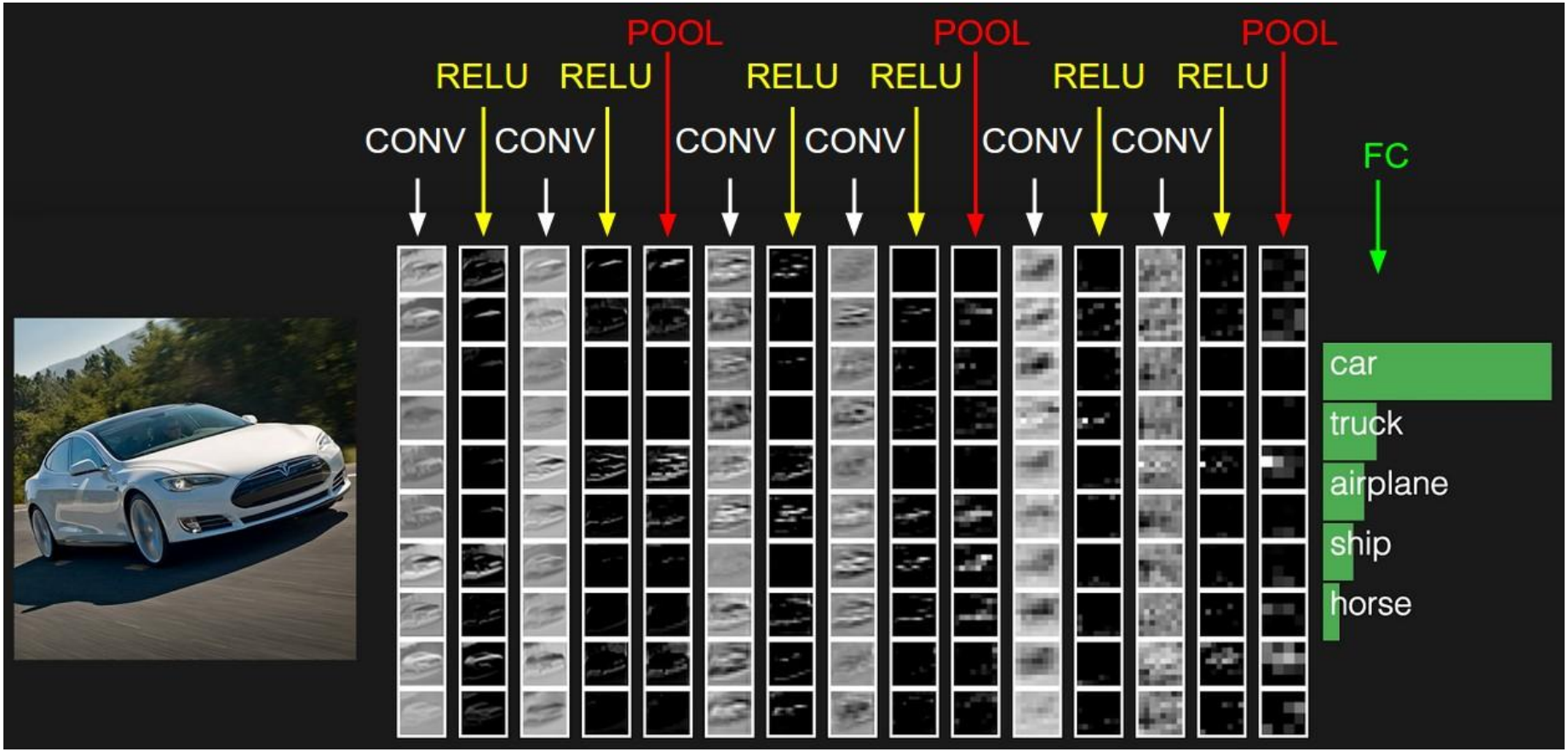
全连接层将这些高维特征图“压平”并整合，学习特征之间的全局组合关系



1.3 深度学习模型

- 卷积神经网络CNN　　全连接层 FC层（Fully Connected layer）

用在CNN网络的最后，完成向任务特征的映射



连接提取出来的特征输入分类器(softmax等)

1.3 深度学习模型

■ 卷积神经网络CNN　　全连接层的特性

问题	说明
参数量巨大	例如 VGG16 中，仅 FC1 层就有 $25088 \times 4096 \approx 1.02$ 亿参数，占全网 90% 以上！
容易过拟合	高维全连接 + 少量数据 → 过拟合风险高（需 Dropout 正则化）
破坏空间结构	Flatten 操作丢失了特征图的 2D 空间关系（如“眼睛在鼻子上方”）
不适用于密集预测	无法用于语义分割、目标检测等需要保留空间分辨率的任务

全连接层的角色演变

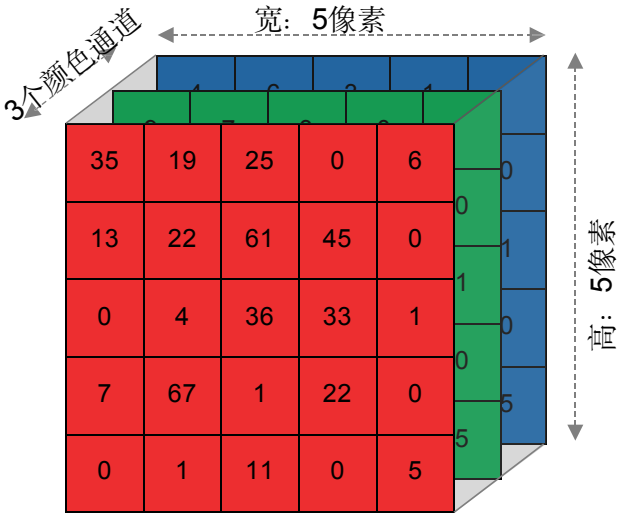
时期	代表网络	FC 层作用	现代观点
早期	LeNet, AlexNet, VGG	核心分类器	参数爆炸，需 Dropout
中期	GoogLeNet	开始减少 FC	引入 GAP 思想
现代	ResNet, EfficientNet	**基本被 GAP + 1×1 Conv 替代**	更高效、更鲁棒

1.3 深度学习模型

■ 卷积神经网络CNN

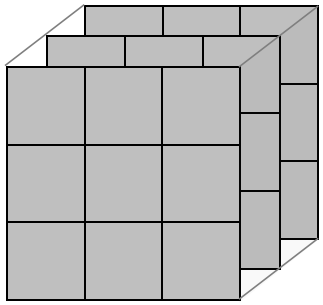
多通道？





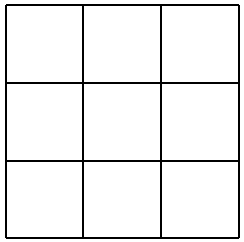
输入3D图像

5 × 5 × 3



Filter

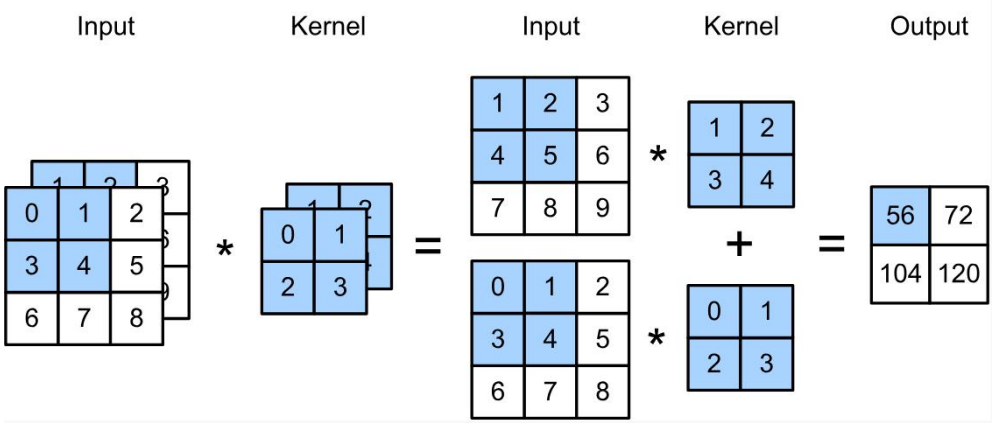
3 × 3 × 3



Feature Map

3 × 3

- 每个通道都有一个卷积核，结果取所有通道卷积结果的和

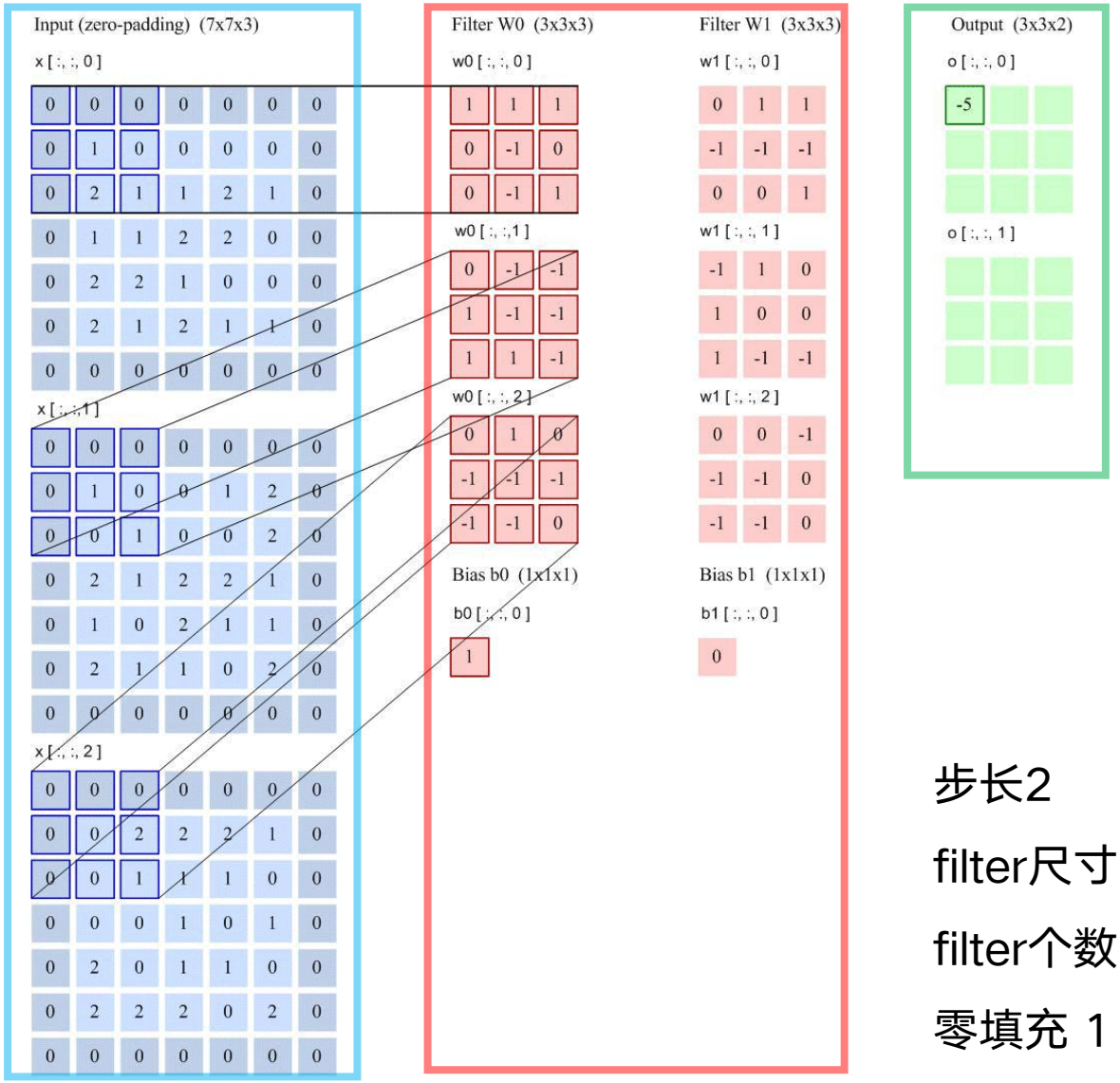


$$(1\times1+2\times2+4\times3+5\times4)$$
$$+(0\times0+1\times1+3\times2+4\times3)$$
$$=56$$

1.3 深度学习模型

- 卷积神经网络CNN

多通道+多核



步长2

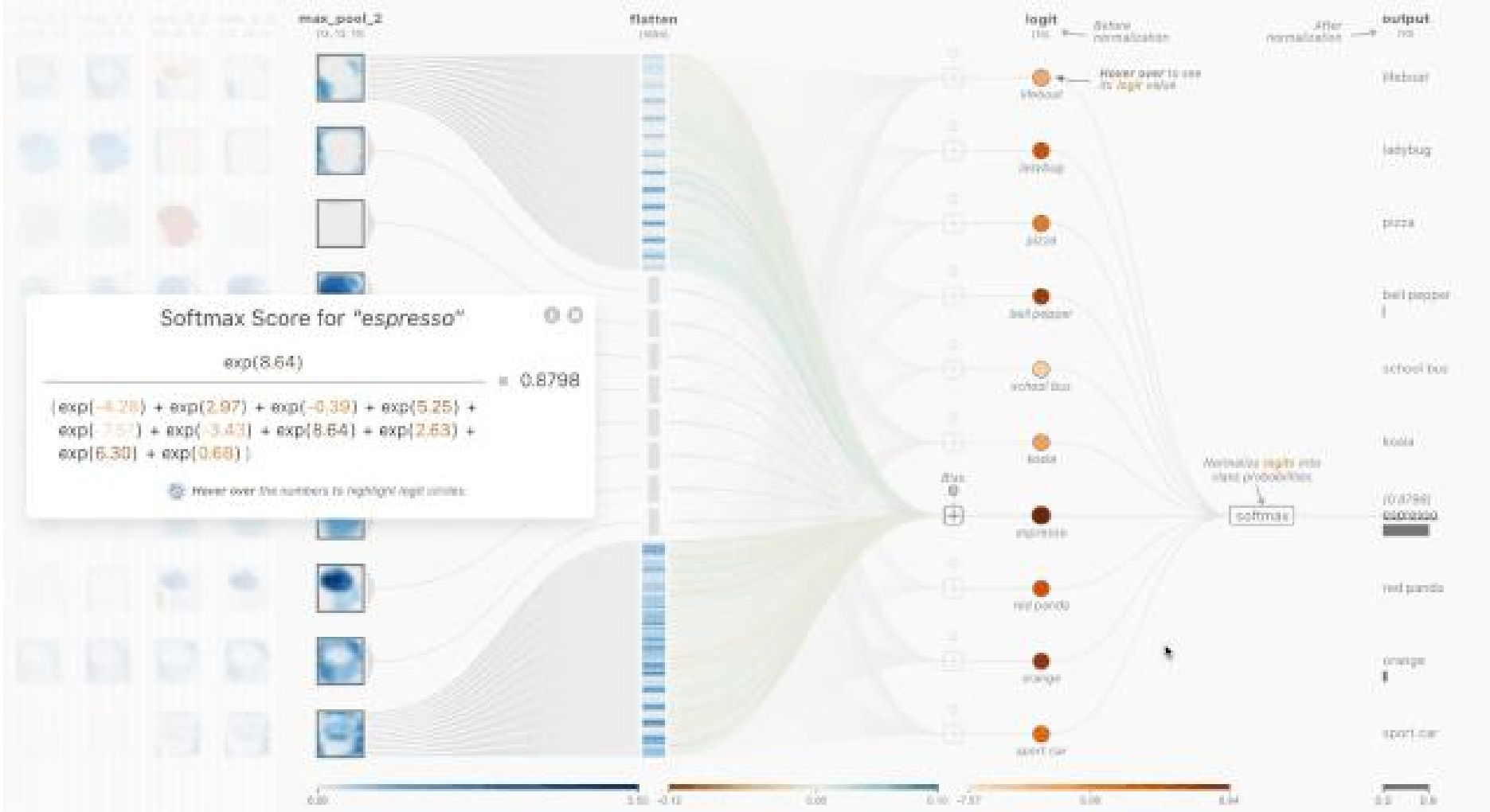
filter尺寸 3*3

filter个数(3通道*2核=6)

零填充 1

1.3 深度学习模型

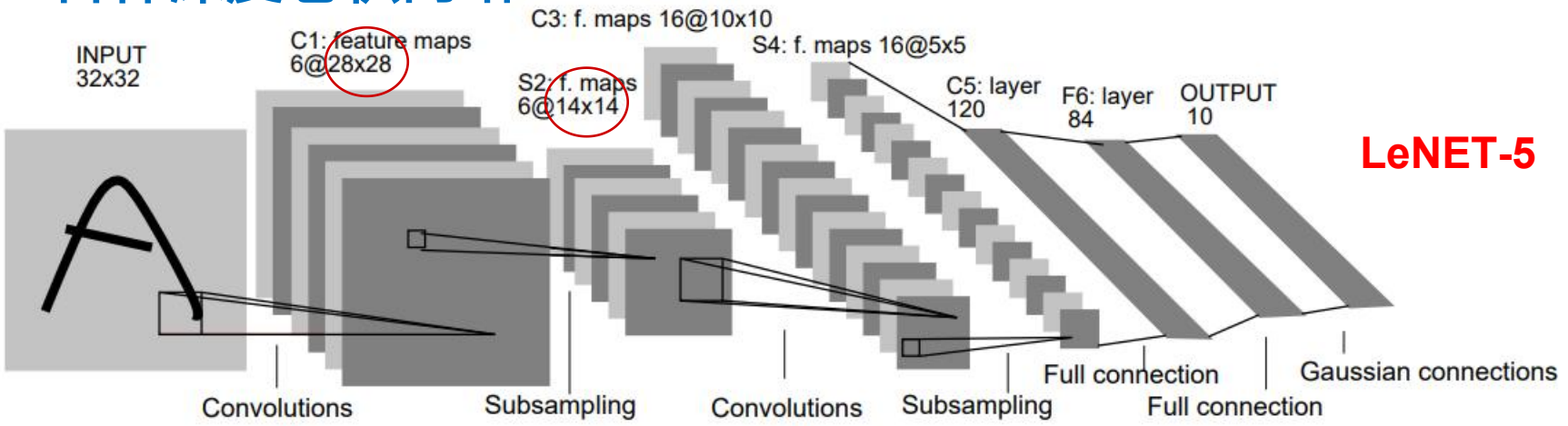
- 卷积神经网络CNN　　直观理解



CNN内部结构可视化
https://poloclub.github.io/cnn-explainer

1.3 深度学习模型

各种深度卷积网络



LeNET-5

Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

层数	层类型	对应名称	相关参数
1	输入层	INPUT	输入尺寸 32 * 32 * 1
2	第1个卷积层	C1	卷积核尺寸 5 * 5 * 6, 步长 1, 填充 **VALID**
3	第1个池化层	S2	池化核尺寸 2 * 2 * 6, 步长 2, 填充 **VALID**
4	第2个卷积层	C3	卷积核尺寸 5 * 5 * 16, 步长 1, 填充 **VALID**
5	第2个池化层	S4	池化核尺寸 2 * 2 * 16, 步长 2, 填充 **VALID**
6	第3个卷积层	C5	卷积核尺寸 5 * 5 * 120, 步长 1, 填充 **VALID**
7	第1个全连接层	F6	压扁为84个神经元
8	输出层	OUTPUT	10种分类

$$\text{输出宽度} = \frac{(\text{输入宽度} + 2 \cdot \text{填充宽度} - \text{卷积/池化核宽度})}{\text{步长}} + 1$$

$$\text{输出高度} = \frac{(\text{输入高度} + 2 \cdot \text{填充高度} - \text{卷积/池化核高度})}{\text{步长}} + 1$$

$$\text{输出通道数} = \text{卷积/池化核数量}$$

CNN参数理解

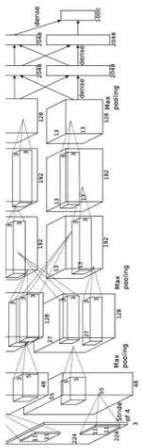
1.3 深度学习模型

- 各种深度卷积网络

ImageNet测试集上的
错误率对比

8 层

16.4%



AlexNet (2012)

19 层

7.3%

VGG (2014)

22 层

6.7%

GoogleNet (2014)

152 层

3.57%

Residual Net(2015)

1.3 深度学习模型

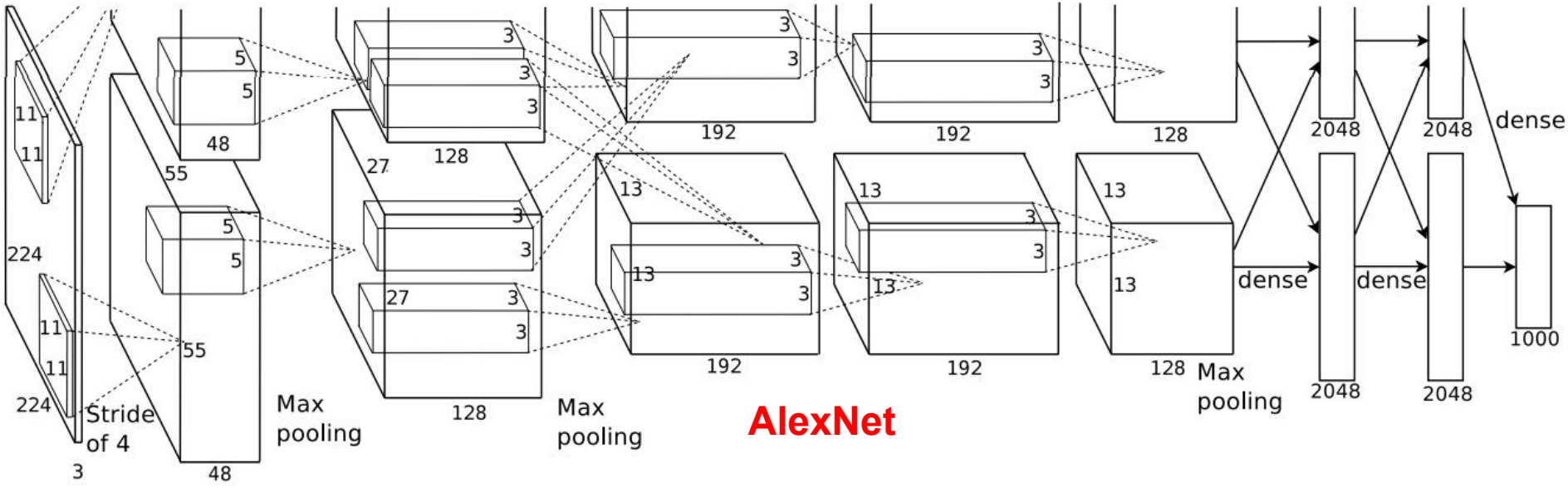
各种深度卷积网络

- ✓ AlexNet:2012 ILSVRC winner

  （top 5 error of 16\% compared to runner-up with 26\% error）

  共有8层，其中前5层卷积层，后边3层全连接层

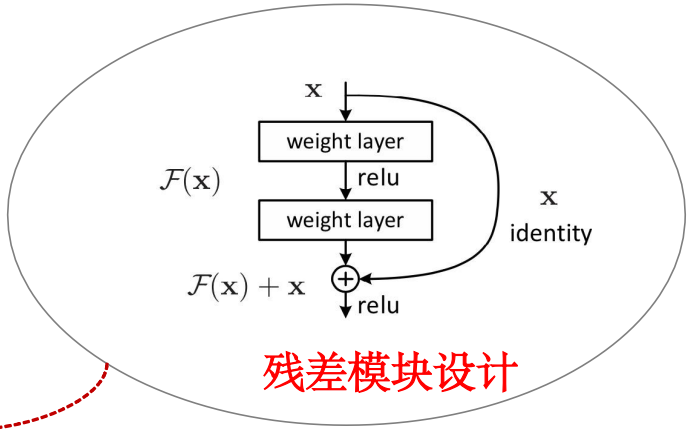
Layer	Type	Size	Filter Size	Stride	Padding	Activation
Input	Input	224 × 224	-	-	-	-
CONV 1	Convolution	55 × 55	11 × 11	4	0	ReLU
Max Pool 1	Max Pooling	27 × 27	3 × 3	2	-	-
CONV 2	Convolution	27 × 27	5 × 5	1	2	ReLU
Max Pool 2	Max Pooling	13 × 13	3 × 3	2	-	-
CONV 3	Convolution	13 × 13	3 × 3	1	1	ReLU
CONV 4	Convolution	13 × 13	3 × 3	1	1	ReLU
CONV 5	Convolution	13 × 13	3 × 3	1	1	ReLU
Max Pool 3	Max Pooling	6 × 6	3 × 3	2	-	-
FC	Fully Connected	4,096	-	-	-	ReLU
FC	Fully Connected	4,096	-	-	-	ReLU
Output	Fully Connected	1,000	-	-	-	Softmax

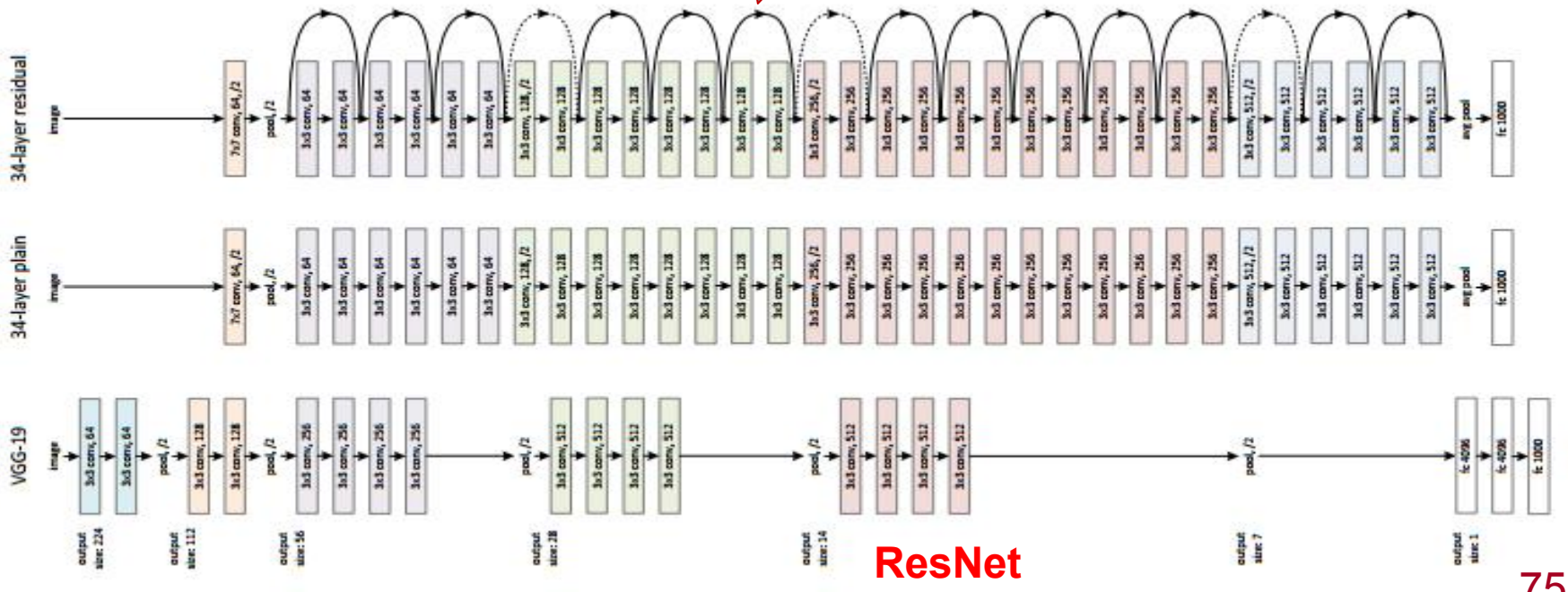


1.3 深度学习模型

各种深度卷积网络

- **ResNet: 2015 ILSVRC**
  winner （152层）
  错误率：3.57%





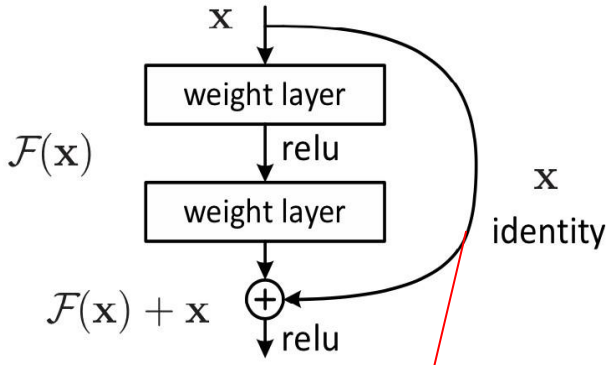
1.3 深度学习模型

■ 各种深度卷积网络

残差模块为什么有效?

问题背景：深度网络的“退化”

- 理论上，更深的网络至少不应比浅层网络差。但实验发现 当网络深度增加到一定程度（如 >20 层），训练误差反而上升——并非过拟合，而是优化困难，称为 **退化问题**



核心思想：

- 不直接学习目标映射 H(x)，而是学习残差映射 F(x)=H(x)−x
- 标准网络：希望几层堆叠后输出 H(x)
- 残差网络：让堆叠层学习 F(x)，最终输出为：

$$H(x) = F(x) + x$$

其中 x 通过捷径连接（shortcut/skip connection）直接加到输出上

优势：

- 在反向传播时，梯度可通过 shortcut 直接回传到浅层：$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial H} \cdot \left(\frac{\partial F}{\partial x} + 1 \right)$

即使 $\frac{\partial F}{\partial x} \to 0$，梯度仍能通过 +1 项传递，避免完全消失

- 加速收敛：梯度更稳定，训练更快
- 模块化设计，残差块可灵活堆叠

1.3 深度学习模型

■ 卷积神经网络的特性

优势	说明
✅ 参数效率高	局部连接 + 权值共享 → 参数远少于全连接网络（例如：3×3 卷积 vs 全连接）
✅ 自动特征提取	无需手工设计特征（如 SIFT、HOG），网络自动学习最优表示
✅ 强大的空间建模能力	能有效捕捉图像中的局部结构、纹理、形状等
✅ 良好的泛化能力	在大规模数据上训练后，可迁移到相似任务（迁移学习）
✅ 硬件友好	卷积操作高度并行，适合 GPU/TPU 加速

局部连接（Local Connectivity）

•每个神经元只与输入的局部区域（感受野）连接，而非全连接。

•大幅减少参数量，避免过拟合。

权值共享（Weight Sharing）

•同一个卷积核在整个输入上滑动，参数共享。

•进一步减少参数，提升平移鲁棒性。

平移等变性 / 平移不变性

•卷积层具有平移等变性（输入平移 → 特征图平移）；

•池化/全连接层引入平移不变性（小范围平移不影响最终分类）。

1.3 深度学习模型

■ 卷积神经网络的特性

1. 对几何变换敏感

- 缺乏旋转、缩放、仿射不变性：标准 CNN 无法识别旋转后的物体（除非数据增强覆盖该角度）。
- *解决方案：*数据增强、STN（空间变换网络）、等变 CNN。

2. 长距离依赖建模弱

- 卷积核感受野有限，难以建模图像中远距离像素的关系（如“天空在顶部，地面在底部”）。
- *解决方案：*引入注意力机制。

3. 空间信息在深层丢失

- 多次下采样（stride/pooling）导致精确定位能力下降，不利于分割、关键点检测。
- *解决方案：*编码器-解码器结构（如 U-Net）、空洞卷积、特征金字塔。

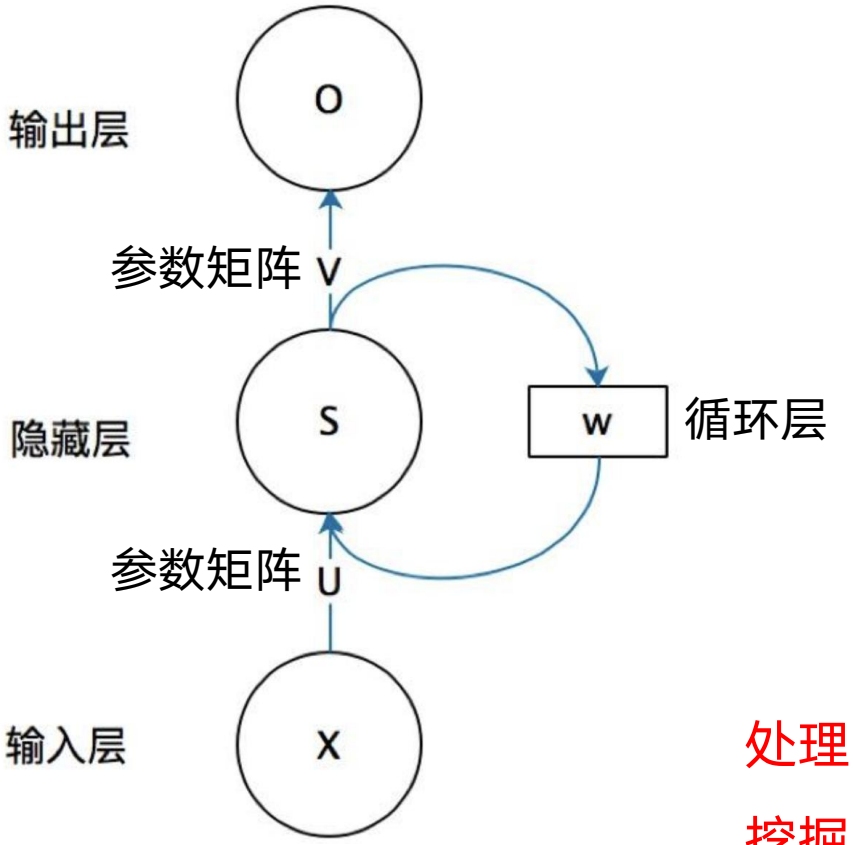
4. 固定输入尺寸限制（传统 CNN）

- 全连接层要求固定输入尺寸。
- *现代改进：*用全局平均池化（GAP）替代 FC 层，支持任意尺寸输入。

1.3 深度学习模型

循环神经网络RNN（Recurrent Neural Network）

循环神经网络（Recurrent neural Network，RNN）是一种神经元节点定向连接成环的人工神经网络。这种网络的内部状态可以展示动态时序行为，可以利用其内部的记忆来学习任意时序的输入序列。

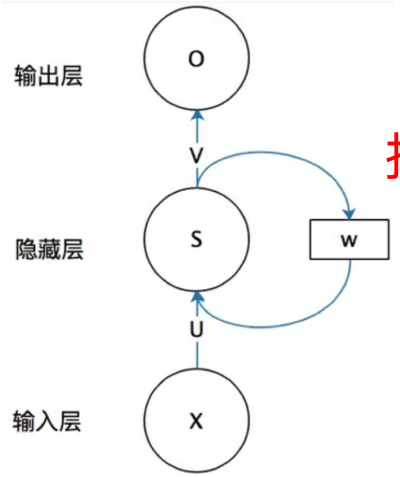


处理序列数据

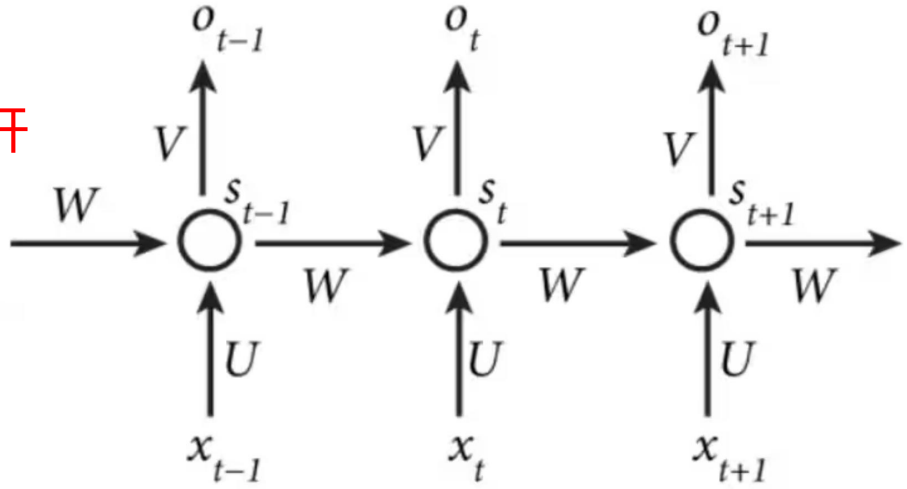
挖掘上下文的语义关系

1.3 深度学习模型

循环神经网络RNN（Recurrent Neural Network）



按照时间步展开



- X_t是时间t处的输入
- S_t是时间t处的“记忆”，$S_t=f(UX_t+WS_{t-1})$，f可以是tanh等
- O_t是时间t处的输出，比如是预测下个词的话，可能是softmax输出的属于每个候选词的概率，$O_t = \text{softmax}(VS_t)$

$$O_t = g(V \cdot S_t)$$

$$S_t = f(U \cdot X_t + W \cdot S_{t-1})$$

f, g为激活函数

1.3 深度学习模型

■ 循环神经网络RNN（Recurrent Neural Network）

典型应用场景

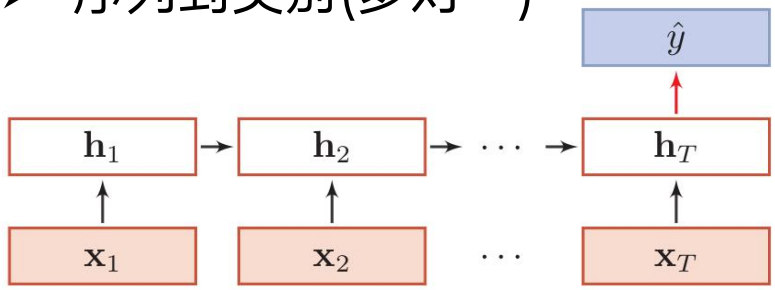
任务类型	输入 → 输出	示例
一对一	固定 → 固定	图像分类（非序列，一般不用 RNN）
一对多	固定 → 序列	图像描述生成（Image Captioning）
多对一	序列 → 固定	情感分析（整句 → 正/负）
多对多（同步）	序列 → 序列（等长）	词性标注、视频帧分类
多对多（异步）	序列 → 序列（不等长）	机器翻译（Encoder-Decoder）

RNN 最擅长的是多对一和多对多（尤其带 Encoder-Decoder）任务。

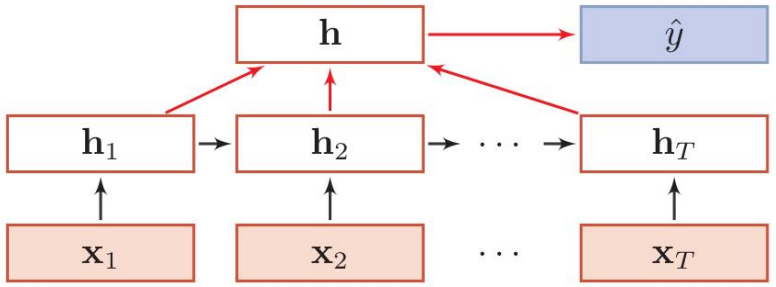
1.3 深度学习模型

循环神经网络RNN（Recurrent Neural Network）

- 序列到类别(多对一)

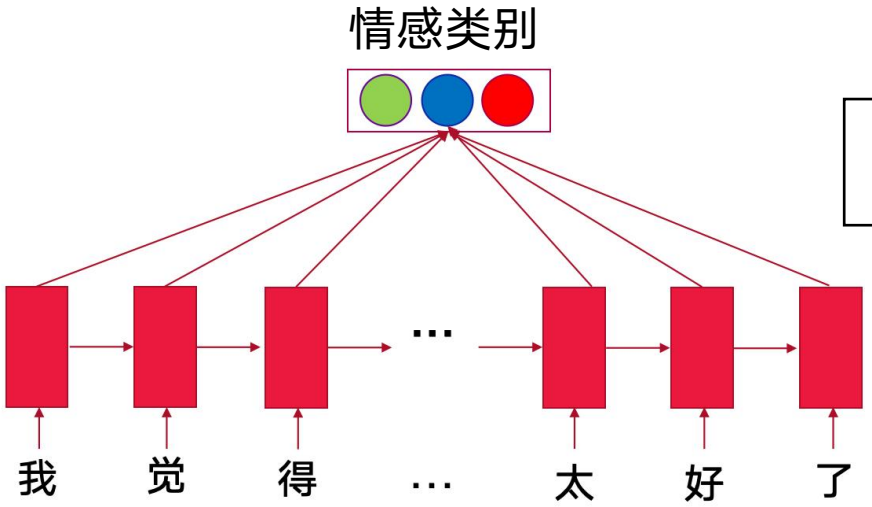


(a) 正常模式



(b) 按时间进行平均采样模式

如：文字情感分类



Positive (正面)

这部电影很棒

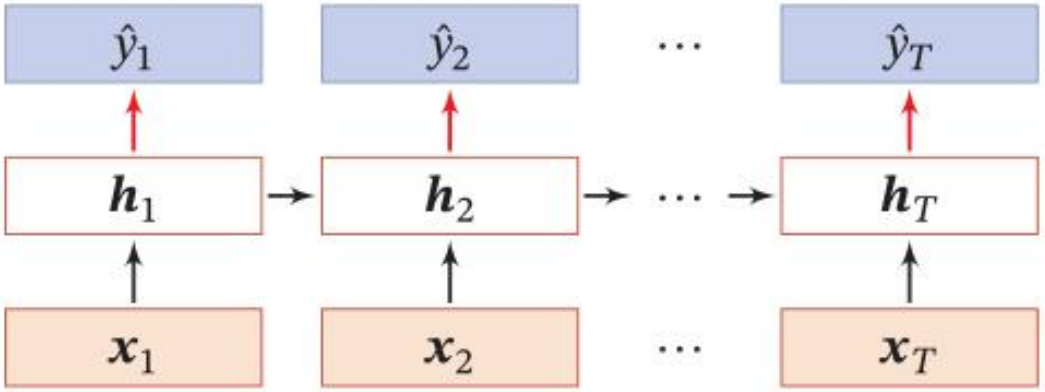
Negative (负面)

这部电影太糟了

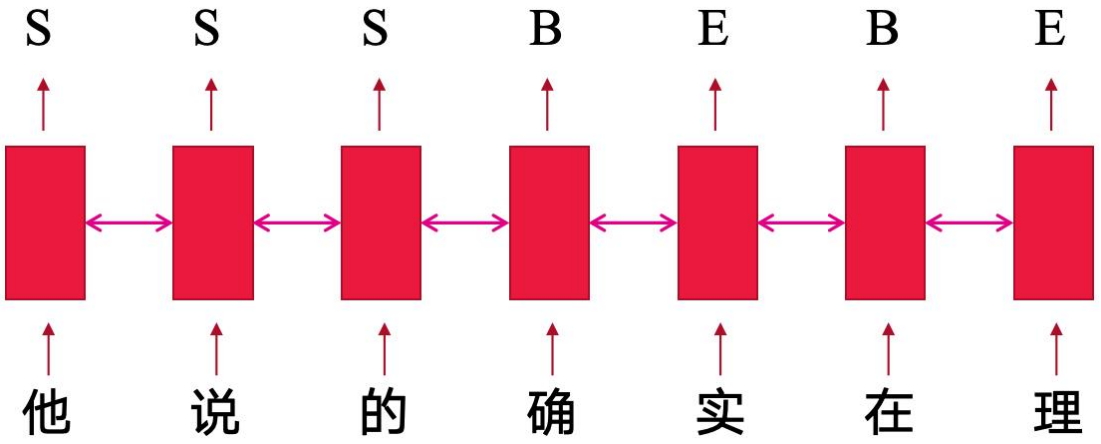
1.3 深度学习模型

循环神经网络RNN（Recurrent Neural Network）

- 同步的序列到序列模式(多对多)



如：中文分词



S 单独成词（单字词）

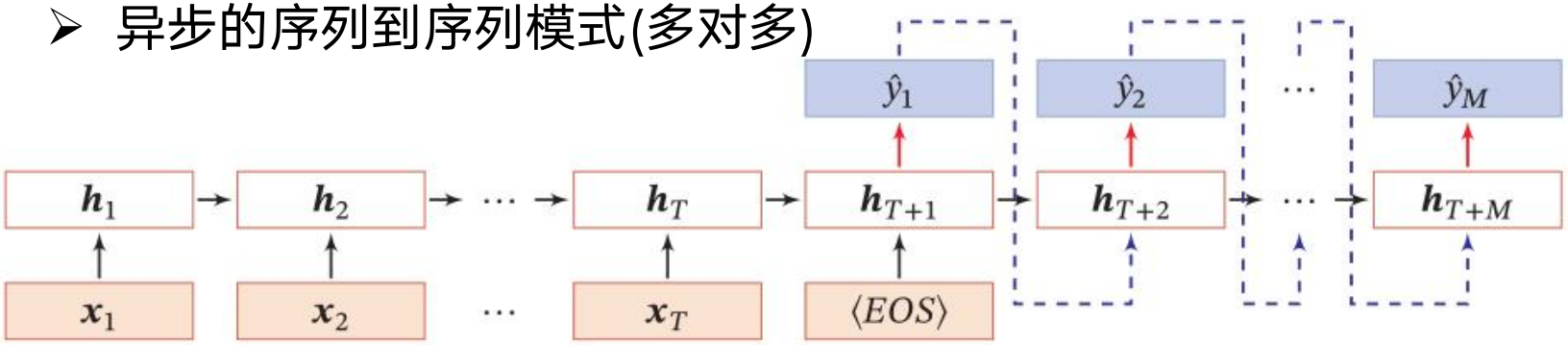
B 词语的开始

E 词语的结束

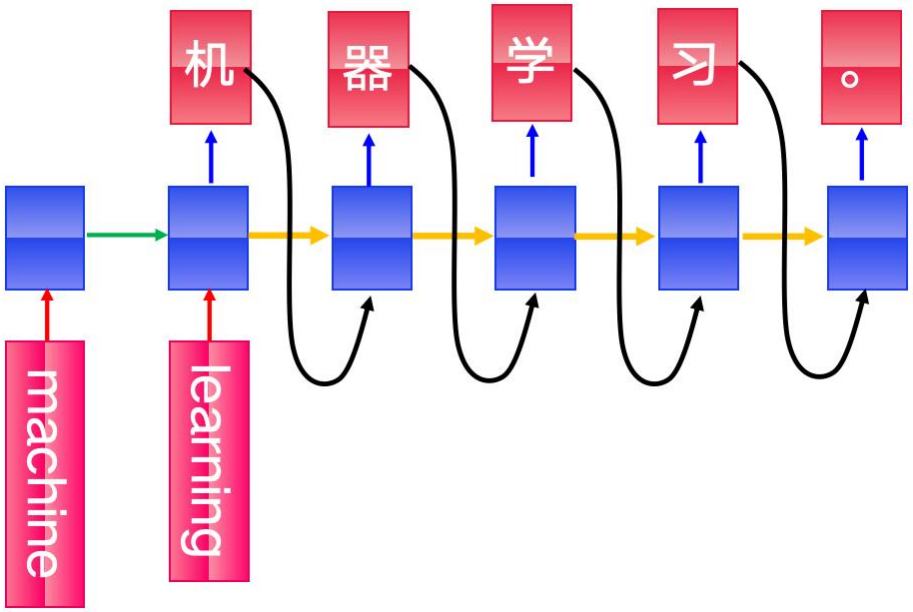
1.3 深度学习模型

循环神经网络RNN（Recurrent Neural Network）

- 异步的序列到序列模式(多对多)



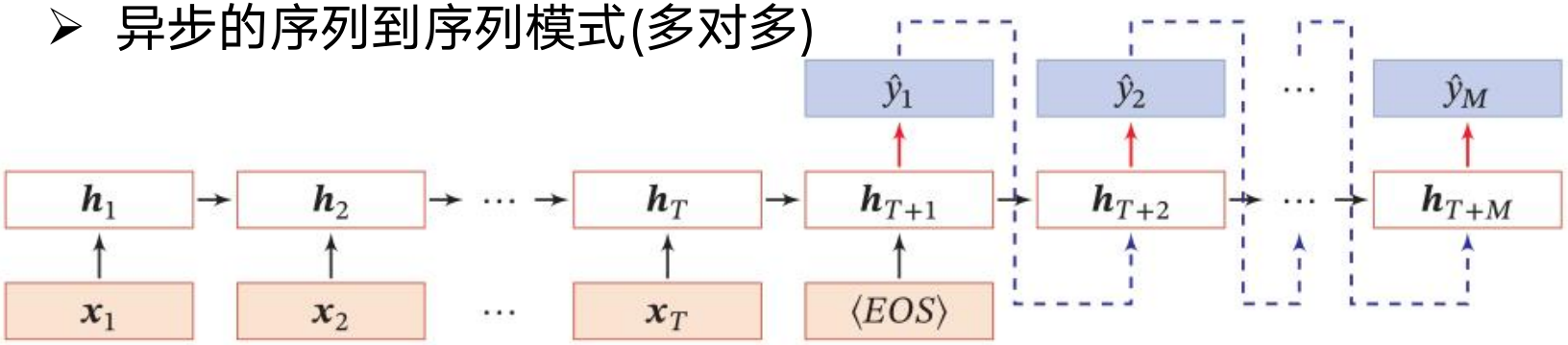
如：机器翻译



1.3 深度学习模型

■ 循环神经网络RNN（Recurrent Neural Network）

➢ 异步的序列到序列模式(多对多)



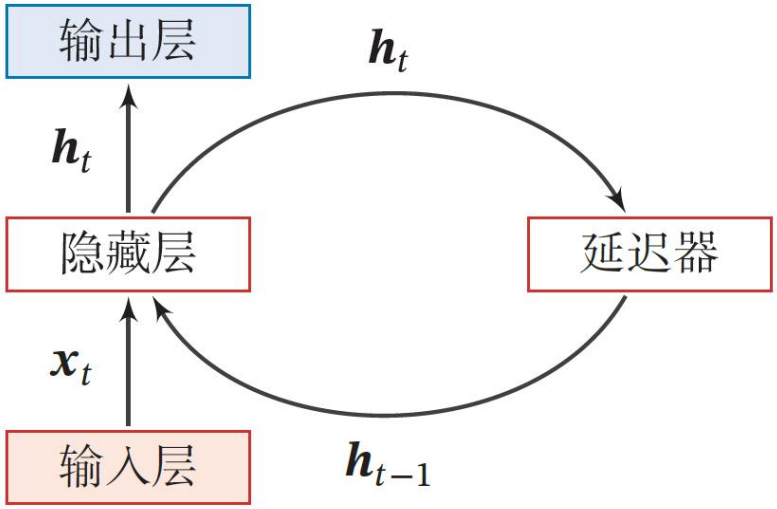
如：作词机

□ RNN在“学习”过汪峰全部作品后自动生成的歌词
https://github.com/phunterlau/wangfeng-rnn

我在这里中的夜里
就像一场是一种生命的意叶
就像我的生活变得在我一样
可我们这是一个知道
我只是一天你会怎吗
可我们这是我们的是不要为你
我们想这有一种生活的时候

1.3 深度学习模型

■ 循环神经网络RNN（Recurrent Neural Network）

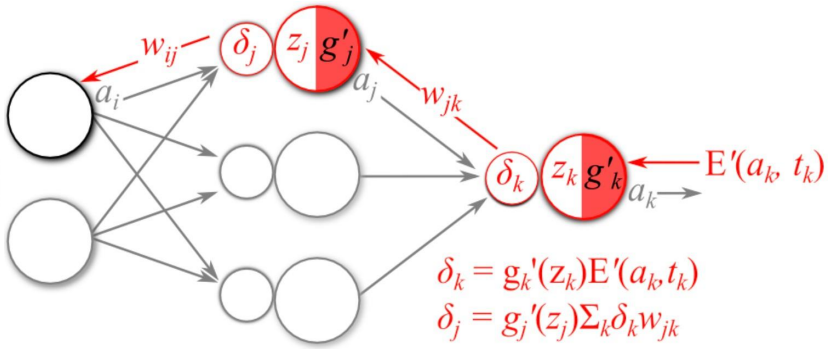


$$h_t = f(Uh_{t-1} + Wx_t)$$

$$\hat{y}_t = softmax(W^{(s)}h_t)$$

模型： $y = f^5(f^4(f^3(f^2(f^1(x)))))$

损失函数： $L(y, \hat{y})$



链式梯度优化： $\frac{\partial L(y,\hat{y})}{\partial f^1} = \frac{\partial f^2}{\partial f^1} \times \frac{\partial f^3}{\partial f^2} \times \frac{\partial f^4}{\partial f^3} \times \frac{\partial f^5}{\partial f^4} \times \frac{\partial L(y,\hat{y})}{\partial f^5}$

当时间维度非常深时，同样面临梯度消失或爆炸问题…

1.3 深度学习模型

■ 循环神经网络RNN（Recurrent Neural Network）

RNN特性

特性	说明
✅ 处理变长序列	不要求输入/输出长度固定（通过 padding 或动态计算）
✅ 参数共享	同一套权重处理所有时间步，极大减少参数量
✅ 具有记忆能力	隐藏状态 h_t 理论上可编码整个历史信息
✅ 时间依赖建模	能捕捉序列中的前后依赖关系（如语法、趋势）
⚠️ 计算串行	无法像 CNN 那样完全并行（h_t 依赖 h_{t-1}）

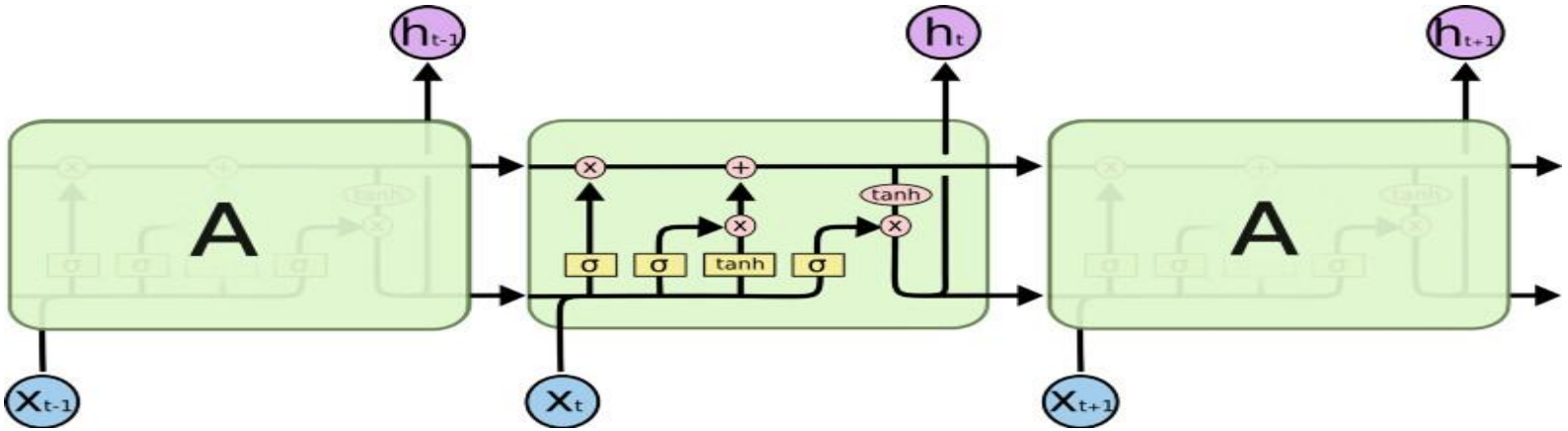
- 易出现梯度消失与梯度爆炸问题
- 短期记忆问题（5~10步）

1.3 深度学习模型

RNN变体-长短时记忆网络LSTM Long Short-Term Memory

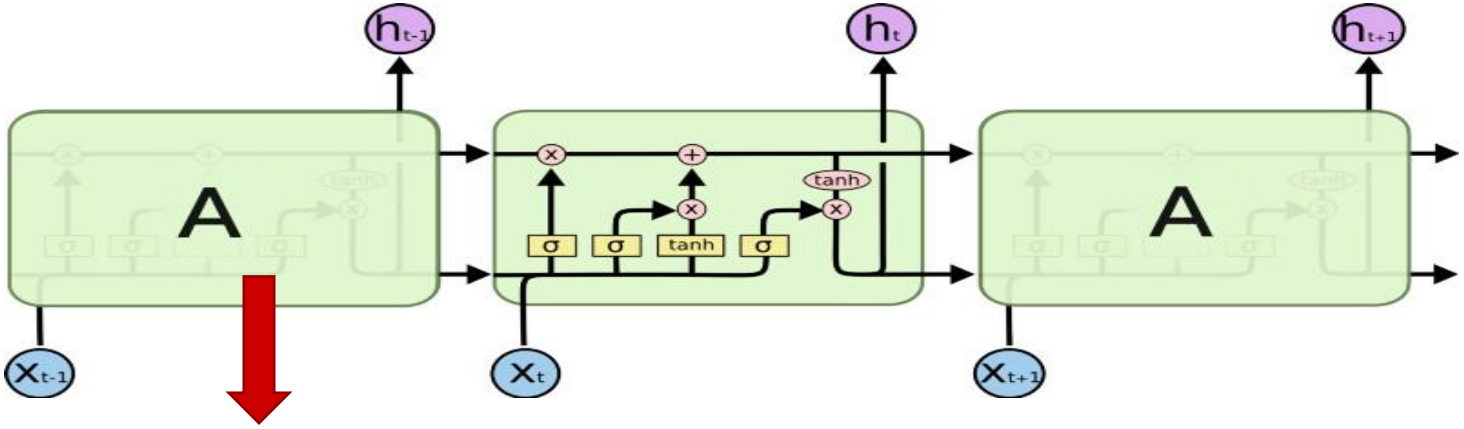
LSTM 的核心思想：不是让所有信息都无差别地传递，而是通过‘门’来决定：哪些信息要保留，哪些要丢弃，哪些要输出

- 标准 RNN 的隐藏状态更新是“全有或全无”的，容易遗忘早期信息；
- LSTM 引入了细胞状态（Cell State）作为“主干记忆通道”，并通过三个门（Gate）来调控信息的写入、删除和读取；
- 这种设计使得 LSTM 能够选择性地长期保存重要信息，同时过滤无关噪声。



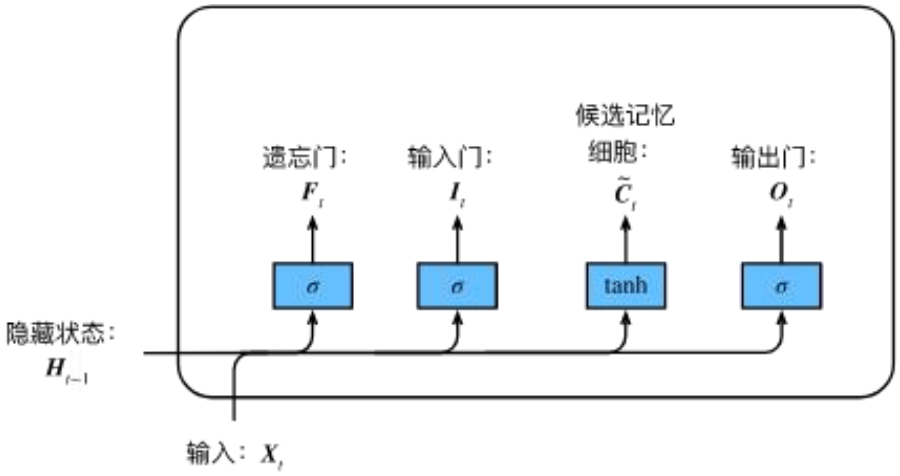
1.3 深度学习模型

RNN变体-长短时记忆网络LSTM



改进方案

门控机制：输入门、遗忘门、输出门



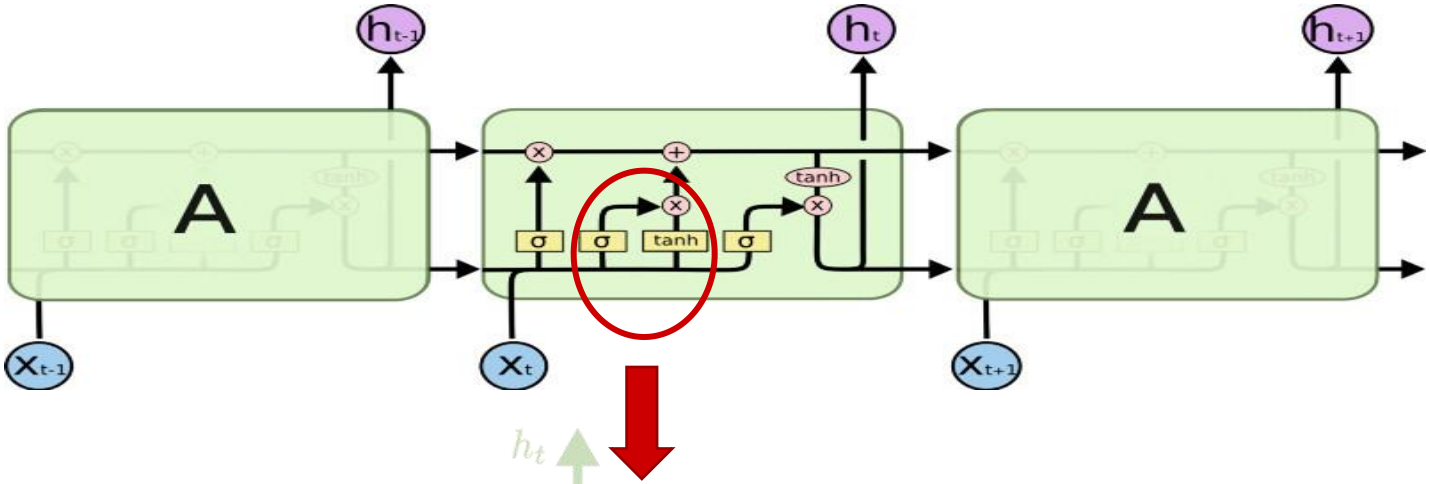
核心公式

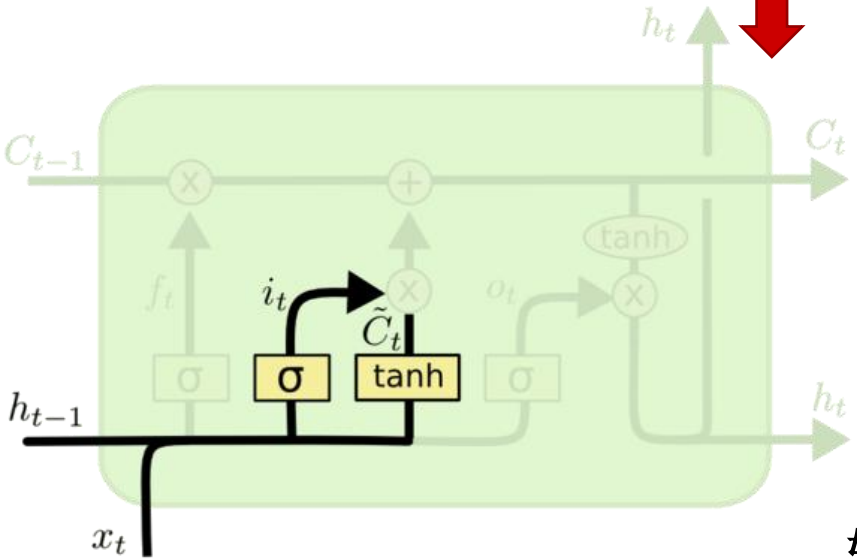
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

σ 全连接层和激活函数　按元素运算符　复制　连结

1.3 深度学习模型

- RNN变体-长短时记忆网络LSTM





i_t 输入门

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

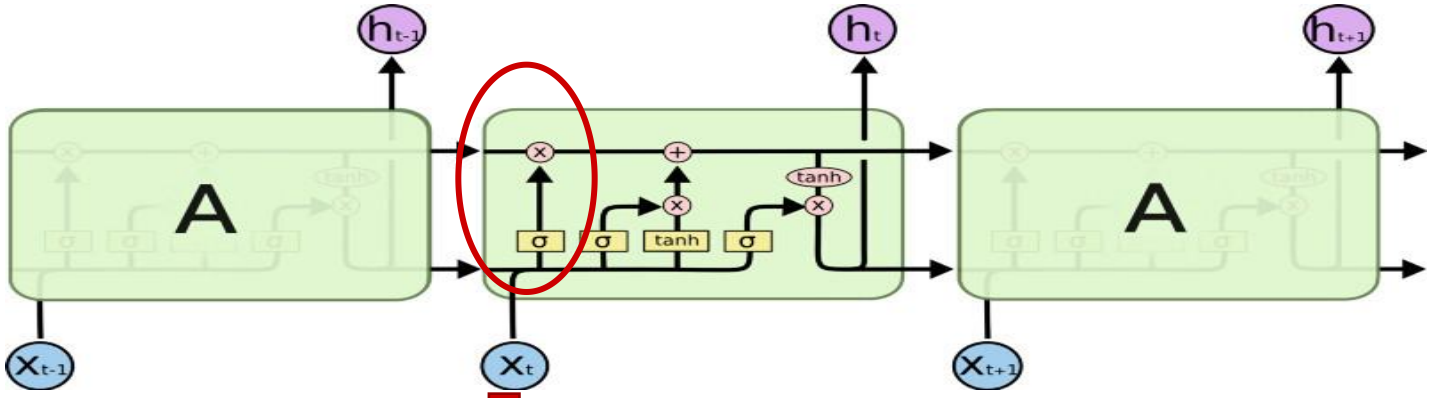
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

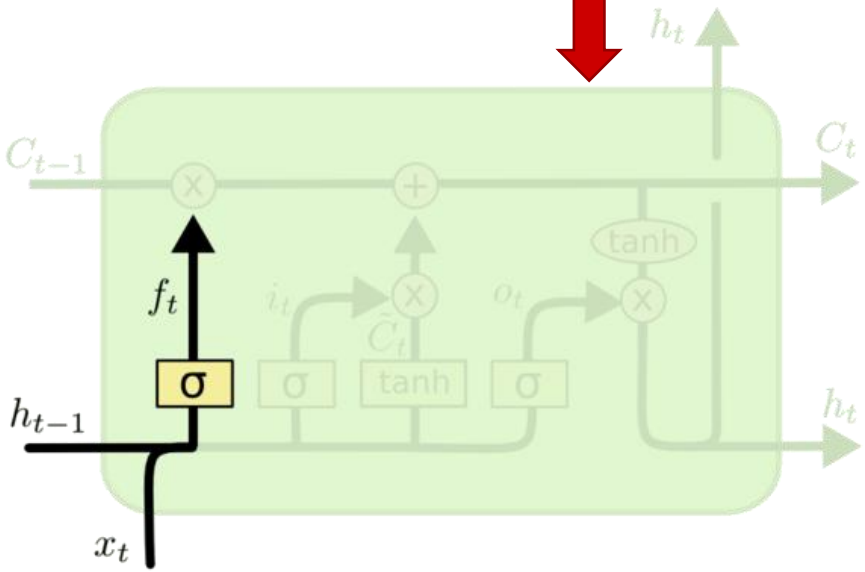
候选记忆细胞

输入门 i_t: 输出0~1
控制每一个记忆单元C_t加入多少新的信息
候选记忆[−1, +1]: 可能写入的新信息

1.3 深度学习模型

RNN变体-长短时记忆网络LSTM





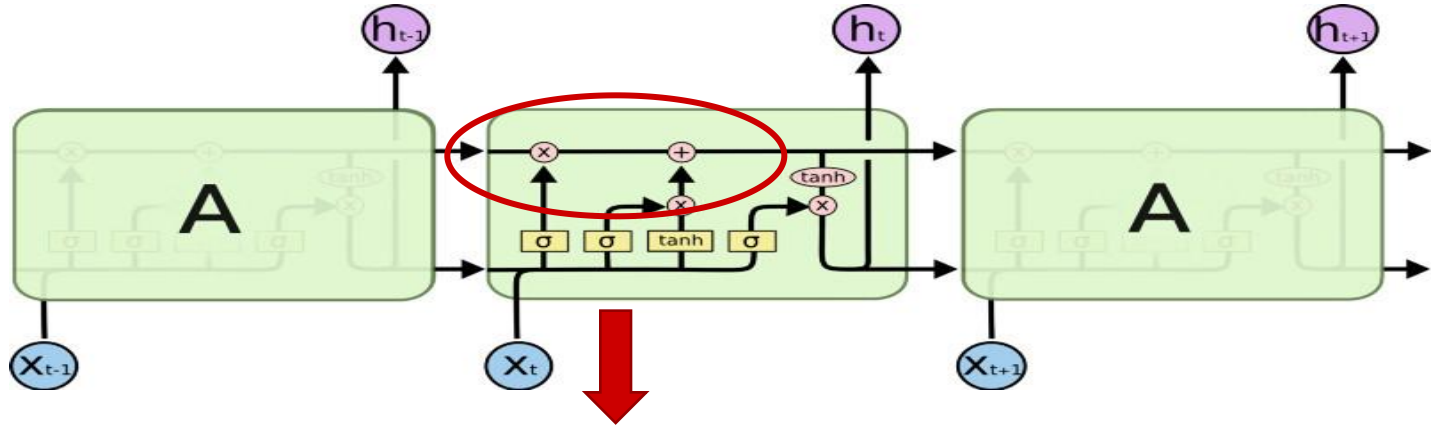
遗忘门

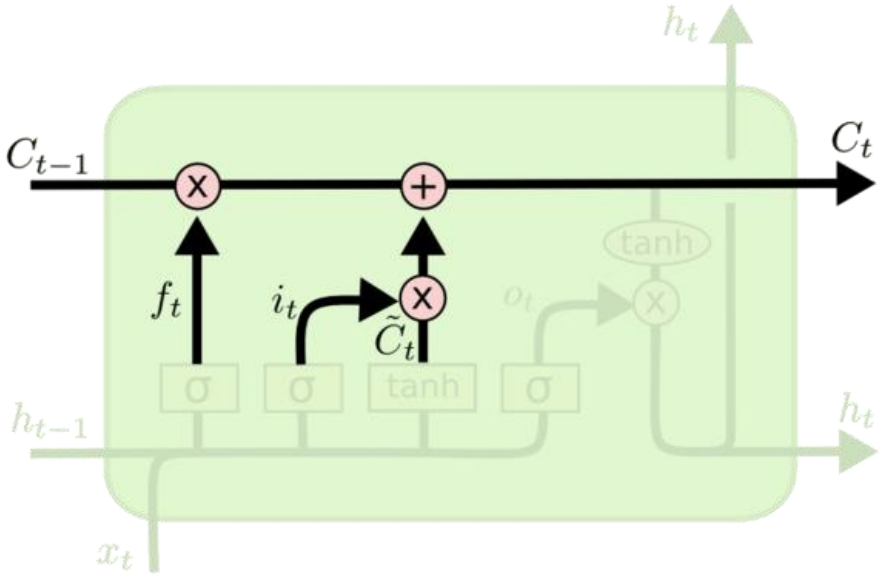
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

遗忘门 $\mathbf{f}_t$:
控制每一个记忆单元$\mathbf{C}_t$需要遗忘多少信息

1.3 深度学习模型

- RNN变体-长短时记忆网络LSTM





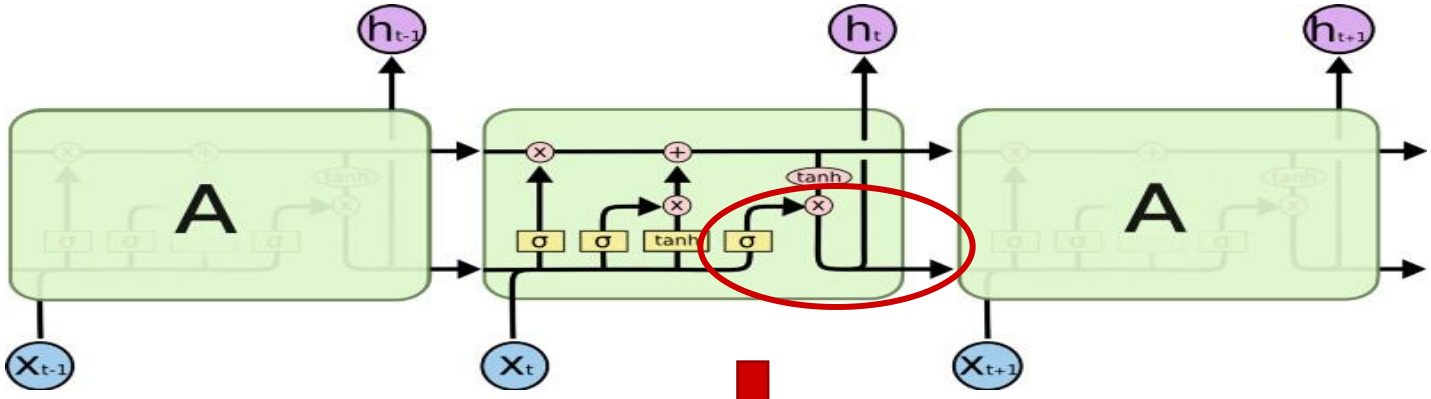
记忆单元状态更新

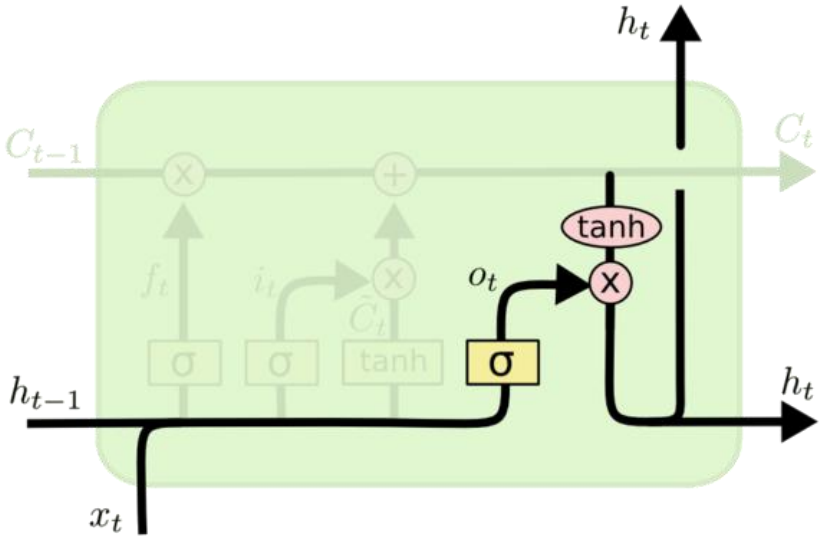
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

内部记忆单元 C_t 记录了到当前时刻为止的所有历史信息

1.3 深度学习模型

- RNN变体-长短时记忆网络LSTM





输出门

$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

外部状态

输出门$\mathbf{o}_t$:
控制每一个记忆单元$\mathbf{C}_t$输出多少信息

1.3 深度学习模型

RNN变体-长短时记忆网络LSTM

为什么使用tanh激活函数?

候选记忆细胞 $\tilde{C}_t = \tanh\left(W_C \cdot [h_{t-1}, x_t] + b_C\right)$

$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ 记忆单元的更新

1. 将值压缩到 [−1,+1]区间，保证数值稳定性

如果 $\tilde{C}_t$ 值过大(如使用 ReLU 可能输出几百)，会导致 C_t 指数级增长，引发梯度爆炸

2. 支持正负信息表示，增强表达能力

- 这意味着候选状态可以表示两种方向的信息：

  +1：增强某种特征（如“情感积极”）；

  -1：抑制某种特征（如“否定语气”）。

- 相比之下，sigmoid 只能输出 [0,1]，无法表达“负向贡献”。

3. 与细胞状态的线性更新机制相匹配

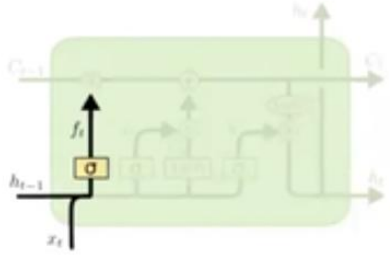
LSTM 的核心创新之一是细胞状态的线性传递路径 $C_t \approx C_{t-1} +$ （小量更新）

小量更新本身在一个有界、对称的区间内，tanh 完美满足这一要求

1.3 深度学习模型

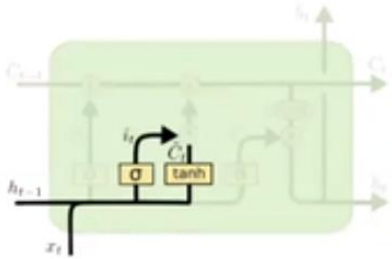
RNN变体-长短时记忆网络LSTM　Long Short-Term Memory

1. Forget gate layer



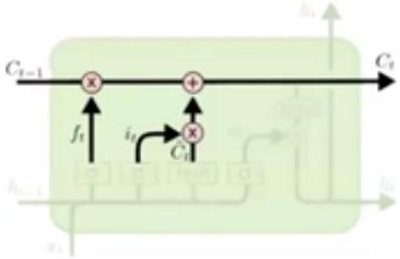
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

2. Input gate layer



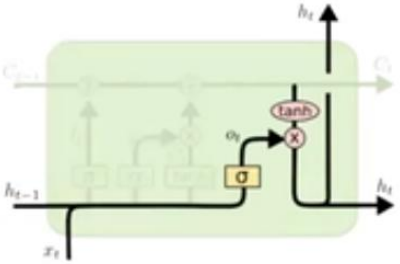
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

3. The current state



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

4. Output layer



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

LSTM小结

- 首先利用上一时刻的外部状态h_{t-1}和当前时刻的输入x_t，计算出三个门，以及候选状态$\tilde{c}_t$；
- 结合遗忘门f_t和输入门i_t来更新记忆单元c_t;
- 结合输出门o_t，将内部记忆单元状态c_t的信息传递给外部状态h_t.

1.3 深度学习模型

RNN变体-长短时记忆网络LSTM

LSTM特性

特性	说明
✅ 长期依赖建模能力强	能有效捕捉时间跨度很大的依赖关系（如几百步）
✅ 门控机制灵活	可动态决定记忆/遗忘/输出，适应不同序列模式
✅ 梯度传播稳定	细胞状态的线性更新路径使梯度不易消失
✅ 适用于多种序列任务	机器翻译、语音识别、文本生成、时间序列预测等
⚠️ 计算开销较大	每个时间步需计算 4 个线性变换（3 门 + 1 候选），比 RNN 慢
⚠️ 仍为串行计算	无法像 Transformer 那样完全并行

1.3 深度学习模型

RNN变体-长短时记忆网络LSTM

LSTM
应用领域

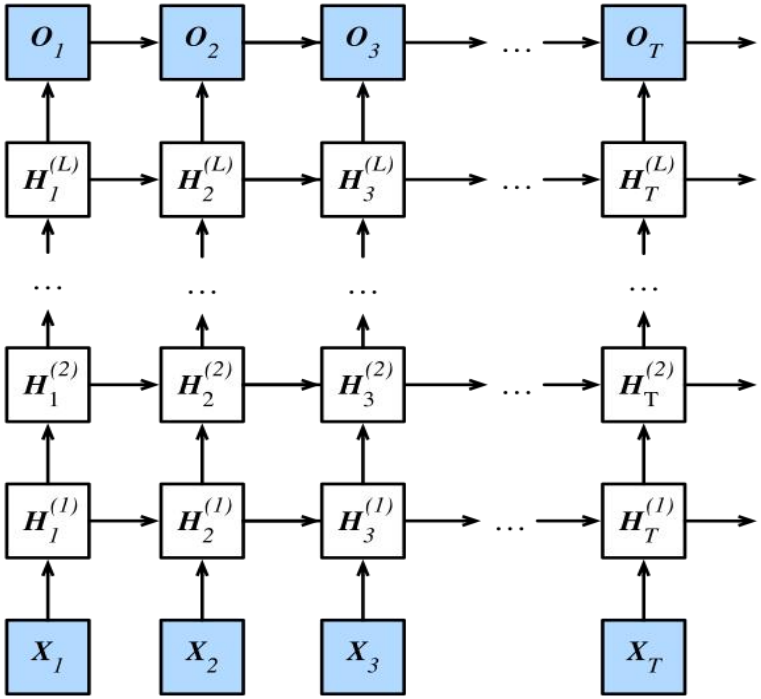
领域	应用示例
自然语言处理（NLP）	机器翻译（早期 Seq2Seq）、文本生成、情感分析
语音处理	语音识别（ASR）、语音合成（TTS）
时间序列预测	股价预测、电力负荷预测、天气预报
生物信息学	DNA 序列建模、蛋白质结构预测

LSTM
局限性

问题	说明
❌ 无法并行计算	时间步必须顺序处理，训练慢
❌ 超长序列仍困难	虽优于 RNN，但对数千步以上的依赖仍有限
❌ 缺乏全局注意力	无法像 Transformer 那样直接建模任意两位置的关系
❌ 调参复杂	学习率、初始化、梯度裁剪等需精细调整

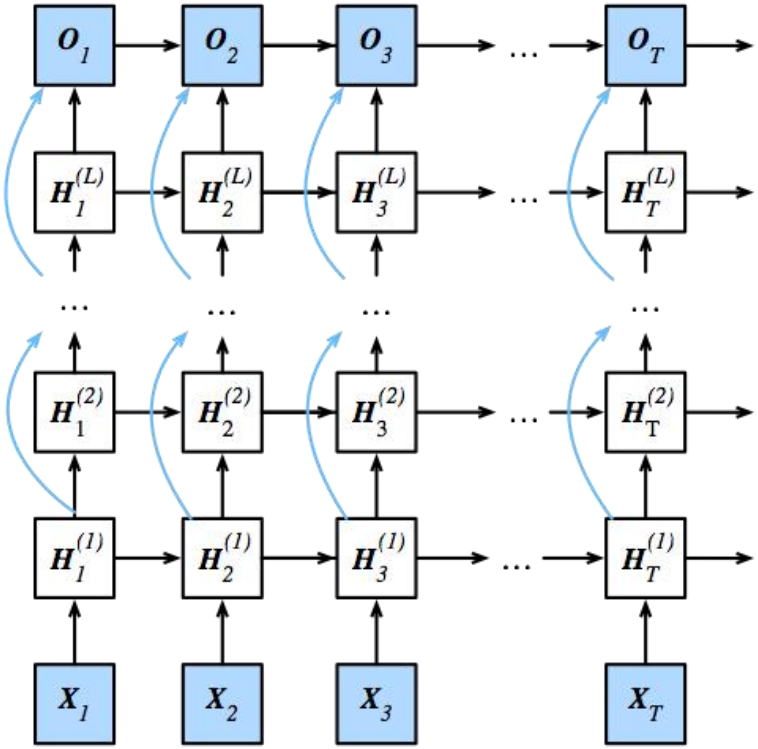
1.3 深度学习模型

- 各种丰富的RNN变体模型



Stacked RNN / Deep RNN

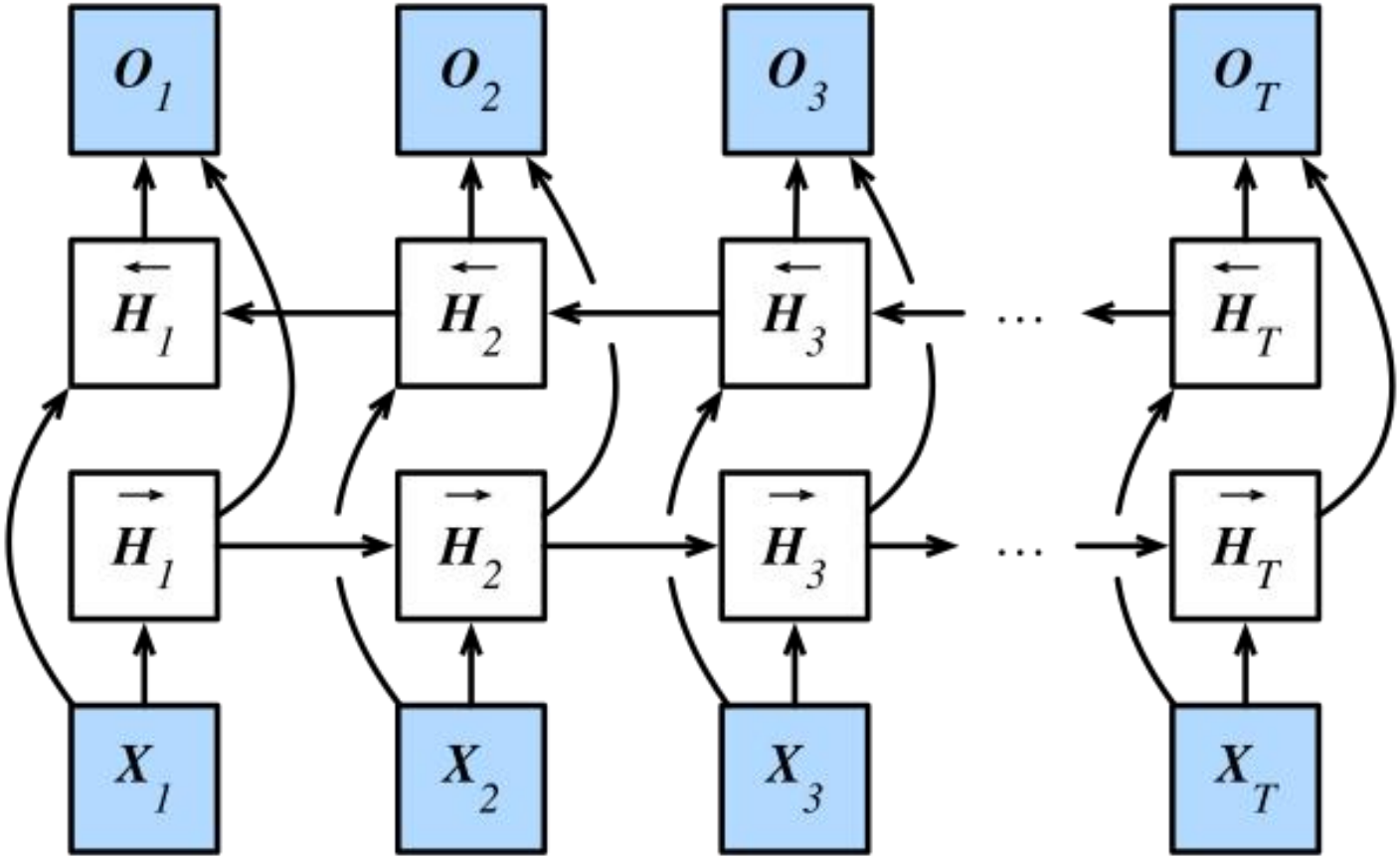
堆叠循环网络



引入残差连接，增加深度

1.3 深度学习模型

- 各种丰富的RNN变体模型

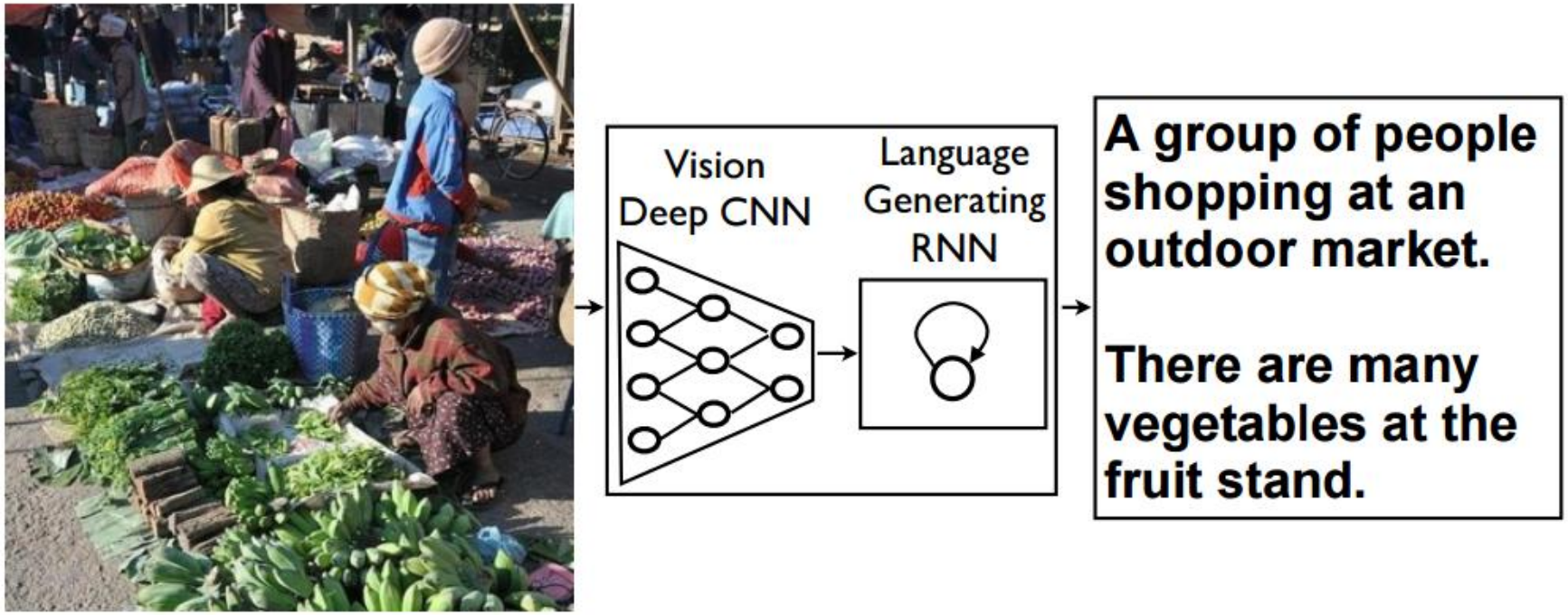


Bi-directional RNN
双向循环网络

1.3 深度学习模型

- 循环网络应用

  看图说话

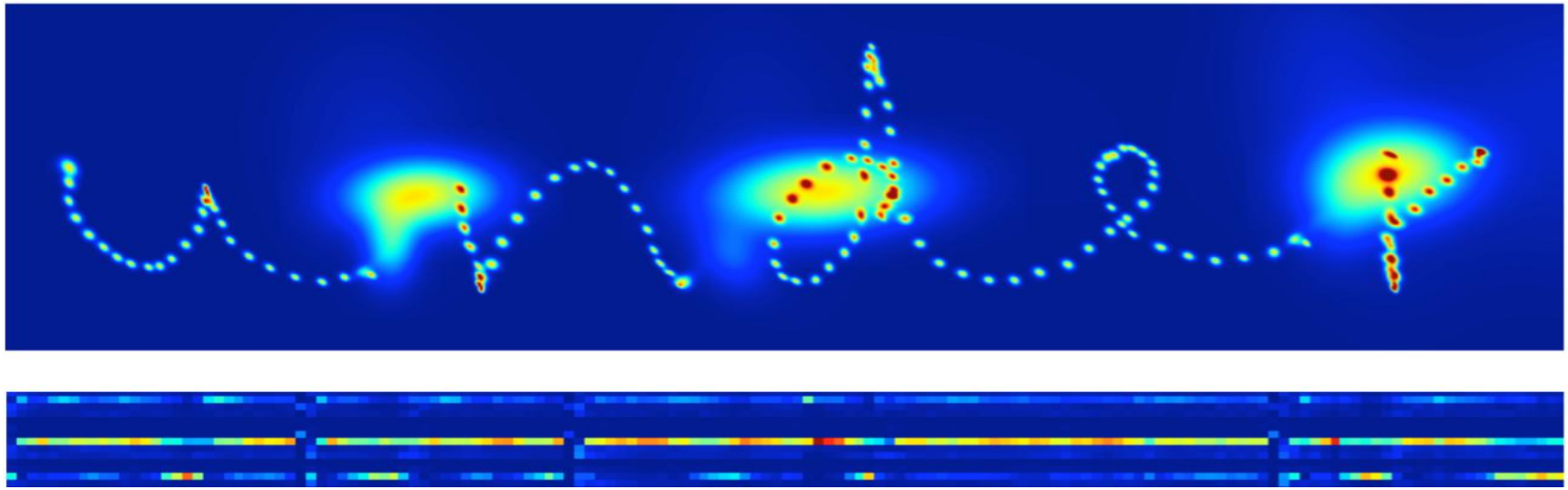


1.3 深度学习模型

循环网络应用

写字

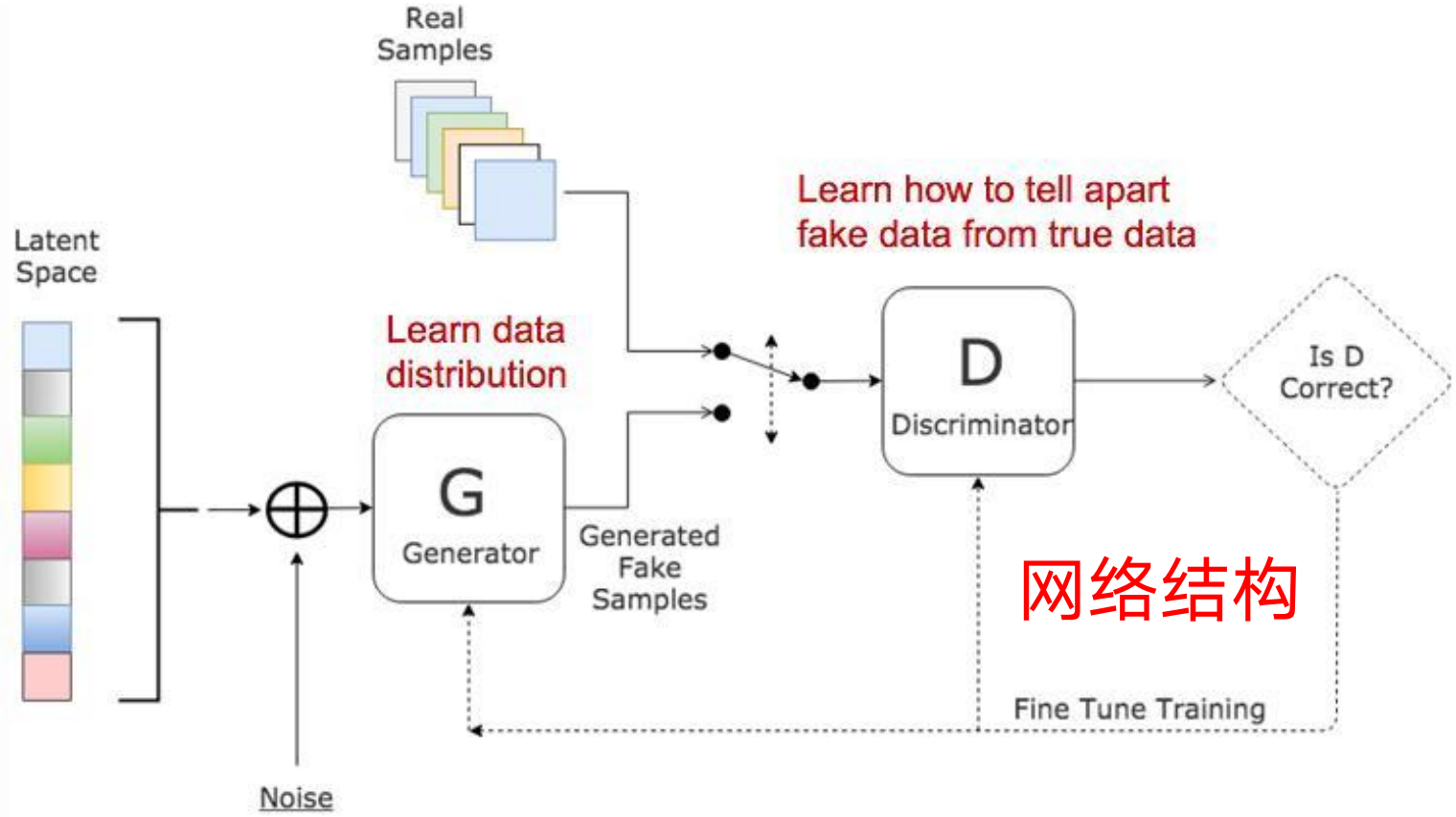
把一个字母的书写轨迹看作是一连串的点。一个字母的“写法”其实是每一个点相对于前一个点的偏移量，记为(offset x, offset y)。再增加一维取值为0或1来记录是否应该“提笔”。



1.3 深度学习模型

■ 生成式网络GAN

Generative Adversarial Networks，Goodfellow et al., 2014



- ✓ 生成对抗网络由生成网络G与判别网络D组成。G从潜在空间中随机采样作为输入，其输出结果尽量模仿训练集中的真实样本。
- ✓ D的输入则为真实样本和生成网络的输出，其目的是将G的输出从真实样本中尽可能分辨出来。G要尽可能地欺骗D，D要将G生成的样本与真实样本中尽可能区分出来。

1.3 深度学习模型

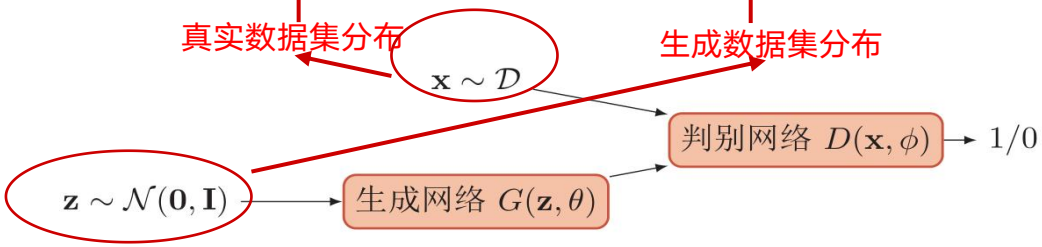
■ 生成式网络GAN

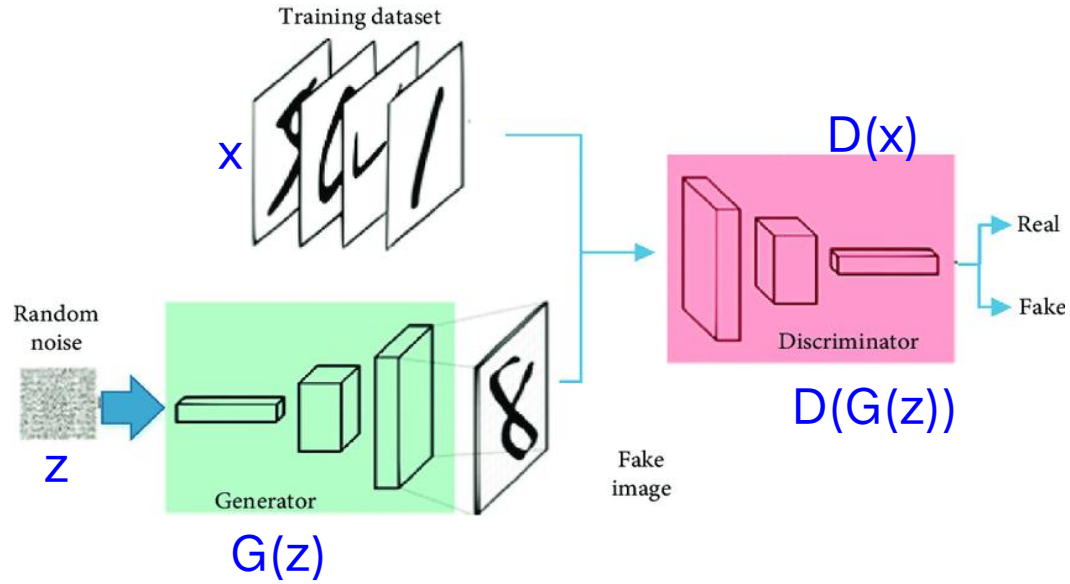
目标函数

$$\min_G \max_D V(D,G)$$ 网络优化目标

$$V(D,G) = \mathbb{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z))]$$

真实数据集分布　　生成数据集分布





- ✓ x表示真实图片，z表示输入G网络的噪声，而G(z)表示G网络生成的图片。D(x)表示D网络判断真实图片是否真实的概率。
- ✓ D(G(z))是D网络判断G生成的图片是否真实的概率，**G希望**生成的图片“越真实越好”即G希望D(G(z))尽可能大，这时V(D, G)会变小。因此目标函数对G来讲是求最小（min_G）。
- ✓ D的能力越强，**D希望**D(x)越大，而D(G(x))应该越小。这时V(D,G)会变大。因此目标函数对于D来说是求最大（max_D）

1.3 深度学习模型

■ 生成式网络GAN

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

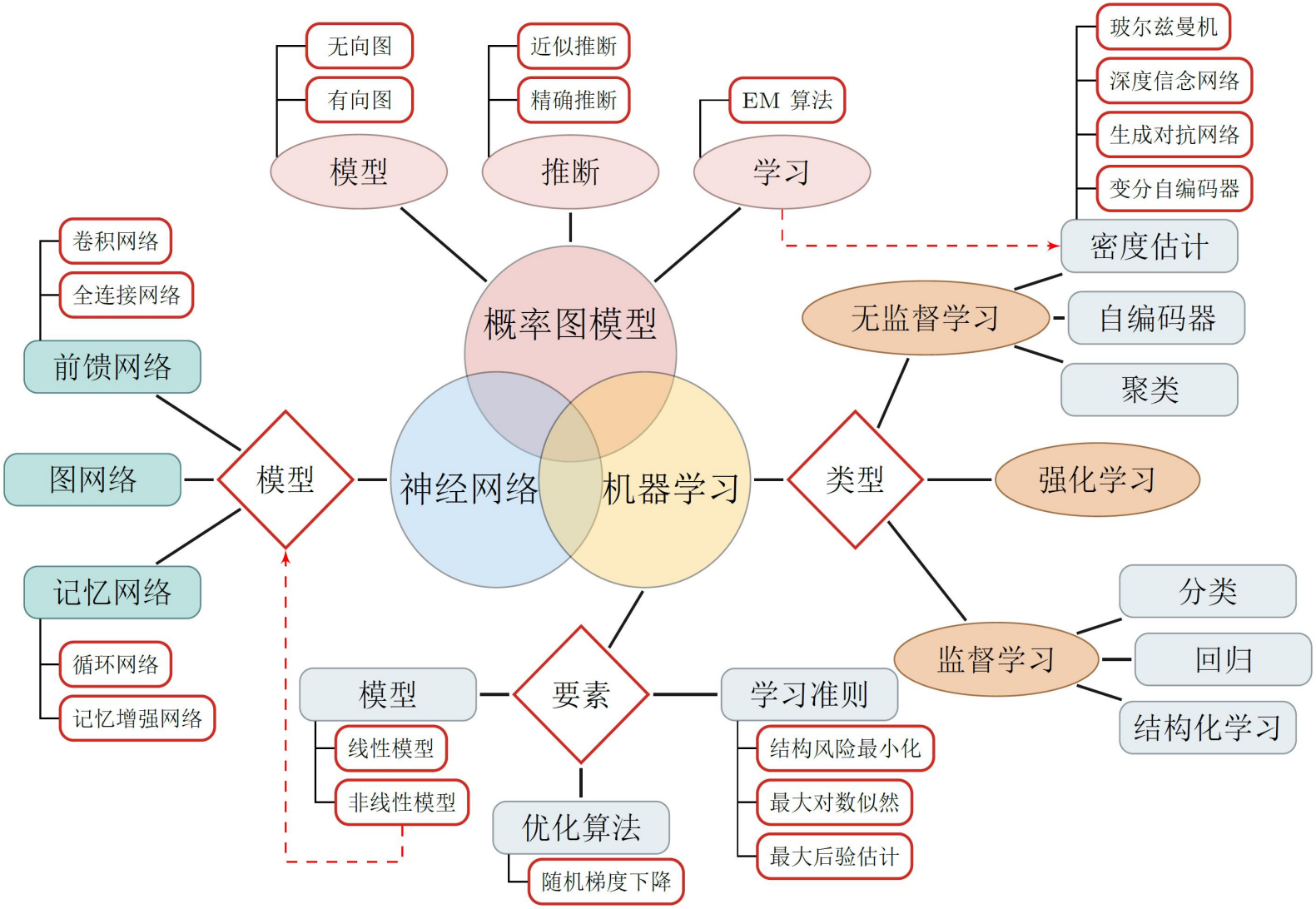
end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

训练算法

1.3 深度学习模型

深度学习模型总结



1.3 深度学习模型

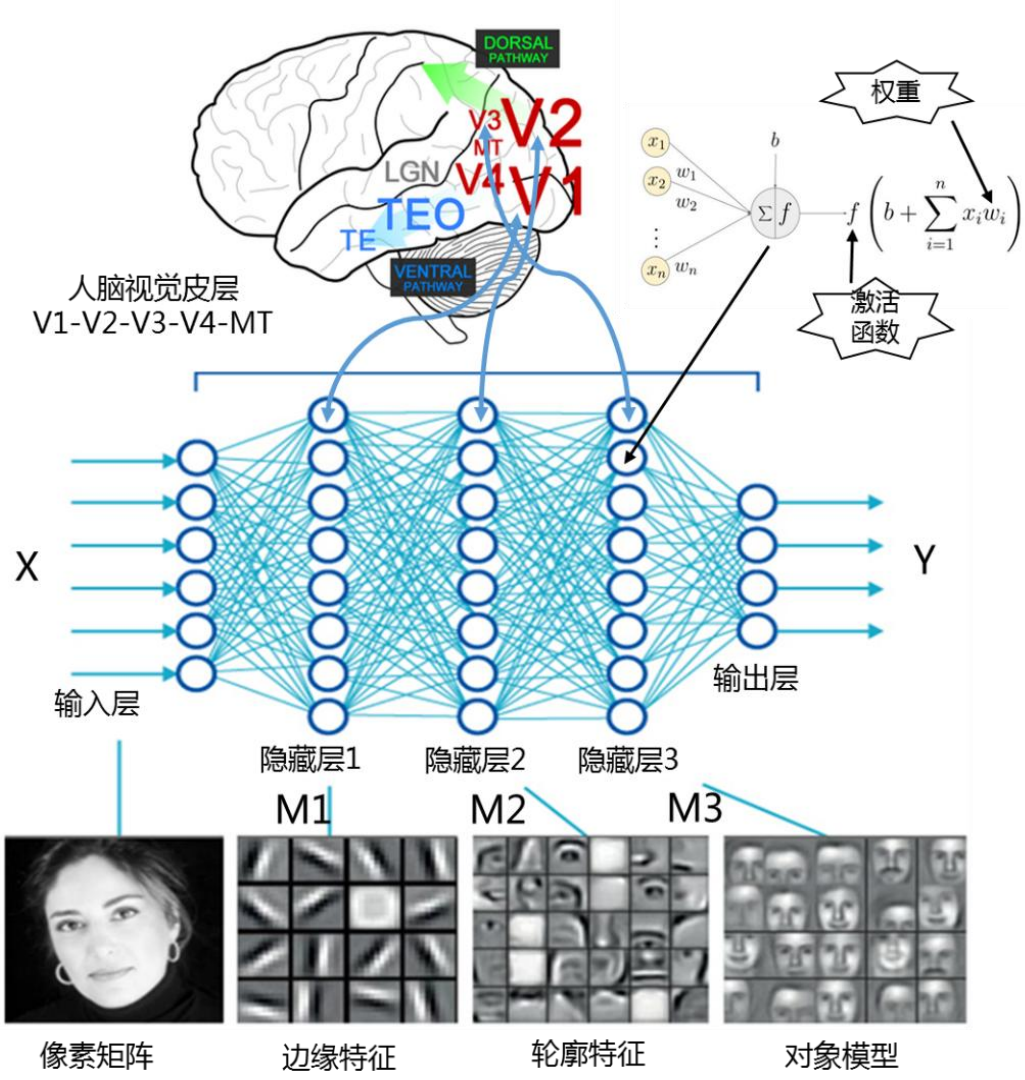
深度学习模型优化技巧

- 超参数项
  - ✓ 神经元层数
  - ✓ 每层神经元个数
  - ✓ 激活函数
  - ✓ 学习率
  - ✓ 正则化系数
  - ✓ Mini-batch 大小
  - ✓ …
- 优化方法
  - ✓ 网格搜索
  - ✓ 随机搜索
  - ✓ 贝叶斯优化
  - ✓ 动态资源分配
  - ✓ 神经架构搜索
  - ✓ …

- 优化经验-Trick
  - ✓ 用 ReLU 作为激活函数
  - ✓ SGD+mini-batch
  - ✓ 提前停止 early stop
  - ✓ 数据预处理（标准归一化）
  - ✓ 动态学习率（越来越小）
  - ✓ 用 L1 或 L2 正则化
  - ✓ 逐层归一化BN
  - ✓ dropout
  - ✓ 数据增强
  - ✓ …

1.3 深度学习模型

■ 深度学习小结



- 模拟视觉皮层机制逐层特征抽象
- 多隐藏层通用近似学习
- 一种端到端学习过程
- 主流模型：卷积/循环/生成…
- 主要优化技巧trick

深度学习

1.1 人工神经网络

1.2 深度学习原理

1.3 深度学习模型

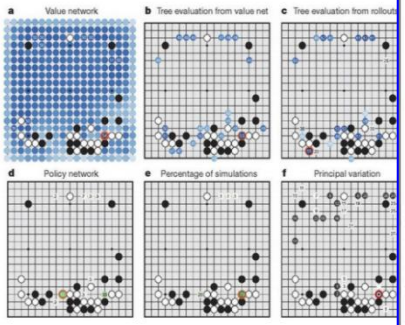
1.4 深度学习应用

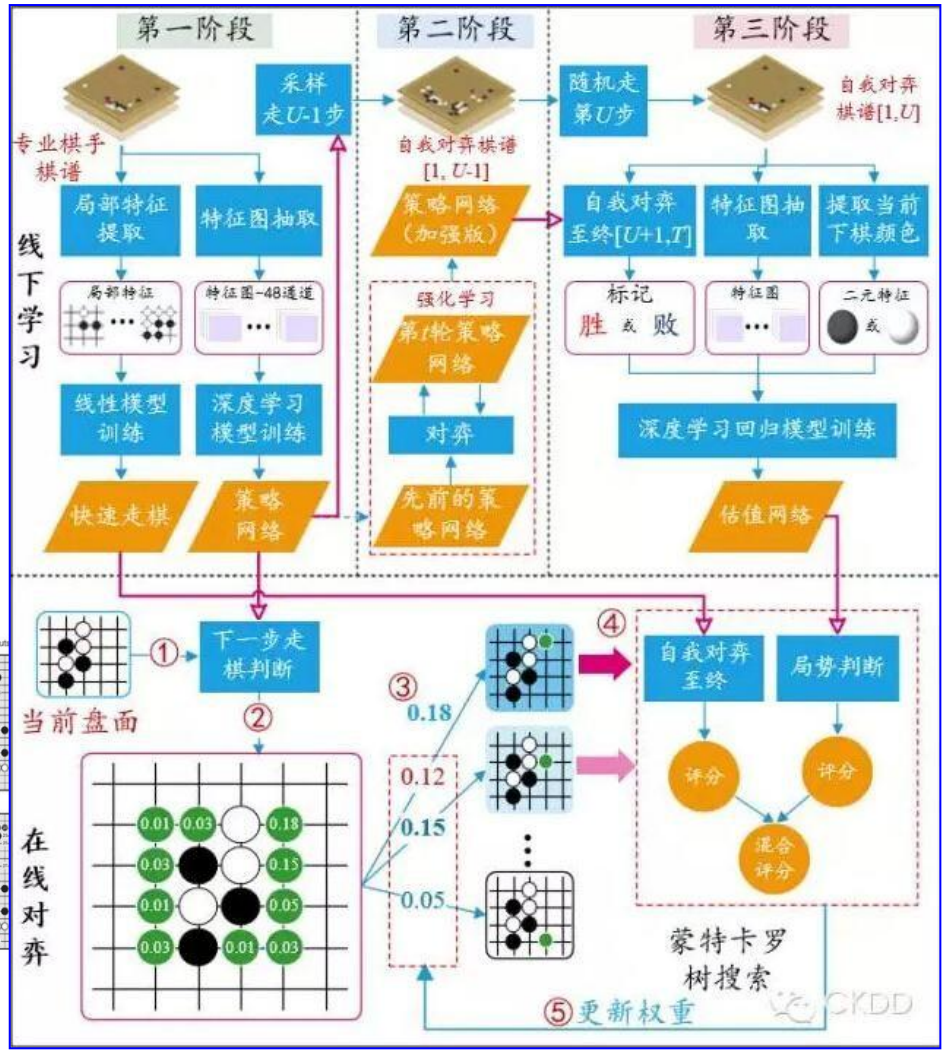
1.4 深度学习应用

■ 下棋：AlphaGo

- ✓ 使用 64 个 GPU 和 19 个CPU，AlphaGo Zero 用三天时间完成了自我对弈490 万局。
- ✓ 几天之内它就发展出击败人类顶尖棋手的技能。



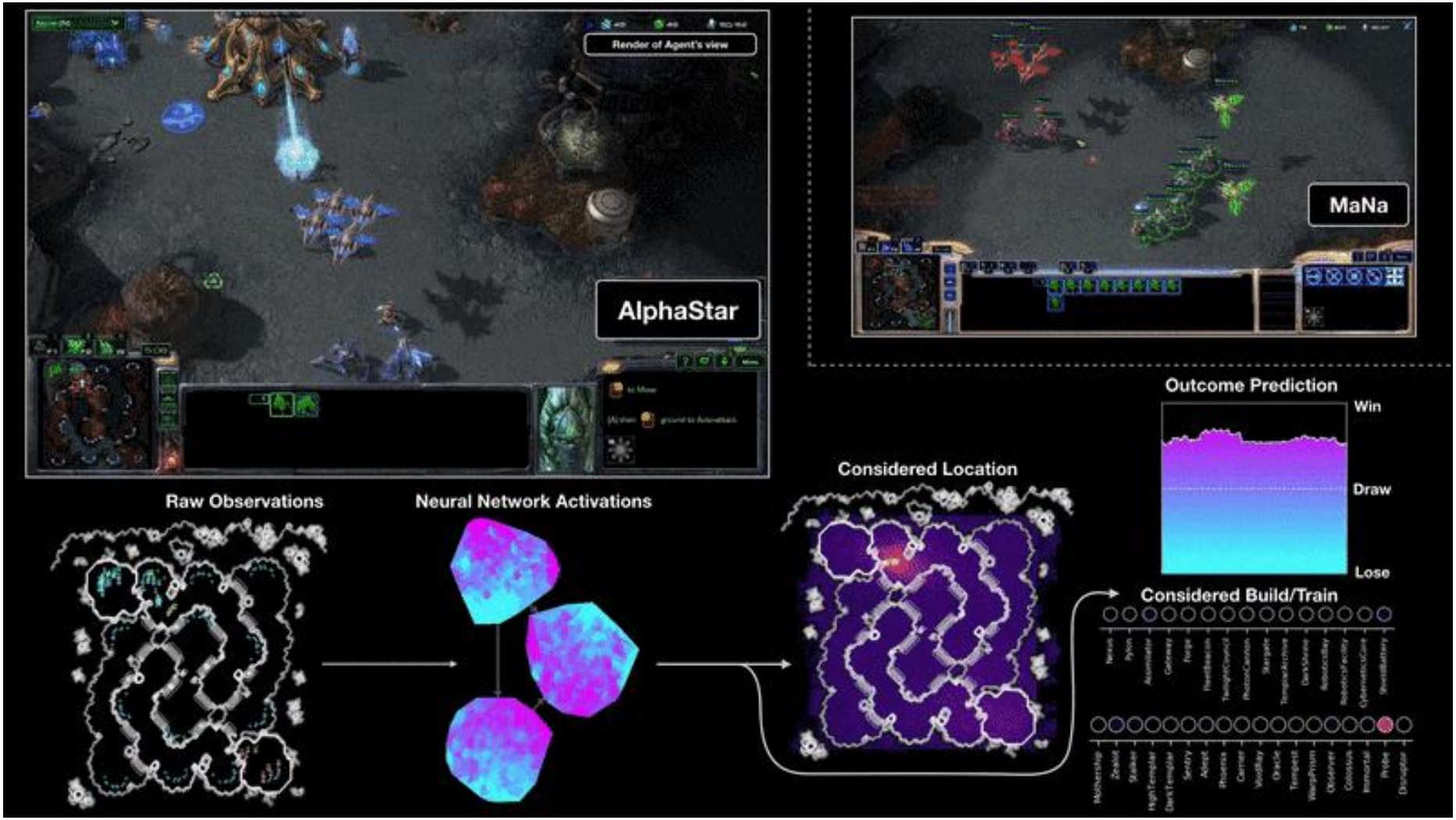




policy network:
[19x19x48] Input
CONV1: 192 5x5 filters , stride 1, pad 2 => [19x19x192]
CONV2..12: 192 3x3 filters, stride 1, pad 1 => [19x19x192]
CONV: 1 1x1 filter, stride 1, pad 0 => [19x19] *(probability map of promising moves)*

1.4 深度学习应用

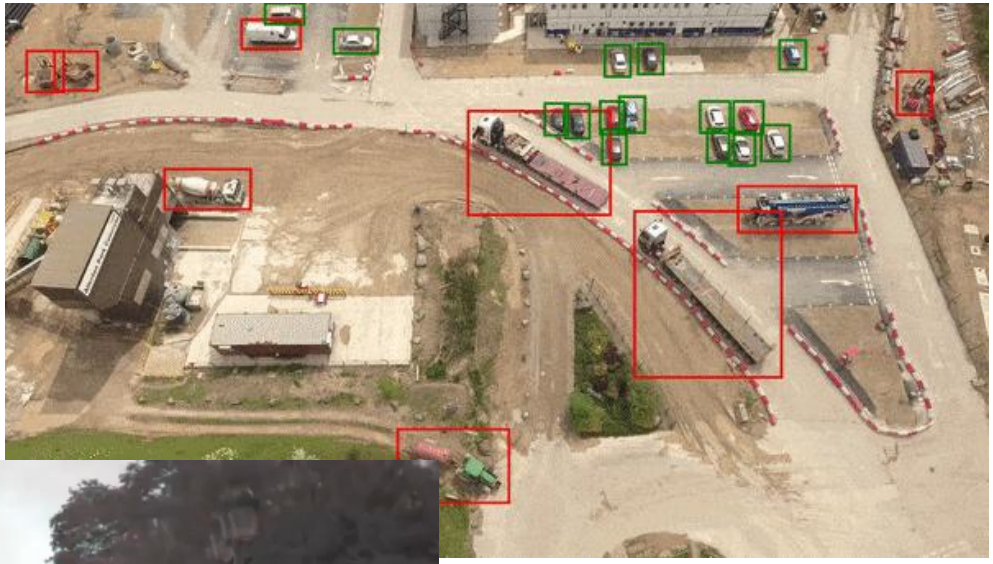
- 游戏：星际大战



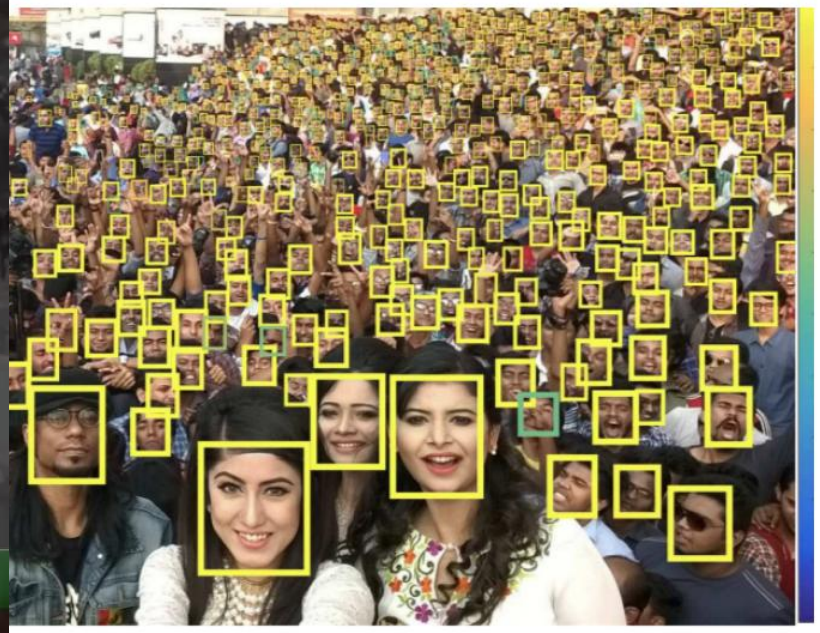
https://deepmind.com/blog/alphastar-mastering-real-time-strategy-game-starcraft-ii/

1.4 深度学习应用

- **目标识别**
  - ✓ 人脸识别
  - ✓ 车牌识别
  - ✓ 目标检测
  - …

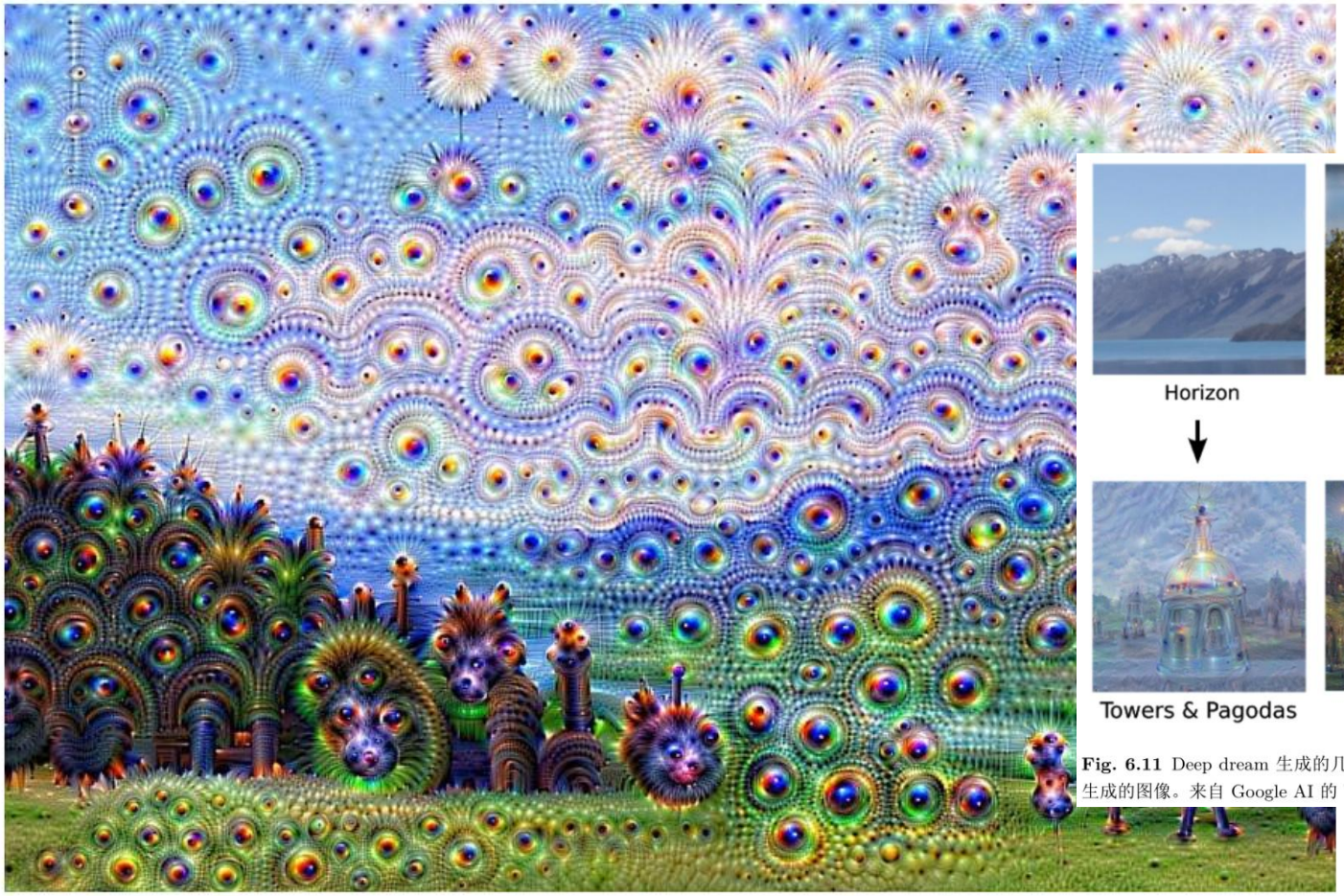






1.4 深度学习应用

- DeepDream



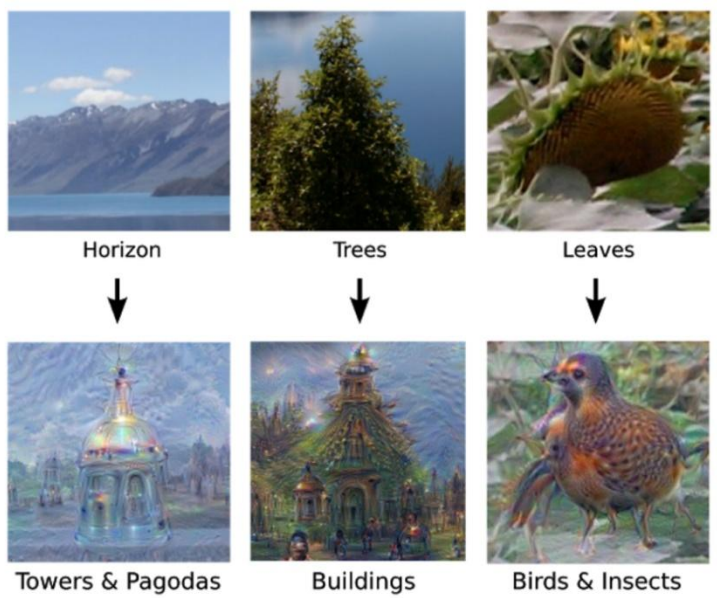
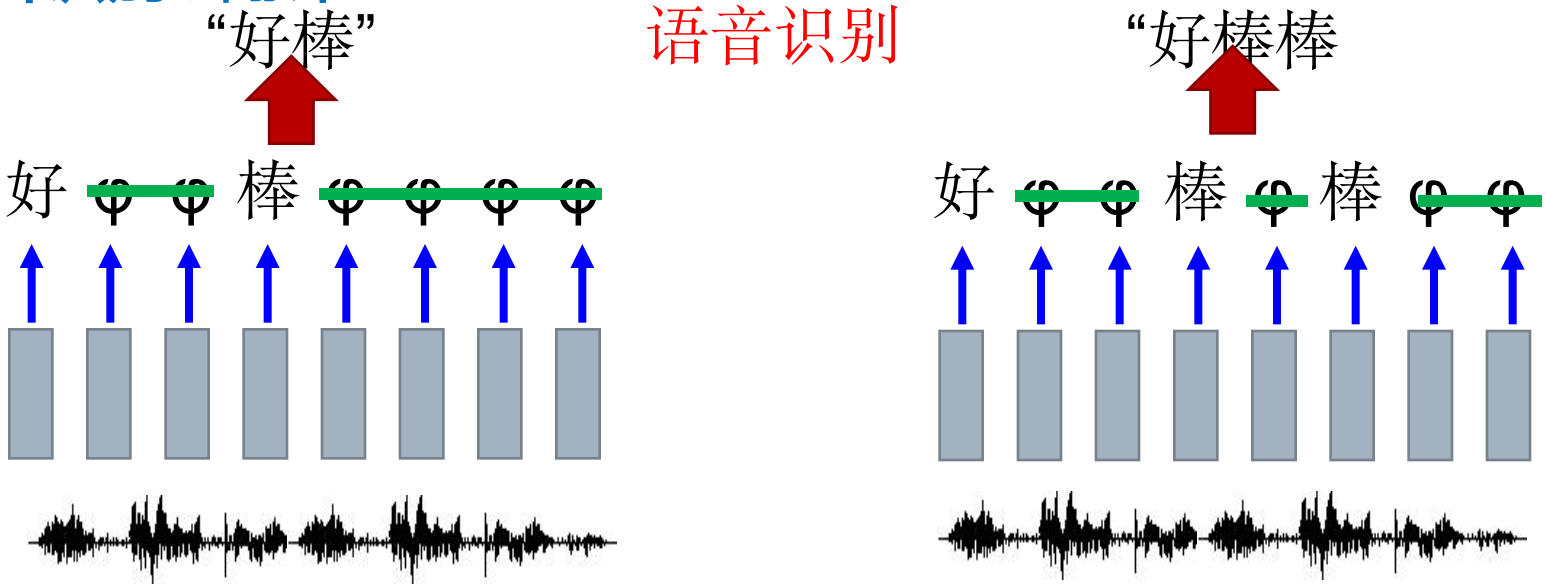


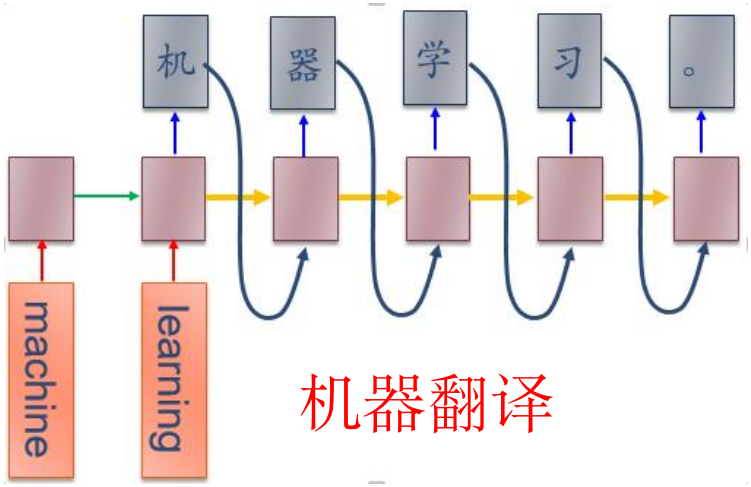
Fig. 6.11 Deep dream 生成的几幅图像。上面一行是原图，下面一行是由 Deep Dream 生成的图像。来自 Google AI 的 blog。

Fig. 6.12 Deep dream 合成的复杂图像，其中局部特征包含一定真实性，整体照片则是合成出来的非现实场景。图片来源于 WIRED。[5]

1.4 深度学习应用

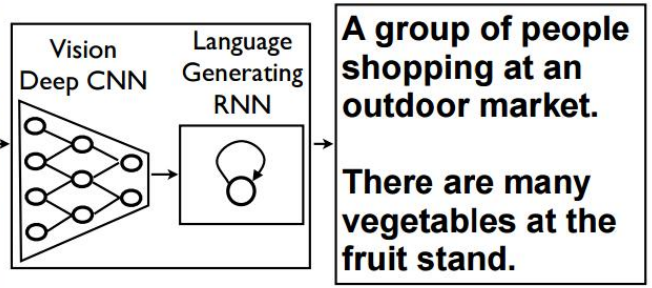
- 语音识别&翻译





机器翻译





看图说话

1.4 深度学习应用

■ 语言生成

```
/*
 * If this error is set, we will need anything right after that BSD.
 */
static void action_new_function(struct s_stat_info *wb)
{
  unsigned long flags;
  int lel_idx_bit = e->edd, *sys & ~((unsigned long) *FIRST_COMPAT);
  buf[0] = 0xFFFFFFFF & (bit << 4);
  min(inc, slist->bytes);
  printk(KERN_WARNING "Memory allocated %02x/%02x, "
    "original MLL instead\n"),
    min(min(multi_run - s->len, max) * num_data_in),
    frame_pos, sz + first_seg);
  div_u64_w(val, inb_p);
  spin_unlock(&disk->queue_lock);
  mutex_unlock(&s->sock->mutex);
  mutex_unlock(&func->mutex);
  return disassemble(info->pending_bh);
}

static void num_serial_settings(struct tty_struct *tty)
{
  if (tty == tty)
    disable_single_st_p(dev);
  pci_disable_spool(port);
```

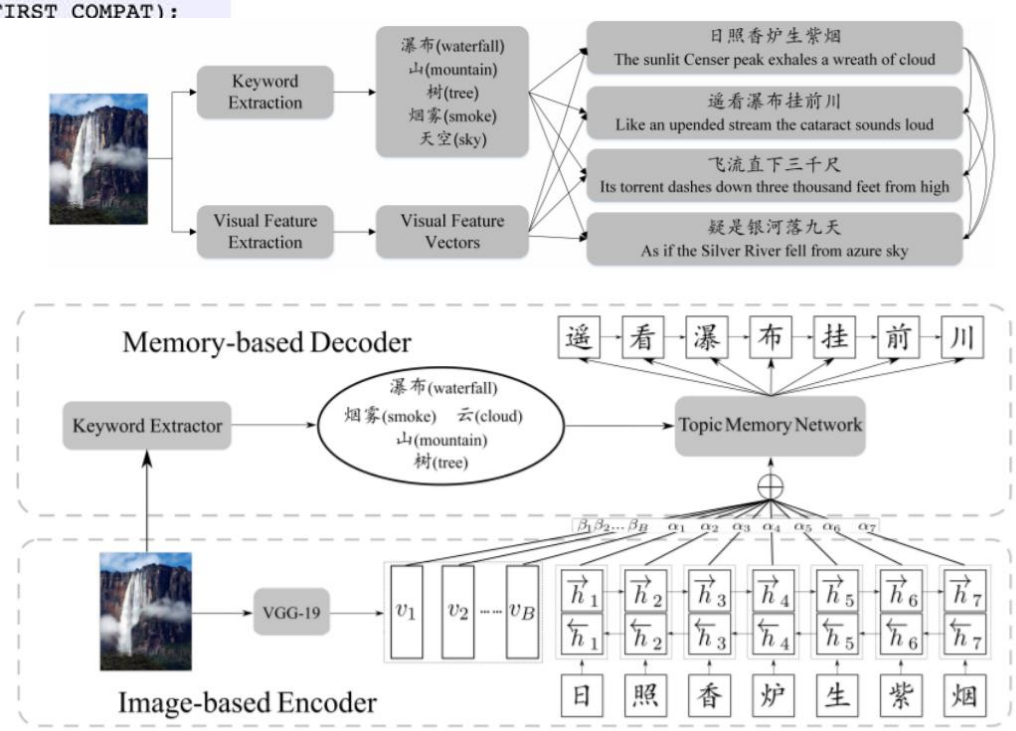


Fig. 6.6 基于图像作诗的神经网络模型 [164]。上图是系统框架。首先由图像得到关键词信息和视觉信息，以这两种信息作为输入，逐句生成整首诗。下图是系统所用的神经网络模型。生成每句诗时，将已经生成的所有句子和视觉信息作为输入，同时参考图像关键词信息。

1.4 深度学习应用

- 行为模拟



机器助手



机器狗



工业机器人



消防机器人



探测机器人



自动驾驶

1.4 深度学习应用

- 军事机器人







1.4 深度学习应用

- LLM应用



1.4 深度学习应用

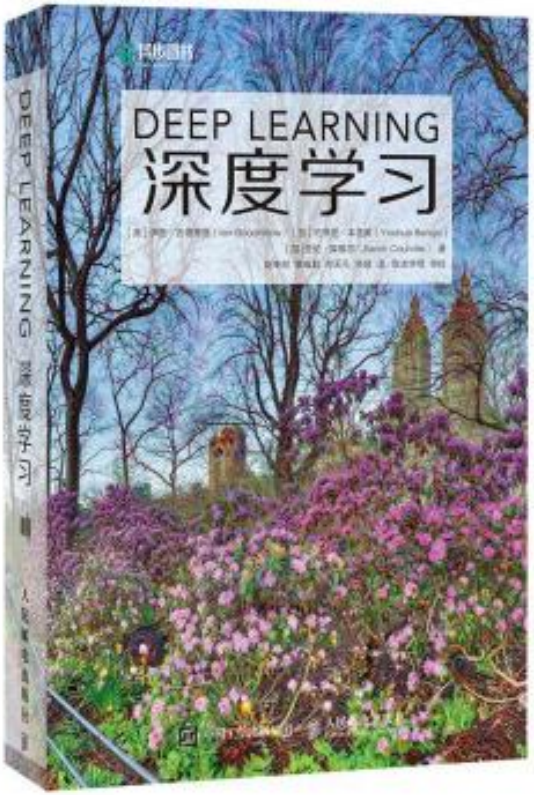
- 应用局限与瓶颈？

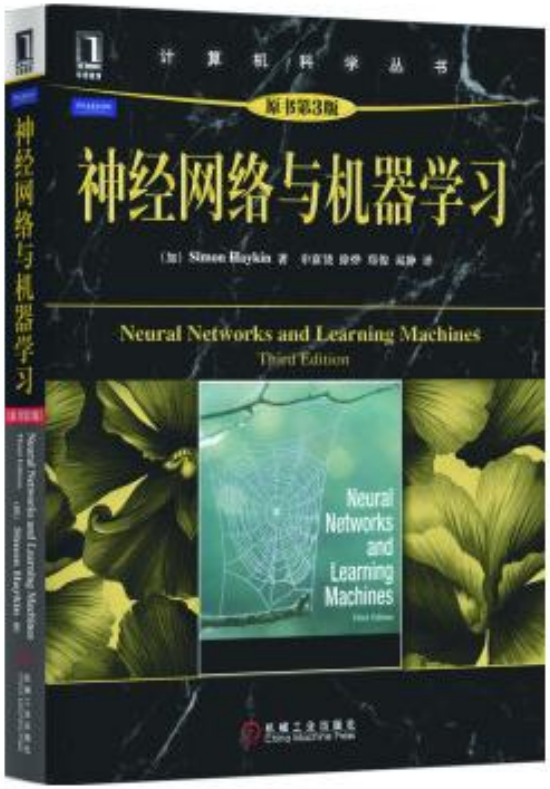
What's wrong with Deep Learning ?

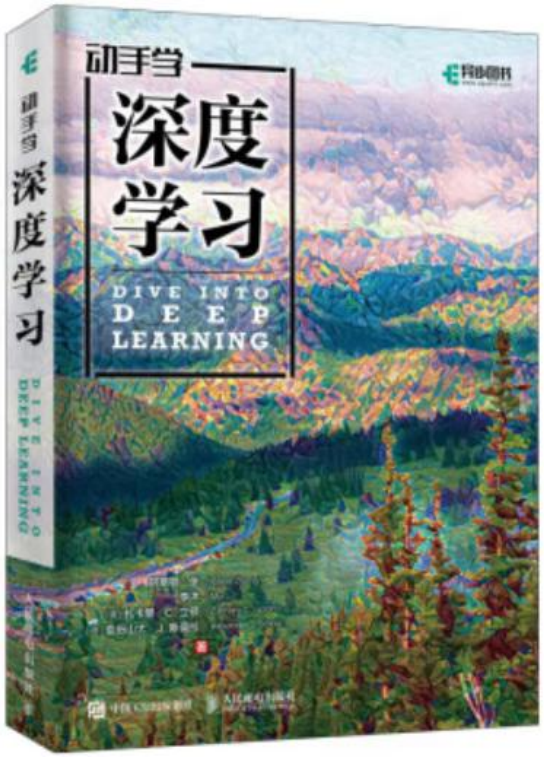
——Yann Lecun

- ✓ 理论支持不足
- ✓ 推理能力不足
- ✓ 无监督学习能力不足
- ✓ 深度神经网络模型复杂，计算量大。
- ✓ …

参考学习资料







http://www.deeplearning.net/tutorial/

课后作业

- 采用Pytorch库设计一个深度神经网络，并实现手写字体识别，并进行实验结果分析；
- 采用Python原生语言实现上述功能，不用三方库。 需要完整实现代码及运行结果截图